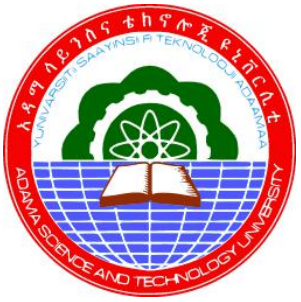


ADAMA SCIENCE AND TECHNOLOGY UNIVERSITY

SCHOOL OF ELECTRICAL ENGINEERING AND COMPUTING

DEPARTMENT OF SOFTWARE ENGINEERING

Smart Entrepreneurial Pitching & Matching System (SEPMS)

For the Partial Fulfilment of the Requirements
for the Degree of Bachelor of
Science in Software Engineering

DECLARATION

We are students of Adama Science and Technology University in the School of Electrical Engineering and Computing in the Department of Computer Science and Engineering. The information found in this project is our original work. And all sources of materials that will be used for the project work will be fully acknowledged.

No	Student Name	Student ID	Signature
1	Abdi Sileshi	ugr/25299/14	
2	Dagmawi Adane	ugr/26689/14	
3	Eyob Tariku	ugr/25465/14	
4	Miraf Amare	ugr/25308/14	
5	Yohanan solomon	ugr/25476/14	

Date of Submission: December 31, 2025

This project has been submitted for examination with our approval as a university advisor.

Advisor Name Signature

Anteneh Tilaye _____

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

ACRONOMYS

SEPMS	Smart Entrepreneurial Pitching and Matching System
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
ASR	Automatic Speech Recognition
TTS	Text to Speech
API	Application Programming Interface
DB	Database
KYC	Know Your Customer
IP	Intellectual Property
UI	User Interface
UX	User Experience
CRUD	Create, Read, Update, Delete
CICD	Continuous Integration and Continuous Deployment
JSON	Javascript Object Notation

Table 1: Acronyms

ACKNOWLEDGEMENT

We would like to thank our advisor, Anteneh Tilaye, for the support and guidance given to us throughout this project. Their comments and suggestions helped us improve the quality of our work. We are also thankful to the School of Electrical and Computing at Adama Science and Technology University for providing us with the facilities and an encouraging learning environment. We appreciate the effort and cooperation of our team members as we worked on this project. Finally, we thank our families and friends for their constant encouragement and for supporting us during the entire process.

ABSTRACT

This project presents a system that helps entrepreneurs submit clear and organized project pitches and helps investors find projects that match their interests. The system guides users through a structured form, checks the completeness of the information, and uses simple machine learning models to organize and score submissions. It also provides a matching feature that recommends projects to investors based on their preferences. The platform supports different languages and includes voice features to make it easier to use. The project is built using a web interface, a mobile application, a backend server, and a database that stores user information.AI components are implemented in a separate Python service communicating with the Node.js backend. We test the system by checking the accuracy of the matching method and by collecting feedback from users. The goal of this project is to make the pitching and investment process easier, more transparent, and more accessible for both entrepreneurs and investors.

Chapter 1: Introduction

1.1. Introduction

Entrepreneurial ventures often face significant barriers when seeking investment, particularly in contexts where formalized investment networks are scarce or fragmented. Although many digital platforms aim to connect business founders and potential investors, a number of these solutions tend to focus on listing services without systematically supporting submission quality, guided evaluation, or investor decision support(Shane, 2003). The Smart Entrepreneurial Pitching & Matching System Platform aims to address these gaps by offering a structured, user-centered environment in which automated tools and human judgement work together to improve match quality and reduce information asymmetry.

This platform is intended primarily for entrepreneurs who are legally registered and operate functioning businesses. It does not target raw ideation-stage concepts for the prototype phase. The rationale for this restriction is pragmatic: relying on formally registered entities reduces intellectual property uncertainty and increases the reliability of automated evaluation. The system is therefore designed to augment rather than to supplant human evaluators. Features such as guided submission templates, automated screening, semantic embeddings for similarity search, and investor-facing recommendation summaries are meant to lower cognitive load for investors and to raise the baseline quality of submissions.

Motivation for the platform also draws on the observable benefits of structured pitching formats that appear in televised investor programs. Those formats show how disciplined presentation and standardised evaluation can reveal viable ventures efficiently. They are, however, limited in scale and accessibility. The present platform seeks to replicate the structure of those approaches in a digital form that would be available continuously and across languages. To this end, the prototype integrates conversational guidance for entrepreneurs, multilingual and voice-enabled investor explanations, and an embedding-driven matching pipeline that together may broaden participation in early-stage funding opportunities(Hsu, 2007).

From a design perspective, machine intelligence is positioned as a supportive agent.For example, automated checks might flag incomplete submissions or low quality fields while an

explainable classifier could indicate why certain proposals were deprioritised. Final authority over account approvals, meeting confirmations, and investment commitments resides with human actors. This hybrid stance aims to capture the benefits of automation, such as consistency and scale, while preserving the contextual sensitivity that experienced investors bring to complex decisions (Brynjolfsson & McAfee, 2014).

1.2. Background of the Project

Traditionally, connecting entrepreneurs with investors depended on social capital, incubators and episodic pitch events. The rise of digital platforms, including AngelList and SeedInvest, sought to scale these connections by offering project listings and investor discovery tools. Many of those platforms accept largely unstructured submissions and therefore require substantial human effort to evaluate opportunities, producing bottlenecks and inconsistent outcomes (Kuratko, 2016). In contexts where formal business registration and patenting are uneven, the challenge is compounded; entrepreneurs may find it difficult to show credibility and investors face higher uncertainty.

In the Ethiopian context, these difficulties are intensified by uneven registration practices, limited access to patent documentation, and variable financial literacy. Such conditions can reduce platform effectiveness and discourage participation by both entrepreneurs and investors (Gebrehiwot, 2015). A design that emphasises structured template, practical KYC checks, and clear guidance for document uploads may therefore be particularly valuable in such settings.

Recent advances in natural language processing and lightweight machine learning make practical interventions more feasible than previously. For this prototype, semantic embeddings produced by compact sentence-transformer models such as all-MiniLM-L6-v2 can support similarity search and clustering without large cloud costs. Vector indices stored in a specialised database are likely to speed up recommendation routines, while a small scikit-learn classifier may be used to detect low-quality or spammy submissions. Voice technologies that rely on local transcription and open-source text-to-speach engines such as Coqui could add accessibility without heavy reliance on commercial services. The proposed

stack therefore prioritises cost control and auditability, which can be important for university-led or early-stage deployments.

Scholars have also warned that automated systems can introduce biases and that excessive reliance on algorithmic outputs risks occluding important contextual information (Raisch & Krakowski, 2021). To address these concerns, the proposed design combines automatic scoring with human review, surfaces reasons for model decisions, and keeps sensitive actions such as account approvals and fund transfers under human control.

1.3. Statement of the Problem

Despite the proliferation of online platforms for investor–entrepreneur matching, several persistent entrepreneurs may struggle to present projects in a consistent, investor-friendly format. Submission requirements often vary and feedback is limited. Investors typically face information overload and may lack convenient tools to prioritise proposals. Such an environment tends to reduce deal discovery efficiency and to increase the likelihood of missed opportunities for both parties (Shane, 2003).

Moreover, many platforms do not provide practical mechanisms for basic intellectual property or identity assurance. Without clear registration and documentation checks, entrepreneurs may be more vulnerable to idea misappropriation. Payment and meeting coordination are frequently handled outside the platform, which reduces transparency and complicates milestone-based investment. The present platform therefore proposes several interlocking features intended to reduce these frictions: guided submission templates, automated completeness checks, semantic matching via embeddings, voice-enabled explanations, a sheltered payments workflow for milestone commitments, and an administrative role for oversight. The research questions guiding this work are:

1. How can compact embedding models and lightweight classifiers be combined to assess the eligibility and quality of entrepreneurial project submissions reliably?
2. In what ways can personalised, embedding-driven recommendations improve investor decision-making and reduce time to decision?

3. To what extent do multilingual and voice-enabled interfaces increase accessibility and perceived usefulness for entrepreneurs and investors?

1.4. Objective of the Project

1.4.1. General Objectives

The main objective of this project is to develop an AI-enhanced platform that bridges the gap between entrepreneurs seeking to scale their businesses and investors seeking contextually relevant opportunities, facilitating structured, transparent, and secure interactions between them.

1.4.2. Specific Objectives

1. Collect and document functional and non-functional requirements from entrepreneurs, investors, administrators, and the advisor; produce a prioritized product backlog of user stories and acceptance criteria.
2. Design an AI-assisted submission module that offers guided templates, form validation, and an automated completeness checker.
3. Implement a project evaluation pipeline that combines semantic embeddings with a lightweight classifier (scikit-learn) to flag low-quality or non-compliant submissions.
4. Build a recommendation system that uses vector similarity and investor profiles to produce ranked candidate projects with explainable rationale.
5. Provide multilingual support and voice-enabled investor summaries to increase accessibility.
6. Integrate a simulated payment & milestone workflow and a meeting scheduling flow to support milestone-based commitments.
7. Create an administrative portal for account approvals, monitoring AI flags, and operational analytics.
8. Conduct usability, fairness and validation testing to assess system performance, AI fairness, and end-user satisfaction; incorporate results into iterative improvements.

1.5. Scope and Limitation

1.5.1. Scope of the Project

The project focuses on developing a functional prototype that demonstrates core workflows for entrepreneurs, investors, and administrators. Core features include guided submission templates, document upload and storage, an embedding-based matching system, a classifier for basic quality assessment, voice and multilingual support, an admin dashboard, and simulated payment flows for milestone-based investments. Multimedia assets such as small videos and pitch decks will be stored in a file service and structured data will be persisted in a cloud document store.

1.5.2. Limitations of the Project

The prototype's machine learning components are compact and may not generalise across every business sector. Speech modules and multilingual components are likely to have accuracy limitations. Payment features are implemented in sandbox or simulated modes; live banking integration is out of scope for the academic prototype. Finally, the recommendation logic depends heavily on the quality and diversity of user-provided data and on the initial dataset used for training.

1.6. Deliverables

This project will deliver a complete ecosystem of tools and resources to facilitate seamless interactions between entrepreneurs and investors. This includes both web and mobile applications, AI assisted workflows and administrative tools:

1) Web application(Next.js):
    - A fully functional platform supporting entrepreneurs, investors and administrators with responsive UI,dashboards and workflow management.
2) Mobile Application(Flutter):
    - A cross platform mobile app providing the same functionality as the web version, enabling users to submit proposals, receive summaries and communicate while on the go.

3) AI- Assisted Submission Module: Integrated via external AI aAPIs through Express backend:
    - Guided templates for entrepreneurs
    - Completeness validation
    - Structured formatting of pitches
4) Semantic Matching and recommendation engine:
    - Vector based matching using embedding Apis
    - Mongodb atlas vector index for fast similarity search
    - Personalized investor recommendations
5) Voice-Enabled investor summaries and Multilingual Support:
    - External TTS APIs for audio summaries
    - Multilingual text support for broader accessibility
6) Secure communication and milestone payment simulation:
    - In-platform messaging
    - Milestone based simulated payments
    - Meeting request scheduling
7) Admin Dashboard
    - approve user accounts
    - Monitor flagged submissions
    - Oversee communication and matching process
    - View analytics
8) Documentation
    - Technical system architecture and API documentation
    - Database schema documentation
    - User manuals for web and mobile
    - Testing reports like unit testing, testing and usability testing

1.7. Feasibility Study

The objective of this study is to determine the advantages and disadvantages and practicality of implementing the Smart Entrepreneurial Pitching & Matching System. The study assesses whether the project is worth investing time, effort and resources. This study is divided into three categories: technical , operational and economic feasibility.

1.7.1. Technical Feasibility

- System requirements:
    - The project requires both web and mobile applications to ensure accessibility. The web will serve investors and administrators while mobile applications will cater primarily to entrepreneurs for submission and progress tracking. Integration with external AI APIs will provide semantic matching, voice-enabled summaries and submission quality checks, eliminating the need for in house model training.
- The Data Requirements:
    - Entrepreneurial project submissions, business registration documents and investors profiles form the primary data sources. This structured data can be used to optimize matching and recommendations while ensuring privacy and compliance with data regulations.
- The Human resources:
    - The project team requires software engineers for frontend and backend development, flutter developers for the mobile app, ui/ux designers for an intuitive interface and project coordinators for overseeing workflow and AI API integration.
- System Reliability:
    - by using proven web and mobile frameworks, cloud storage for media and APIs for AI functionality, the system can maintain high availability and responsiveness. This approach reduces development complexity and technical risks.

1.7.2. Operational Feasibility

- The Staffing and Expertise:
    - The system relies on a small, skilled team with expertise in web, mobile and cloud technologies. Role clarity ensures smooth operations, entrepreneurs submit projects, investors review and interact with recommendations and administrators manage oversight and issue resolution.
- Usability and Accessibility:

    - The system is designed to be user friendly for all stakeholders, with guided submission templates, multilingual support and voice enabled summaries. This enhances adoption and reduces learning curves.
- Scalability:
    - The modular design allows the addition of features over time, such as new AI-based evaluation criteria , additional languages, or expanded investors dashboards. This ensures the system can grow according to user demand without major restructuring.
- Maintenance and support:
    - operational processes including monitoring API integrations updating submission templates and managing cloud resources are straightforward making long term maintenance feasible.

1.7.3. Economic Feasibility

- The Development Costs:
    - The project leverages open source frameworks, cloud free tiers and existing AI APIs, minimizing financial investment. The primary costs are small scale hosting for testing and deployment.
- Potential Benefits:
    - The platform can increase efficiency in matching investors with entrepreneurs, reduce missed opportunities and improve submission quality. This can provide significant value to both investors and startups.
- Revenue Potential:
    - Although the immediate goal is an academic prototype, the system could later support revenue streams as premium investor subscriptions, featured entrepreneur projects or milestone-based transaction fees.

1.8. Significance of the Project

The platform provides both practical and scholarly contributions to the entrepreneurship ecosystem.

- Standardization and quality assurance:
    - By providing guided submission templates, automated completeness checks and structured evaluation , the platform improves the consistency and quality of the project.
- Information Accessibility and Efficiency:
    - Semantic matching and recommendation engines help investors identify high potential ventures quickly.
- Transparency and Trust:
    - By integrating secure workflows for milestone based payments and administrative oversight the platform fosters transparency between entrepreneurs and investors.
- Scholarly and Research Contributions:
    - The project demonstrates the practical integration of AI APIs(recommendation systems and voice interfaces) into real world platforms .
- Accessibility through multilingual support and mobile access

1.9. Beneficiaries of the project

- Primary Beneficiaries: Entrepreneurs with registered businesses gain access to structured submission guidance feedback mechanism and potential opportunities. Investors benefit from curated, context-aware recommendations.
- Secondary: Incubators, accelerators, evaluators and researchers studying AI-assisted entrepreneurship

1.10. Methodology

This project adopts an Agile development methodology to guide the systematic planning, design, and preparation of the proposed system. The Agile approach is selected because it emphasizes iterative refinement, structured stakeholder collaboration, and flexibility in responding to evolving requirements. These characteristics are particularly important for complex systems that integrate web platforms, mobile applications, and artificial intelligence services.

In this project, Agile principles are applied to requirements engineering, system design, integration planning, and implementation preparation. Work is organized into logical iterations that allow continuous review and improvement of documentation artifacts. Stakeholder feedback is incorporated regularly to ensure alignment between the planned system and user expectations.

1. Requirements Analysis

Requirements analysis focuses on identifying and documenting both functional and non-functional requirements of the system. Information is gathered through stakeholder interviews, surveys, and analysis of existing platforms with similar objectives. Functional requirements define the core services the system is expected to provide, while non-functional requirements specify performance, security, usability, and scalability constraints.

All requirements are structured as user stories with clearly defined acceptance criteria. These user stories are organized into a prioritized product backlog, which serves as the primary reference for system planning. This Agile-oriented approach ensures that the proposed system is user centered, traceable, and adaptable to feedback.

2. System Design

System design involves the development of a comprehensive architectural blueprint for the proposed platform. Architecture diagrams are created to describe the interaction between the frontend, backend services, external application programming interfaces, and MongoDB data storage. The design follows a modular structure to support scalability, maintainability, and role-based access control.

Design decisions are documented and refined iteratively. Emphasis is placed on separation of concerns, clear data flow, and extensibility to accommodate future enhancements. These design artifacts provide a clear and structured foundation for subsequent system implementation.

3. API Integration Planning

API integration planning identifies the third-party services and artificial intelligence components required by the system. This includes services for document processing, recommendation logic, voice summaries, and payment workflows. For each external service, integration requirements, data exchange formats, authentication mechanisms, and security considerations are documented.

RESTful API interaction plans are defined using Express as the backend framework. This planning phase ensures that all external dependencies are clearly understood and that integration risks are minimized before development begins.

4. Implementation Planning

Implementation planning defines how the system will be developed using incremental Agile iterations. The planned system includes a web-based frontend, a mobile application with feature parity, and backend services responsible for data management, API handling, recommendation processes, and transaction workflows.

Development tasks are decomposed into manageable Agile backlog items. Each item includes scope definitions, dependencies, and acceptance criteria. This structured planning approach supports efficient execution and continuous validation during the development phase.

5. Integration Planning

Integration planning focuses on defining how individual system components will interact as a unified platform. This includes the coordination between submission modules, recommendation engines, voice summary services, administrative tools, and payment-related workflows.

Data flow diagrams and interface definitions are used to document integration points and dependencies. This planning reduces the likelihood of integration conflicts and supports smooth system assembly during implementation.

6. Testing Strategy

A comprehensive testing strategy is defined to guide quality assurance activities during development. The strategy includes plans for unit testing of individual components, integration testing to verify communication between system modules and external services, and usability testing to evaluate user interaction and workflow efficiency.

Testing objectives, success criteria, and evaluation methods are documented to ensure that system quality, reliability, and usability can be systematically assessed once implementation begins.

7. Documentation

Extensive technical documentation is produced and maintained throughout the project. Documentation includes system requirements specifications, architectural and design diagrams, API integration plans, database schemas, testing strategies, and development guidelines.

These documents collectively serve as the authoritative reference for system implementation, evaluation, and future maintenance. They ensure continuity, knowledge transfer, and alignment with Agile development principles.

1.11. Development tools

We selected tools that we are already familiar with and that match the requirements of the system. Since the platform supports multiple user types as well as AI and voice-based features, we needed a flexible and affordable technology stack that is realistic for a student project.

On the frontend, the system is designed to use Next.js and React. Next.js is chosen for its server-side rendering capabilities, which improve performance and make the application feel more stable. It also simplifies routing and allows us to maintain separate pages for entrepreneurs, investors, and administrators without unnecessary complexity. React enables extensive component reuse, particularly for features such as project submission and profile management forms. For styling, Tailwind CSS is selected to avoid maintaining large CSS files, and shadcn UI is used to provide ready-made components such as buttons, inputs, and cards. This approach helps achieve a professional-looking interface without requiring a dedicated UI designer.

On the backend, Node.js and Express are selected because they offer a simple setup, rapid development, and a large ecosystem of supporting libraries. Using Express, the backend is planned to expose API routes for project submission, investor recommendations, payment tracking, and administrative approvals. Middleware is used to handle authentication, request validation, and file uploads. Firebase Authentication is chosen for user login and registration, as it supports email, phone, and Google sign-in, providing multiple authentication options while removing the need to implement password hashing and security mechanisms from scratch.

For data storage, MongoDB Atlas is chosen. MongoDB fits well with the system's data model because entrepreneur submissions may contain a wide range of fields, including files, patent numbers, and optional metadata that do not always follow a fixed structure. Investor profiles also include dynamic information such as preferred sectors, viewed project history, and feedback. A document-oriented database is therefore more suitable than a rigid relational schema.

Although the core backend is built using Node.js, a separate Python-based service is planned using FastAPI to handle AI-related processing. This service allows the use of native AI libraries such as scikit-learn, sentence transformers, and Coqui for text-to-speech as a fallback option. The AI service is designed to communicate with the main backend through RESTful API calls. Sentence transformer models such as all-MiniLM are selected because they are lightweight, fast, and freely available. The generated embeddings are used to capture semantic meaning and match investor interests with project descriptions. In addition, a small

scikit-learn classifier is planned to help flag low-quality submissions by detecting missing fields, extremely short descriptions, or spam-like content.

For voice functionality, the system is designed to convert investor speech into text so that AI models can process spoken requests. It also supports generating natural-sounding audio to read project summaries aloud to investors. All AI and voice components rely on open-source, budget-friendly solutions appropriate for an academic project. Uploaded images and videos are planned to be stored using Cloudinary, and GitHub is used for version control and collaboration throughout development.

1.12. Required resources with cost

The main required resources for developing the platform were our personal laptops, stable internet connection and access to free cloud service accounts. MongoDB Atlas and Firebase both offer free tiers, so we do not spend money for hosting. Also Cloudinary has a free plan for small storage which is enough for testing.

Most of the AI tools we used are open source and can run locally. This means we avoided expensive API calls. The only cost we faced is the time each team member spent to understand how to connect all these components.

In terms of human resources, we needed five active members. One member leads the group and coordinates meetings. One focuses on backend APIs, one on frontend, one on database structure and one on documentation and testing. Sometimes tasks overlap, especially when connecting AI functions to frontend pages.

We also needed some support from our advisor to check if our requirements are meaningful and if our system follows university standards. As a student project, the main cost is really effort and time rather than money.

1.13. Task and Schedule

Phase I — Documentation & Planning Sprints

Sprint	Dates	Objective	Key tasks	Deliverables	Owners
Sprint 1	Nov 24 – Dec 7	Requirements discovery	Conduct stakeholder interviews and surveys, analyse existing platforms, create and prioritise user stories, define acceptance criteria.	Prioritised product backlog, interview notes, initial user stories with acceptance criteria.	Project Manager; System Analyst
Sprint 2	Dec 8 – Dec 21	Architecture and integration planning	Produce system architecture diagrams, define MongoDB schemas, identify major modules, evaluate third-party AI and media services, document API contracts and security considerations.	Architecture diagrams, data model draft, API integration plan, security checklist.	System Designer; Backend Lead; AI Lead
Sprint 3	Dec 22 – Jan 4	UI, testing and documentation consolidation	Create wireframes and user flows, define testing strategy and test-cases, consolidate SRS and all design artifacts, prepare defence/walkthrough materials.	UI wireframes, test strategy and test-cases, consolidated documentation package, presentation slides.	UI/UX Lead; Testing Lead; Documentation owner

Table 2 : Phase I — Documentation & Planning Sprints

Phase II — Planned Development Sprints (Agile, two-week sprints)

Sprint	Dates	Objective	Key tasks	Deliverables	Owners
Sprint 1	Feb 4 – Feb 17	Environment and CI setup	Setup repositories, development/staging environments, CI pipeline, basic auth scaffolding and DB connectivity.	Repo with CI, environment setup checklist, basic auth and DB connection.	DevOps / Backend Lead
Sprint 2	Feb 18 – Mar 3	Core submission feature	Implement guided submission UI, CRUD endpoints, draft save, basic validation rules.	End-to-end submission flow (web), API endpoints, acceptance tests.	Frontend Lead; Backend Lead
Sprint 3	Mar 4 – Mar 17	Document handling and pre-validation	Design upload flow, integrate chunked upload and Cloudinary plan, define OCR and pre-validation rules.	Document upload module spec, pre-validation tests, storage integration plan.	Backend Lead; AI Lead
Sprint 4	Mar 18 – Mar 31	Embeddings and recommendation prototype	Define embedding pipeline, implement embedding service prototype, design vector	Embedding service spec, prototype similarity	AI Lead; Backend Lead

			index and similarity queries.	queries, evaluation notes.	
Sprint 5	Apr 1 – Apr 14	Investor UI and admin workflows	Design investor dashboard, admin flagged-queue, explainable scoring UI, and recommendation display.	Dashboard wireframes, admin queue spec, UI acceptance criteria.	Frontend Lead; UI/UX Lead
Sprint 6	Apr 15 – Apr 28	Multilingual and voice planning	Define TTS/ASR integration, text localization strategy, audio summary requirements and fallback modes.	TTS/ASR integration plan, localization mapping, audio summary prototypes.	AI Lead; Frontend Lead
Sprint 7	Apr 29 – May 12	Payments and security	Specify milestone payment simulation, credential management, RBAC enforcement, and audit logging.	Milestone payment spec, security checklist, RBAC rules and audit plan.	Backend Lead; DevOps
Sprint 8	May 13 – May 26	Testing, evaluation and adjustments	Finalise unit and integration test suites, conduct AI fairness checks and usability	Test reports, usability feedback log,	Testing Lead; AI Lead; All devs

			sessions, fix critical issues found in tests.	resolved critical defects.	
Release week	May 27 – May 30	Release prep and handover	Prepare demo environment, finalise deployment notes and user manuals, prepare final demonstration.	Deployed demo (staging), final documentation and demo script.	All leads

Table 3 : Phase II — Planned Development Sprints (Agile, two-week sprints)

1.14. Team Composition

Our project team is made of five members, and we tried to divide the roles based on each person's skill and what they are more comfortable with. We also worked together on many shared tasks like development and documentation. The tables below show our main roles and the responsibilities each member handles.

Table 4: General System Roles

Role	Assigned Member(s)	Responsibilities
Project Manager	Abdi Sileshi	Manages the overall project, coordinates the team, checks progress, and makes sure the work meets the requirements.
System Analyst	Yohanan Solomon	Works on understanding the system needs, gathers requirements, and translates user needs into clear technical points.
System Designer	Miraf Amare	Designs the system structure, database schema, architecture and data flow of the whole platform.

System Testing & Evaluation	Dagmawi Adane	Handles the main testing work, checks system functions, and performs integration and reliability testing.
UI/UX Reviewer & Mobile System Designer	Eyob Tariku	Reviews the UI and UX of the system, checks user flow, and also designs the mobile app version and its layout.
System Developers	Entire Team	All team members help in coding, including frontend, backend, mobile app, AI module and system integration.

Table 5:Technical Responsibilities

Task	Assigned Member(s)	Responsibilities
MobileApp Development	Eyob Tariku	Works on the mobile app using Flutter, handles UI design, state management and API integration.
Frontend Development	Abdi Sileshi, Yohanan Solomon	Builds the frontend using Next.js, designs UI with TailwindCSS, and ensures the pages are responsive and easy to use.
Backend Development	Abdi Sileshi, Miraf Amare	Develops Node.js APIs, handles authentication, manages database operations and server logic.
AI & NLP Module Development	Yohanan Solomon, Miraf Amare	Works on the AI models, NLP processing, project completeness checking and connecting the AI parts with the system.
System Testing & Evaluation	Dagmawi Adane	Tests system functions, checks integration between modules and confirms if the features are working correctly.

API Testing & Debugging Engineer	Dagmawi Adane	Tests API responses, finds bugs in backend connections and helps fix integration issues.
Documentation	All Team Members	Every member contributes to the SRS document, UML diagrams, user manuals and final project report.

Chapter 2 :Description of existing system and Literature Review

2.1. Major function of existing system

In our situation, there is actually no proper digital system that supports entrepreneurs and investors in a structured way. Most of the process today is manual. A good example is the TV show called Negadras on ETV. In that show, entrepreneurs come and present their project in front of investors. The input in this manual process is basically the entrepreneur's live pitch, their documents and whatever information they manage to explain during the short time. The investors listen, ask questions and then decide if they want to support the project or not.

The main function of this existing approach is just giving a meeting space where both sides can talk face to face. There is no automatic checking if the project submission is complete, and no guidance that helps the entrepreneur prepare a standard format. Also, the output depends on how well the entrepreneur can explain under pressure, not on a structured evaluation.

Another limitation is that the investors only see a few projects at a time. Since everything is manual, they cannot review a large number of ideas. They rely mostly on their memory and quick judgement. There is no system that recommends projects based on their interest or past investment history.

So the purpose of the existing manual system is mainly to create a pitch moment, but it does not provide support like proper documentation templates, quality checking, multilingual help, or voice explanation. Everything depends on human effort. This is one of the main reasons

why our proposed digital platform is needed, because it tries to bring more structure and guidance for both entrepreneurs and investors.

2.2. Users of current system

Users generally include early-stage and growth-stage entrepreneurs seeking visibility and capital, small business owners aiming to scale operations, investors from angel backers to institutional actors seeking vetted opportunities, incubators and accelerators supporting deal flow. A platform that includes administrative roles may address governance and trust concerns by enabling human review of automated outcomes. Needs are heterogeneous: entrepreneurs frequently require clear templates and constructive feedback while investors prioritize efficient prioritization and trustworthy documentation. To meet these needs, multiple interaction modalities such as text, document upload, and may be necessary.

2.3. Drawback of current system

Current systems commonly suffer from several drawbacks. First , information overload can produce investor fatigue when proposals are unstructured. Second submission variability often leads to inconsistent evaluations outcomes. Third , ai integration is limited on many platforms and automated tools tend to be heuristic or absent. Fourth , accessibility gaps persist since language and literacy differences are not completely addressed. Fifth, trust and verification mechanisms such as IP validation and KYC are often manual or weak. These weaknesses motivate an architecture that includes guided template , automated completeness checks, semantic embeddings for deduplication and matching , multilingual support , and administrative oversight.

2.4. Literature Review

Literature at the intersection of entrepreneurship platforms and AI highlights a number of relevant themes. Natural language processing can structure qualitative inputs and identify missing in consistent information, which support scalability qualitative inputs and identify missing or inconsistent information, which supports scalability and reduces manual burden (Huang et a; ., 2018). Recommender systems adapted from e-commerce contexts can be applied to investment platform to filter and rank opportunities ; hybrid methods that combine content-based and collaborative signals may be especially useful(Bennet & Lanning, 2007) conversational AI and voice interface can increase accessibility for users with limited literacy

or those who prefer spoken summaries, though the literature caution that voice and multilingual solutions must be evaluated for accuracy and fairness (Raisch & Krakowski, 2021).

Technical advances in compact embedding models and vector indices make semantic similarity practical without high running costs. Small sentence-transformer models can produce embeddings suitable for near real-time matching while vector databases allow scalable nearest-neighbour search. Simpler classifiers implemented with scikit-learn can assist with spam detection and low-quality filtering on small datasets. Open-source technologies present feasible alternatives to commercial speech APIs, but they typically require careful tuning and validation.

A critical perspective emphasizes the risks of algorithmic bias and the danger of over-reliance on automated recommendation. Researchers therefore recommend hybrid systems which AI support human judgment and where model outputs are interpretable and auditable (Brynjolfsson & MACAfee, 2014) . The prototype outlined the work seeks to operationalize that guidance by pairing automated screening and semantic matching with administrative oversight, transparent scoring cues, and human-in-the-loop approval process.

Taken together, the reviews literature supports the value of integrating structured guidance, automated evaluation and accessible interface into investment platforms. It also warns that design choices must pay careful attention to fairness, transparency and the limitation of small-scale models. The prototype described in this study aims to balance practical constraints with these scholarly concerns.

Chapter 3: Proposed System

3.1. Overview

The proposed system, referred to here as the Smart Entrepreneurial Pitching and Matching System (SEPMS), aims to mediate discovery between entrepreneurs seeking early-stage support and investors looking for opportunities. SEPMS combines structured submission forms, automated content analysis, and profile-driven recommendation to present investors

with prioritized pitches while preserving a human review step for marginal cases. Conceptually, the platform sits at the intersection of information retrieval and decision support: it must both represent entrepreneurial proposals as rich, searchable artefacts and supply ranked candidate matches tailored to investor preferences.

From an architectural perspective, the system adopts a modular web-backend-mobile arrangement. The user interface layer provides role-specific dashboards for entrepreneurs, investors, and administrators. A stateless API layer handles business logic, delegating long-running tasks such as embedding computation and classification to a specialized Python service. This separation ensures that long running tasks like embedding computation and summarization do not block the main application flow. . Persistent storage comprises a document-oriented database for submissions and user profiles, plus a vector index for similarity search. Ancillary services include media storage, notification delivery, and a lightweight simulated payments component to support milestone workflows. The design favors incremental deployment: early releases can operate with synchronous fallbacks for ML services and progressively integrate the full vector-index pipeline as the project matures.

3.2. Functional requirement

The following functional requirements are stated in a manner intended to be testable and actionable. Each requirement is expressed with an acceptance criterion to aid evaluation.

- Registration and authentication
  - Requirement: The system shall permit users to register and authenticate with email/password and at least one federated provider.
  - Acceptance: A new user can create an account, verify identity, and be assigned one of the roles: Entrepreneur, Investor, or Administrator.
- Role-based access and profile management
  - Requirement: The system shall maintain role-specific profiles and enforce access control so that actions are permitted only to appropriate roles.
  - Acceptance: Entrepreneurs can create and edit submissions; investors can view recommended submissions and request meetings; admins can review flagged content.

- Guided pitch submission
    - Requirement: The system shall present a structured form that captures essential pitch elements such as summary, problem statement, proposed solution, team, business model, and key financials. Draft saving must be supported.
    - Acceptance: An entrepreneur may save a draft and later submit a completed pitch; required fields are validated on submission.
- Automated quality and completeness checks
    - Requirement: Upon submission, the system shall produce automated checks that compute a completeness metric and a quality flag via a classifier.
    - Acceptance: Every submitted pitch receives a completeness score and a classifier decision; flagged pitches are routed for admin review.
- Embedding and similarity indexing
    - Requirement: The system shall compute vector embeddings for textual elements of a submission and store them in a vector index to enable semantic similarity queries.
    - Acceptance: The platform can return nearest neighbours for a sample query within the configured latency bounds.
- Recommendation generation
    - Requirement: The system shall generate ranked recommendations for investors by combining profile filters and vector similarity scores.
    - Acceptance: Investor dashboard displays a ranked list of candidate submissions, with an explainable score or rationale.
- Media and document handling
    - Requirement: Users shall be able to upload supporting documents and short pitch videos; media must be stored and retrievable via secure links.
    - Acceptance: Uploaded files are stored and accessible according to the privacy settings chosen by the entrepreneur.

- ❖ Communication and scheduling
    - ➢ Requirement: The platform shall provide an in-app messaging capability and a simple meeting request flow. Calendar integration may be simulated for MVP.
    - ➢ Acceptance: An investor can send a meeting request that the entrepreneur receives as a notification and can accept or decline.
- ❖ Milestone and simulated payment flows
    - ➢ Requirement: The system shall allow creation of milestones for matched projects and support a sandbox deposit and release process.
    - ➢ Acceptance: A milestone can be created, funds simulated as deposited, and funds released upon milestone completion.
- ❖ Administrative oversight and reporting
    - ➢ Requirement: Administrators shall have interfaces to review flagged submissions, approve accounts, and access basic analytics.
    - ➢ Acceptance: Admin can view flagged items, change submission status, and export simple reports.

3.3. Non-functional requirement

Beyond functionality, SEPMS must satisfy quality attributes that shape design decisions. The following items state measurable targets where feasible.

- Performance
  The user-facing pages should render within acceptable time so as not to impede task flow. For example, initial page loads for authenticated users should aim to be under two seconds on a typical mobile network. Recommendation queries that depend on the vector index should aim for sub-second response times under normal load; where that cannot be ensured, the system should provide a progressive disclosure so the user understands any delay.

- Reliability and availability
  The service should be resilient to component failures. An availability objective of roughly 99.5 percent for core APIs is reasonable for an academic project and encourages designing with stateless services and retryable background tasks.

- Scalability
  The architecture must allow horizontal scaling of API servers and independent scaling of storage tiers. Vector indexing should be selectable as a managed or

self-hosted component that can expand as data grows.

- Security
  All communications must use transport layer encryption. Role-based access control must be enforced server-side. Sensitive documents should be stored encrypted at rest or behind authenticated retrieval endpoints. The system should log security-relevant events for audit purposes.

- Privacy and compliance
  The design should permit data deletion and respect user-specified visibility settings. Where personal data is collected, retention policies should be defined so that test or dummy data may be purged periodically.

- Usability
  The submission workflow should be designed to reduce cognitive load; inline help and contextual examples improve form completion rates. Usability testing with representative users should be planned and the system revised in light of feedback.

- Maintainability
  Modularity and clear API contracts are important for ongoing development. A pragmatic CI pipeline with automated tests is recommended to support iterative improvements.

- Graceful degradation
  ML-dependent features should degrade gracefully: if the embedding service is unavailable, the system should still allow manual browsing and fall back to keyword-based search.

3.4. System model

The System Model provides a comprehensive view of the SEPMS, defining its architecture, scope, and operational flow. This model is essential for understanding how the core functionality, as mediated by the AI services, is delivered to the system's three primary actors.

3.4.1. Scenarios

This detailed scenario illustrates the system's end-to-end operational flow, highlighting the interaction between the user interface, the stateless API layer, the asynchronous workers, and the core AI services.

Scenario 01: Entrepreneur Sign Up & Initial Verification (SC-01)

- Context / Condition: A new visitor attempts to join the platform as an Entrepreneur to begin pitching.
- Input: Registration credentials (email/phone, password), OTP/verification code, and scanned KYC documents.
- Event / Action:
    - The user selects the "Sign Up" option and chooses the Entrepreneur role to begin their journey.
    - The user provides their email address or phone number and creates a secure password.
    - The system contacts the External Authentication Provider to send a unique OTP or verification link to the user.
    - The system creates a basic User Profile record and sets the status to "Registered/Unverified."
    - The user is redirected to a restricted "Pre-Verification" dashboard where they are prompted to complete their identity check.
    - The user uploads the required KYC and Business documents as requested by the system.
    - The system performs an immediate check on the file types and sizes to ensure they meet platform standards.
    - The system updates the user's status to "Pending Admin Review" and notifies the staff that a new profile is ready for vetting.
- Output / Outcome: Identity verified via OTP; profile created with "Entrepreneur" role; documents stored securely for review.
- Resulting State: Account set to Pending Admin Review; user restricted to a "Read-Only" dashboard.

Scenario 02: Investor Sign Up & Financial Verification (SC-02)

- Context / Condition: An unauthenticated visitor seeks to register as a Domestic or Foreign Investor.

- Input: Registration credentials, OTP verification code, and financial/KYC standing documents.
- Event / Action:
    - The user starts the process by selecting "Sign Up" and identifying as an Investor.
    - The user enters their contact information and sets a password to secure their account.
    - The system verifies the communication channel by sending a secure OTP or link for the user to confirm.
    - The system establishes a restricted profile and prompts the user to provide proof of their financial standing.
    - The user uploads the necessary accreditation or financial documents to prove they are a qualified investor.
    - The system validates the file integrity and moves the account status to "Pending Admin Review."
    - The administrator accesses the dashboard to review the investor's financial credentials and background.
    - The administrator approves the account, and the system fully unlocks the matching engine features for the user.
- Output / Outcome: Identity verified; profile created; documents validated and queued for Admin review/approval.
- Resulting State: Account set to Verified & Active (after Admin approval); full matching engine access granted.

Scenario 03: Successful User Login (SC-03)

- Context / Condition: A verified user (Entrepreneur, Investor, or Admin) attempts to access their account.
- Input: Registered credentials (Email/Phone and Password) and Authentication Provider confirmation.
- Event / Action: The system validates credentials against the database and identifies the assigned user role.
- Output / Outcome:

    - The user navigates to the login page and enters their verified email or phone number along with their password.
    - The system securely transmits these credentials to the Authentication Provider for a match check.
    - The Authentication Provider confirms the identity and sends a success token back to the system.
    - The system performs a quick internal scan to ensure the user's account is active and not currently suspended.
    - The system retrieves the specific role assigned to the user (Entrepreneur, Investor, or Admin).
    - The system redirects the user to their unique, role-based dashboard to begin their session.
- Resulting State: User status set to Authenticated; role-specific features and data are fully unlocked for the session.

Scenario 04: Password Recovery Workflow (SC-04)

- Context / Condition: A registered user is unable to authenticate and initiates a secure password reset.
- Input: Registered Username (Email/Phone) and a valid Time-based One-Time Password (OTP).
- Event / Action: The system verifies the identity via a second factor and replaces the old hashed password with a new one.
- Output / Outcome:
    - The user clicks the "Forgot Password" link on the login page after losing access to their account.
    - The system asks the user to provide their registered email or phone number to identify the account.
    - The system generates a time-sensitive OTP and sends it to the user's verified communication channel.
    - The user retrieves the code and enters it into the recovery form within the allowed timeframe.
    - The system validates the code and permits the user to enter a brand-new password.

    - The system securely hashes the new password and updates the user's login credentials.
    - The system confirms the reset is successful and sends the user back to the login page to sign in.
- Resulting State: User credentials updated; account remains Verified and ready for login with the new password.

Scenario 05: Full Pitch Creation & Document Submission (SC-05)

- Context / Condition: A verified Entrepreneur initiates the submission of a new project for funding and evaluation.
- Input: Structured project data (Title, Funding, Summary) and a complete set of required business/legal documents.
- Event / Action: The system performs automated file validation, OCR text extraction, and a content completeness check via AI.
- Output / Outcome:
    - The user navigates to the "Start New Pitch" section and follows the guided templates to enter their business vision.
    - The system saves the structured data and creates a new, unique Project Record in the database.
    - The system presents a dynamic checklist of the mandatory business documents required for this specific pitch.
    - The user uploads the requested files, including their License, TIN, and Pitch Deck.
    - The system checks the files for the correct size and type before passing them to the AI engine.
    - The system uses OCR and a Local Classifier to read the documents and confirm they are categorized correctly.
    - The system calculates a Completeness Score to ensure all necessary data points are present.
    - The system sets the final submission status to "Pending Admin Review" for a compliance check.

- Resulting State: Pitch status set to Pending Admin Review; submission locked for final compliance verification before matching.

Scenario 06: Submission Data Validation Failure (SC-06)

- Context / Condition: An Entrepreneur attempts to finalize a pitch submission with incomplete mandatory documentation.
- Input: Partial document set (missing TIN Certificate and Financial Statements).
- Event / Action:
    - The user enters the document upload section but only provides a few of the requested files.
    - The user attempts to finalize the submission by clicking the "Submit" button.
    - The system runs a real-time check against the mandatory document list for that project type.
    - The system identifies that critical items, such as the TIN Certificate, are still missing.
    - The system halts the submission and displays a clear notification detailing exactly which documents are required.
    - The user uploads the missing files to satisfy the system's requirements.
    - The system re-evaluates the checklist and allows the user to successfully complete the submission.
- Output / Outcome: Submission halted; specific error alerts generated; system accepts corrective uploads to satisfy the checklist requirements.
- Resulting State: Pitch remains in Draft/Incomplete state until all mandatory flags are cleared, then transitions to Complete.

Scenario 07: Document Content Mismatch Failure (SC-07)

- Context / Condition: An Entrepreneur uploads a file that matches the permitted extension but contains irrelevant content (e.g., a photo instead of a legal document).
- Input: A non-text or miscategorized file (e.g., product photo) uploaded to a specific document slot (e.g., MoA/AoA).
- Event / Action:
    - The user goes to the specific upload slot intended for their legal "MoA & AoA" documents.

    - The user mistakenly selects and uploads a product photograph instead of the legal PDF.
    - The system accepts the file format but immediately triggers the OCR and AI Classifier to read the content.
    - The system determines that the extracted content is an image and does not contain the expected legal text.
    - The system flags the document status as "Incorrect Upload" and prevents it from being verified.
    - The system alerts the user that the file does not match the required document type for that slot.
    - The user replaces the photo with the correct legal document, which the system then verifies.
- Output / Outcome: System identifies a classification mismatch; rejection alert displayed; user is prompted to upload a file with the correct semantic content.
- Resulting State: Document status set to Incorrect Upload; checklist remains incomplete until the specific content validation passes.

Scenario 08: Document Expiration Failure (SC-08)

- Context / Condition: An Entrepreneur uploads a time-sensitive legal document (e.g., Business License) that has passed its validity period.
- Input: A document containing an expired date; OCR-extracted data identifying the Expiry Date field.
- Event / Action: The system's Rule-Based Logic compares the extracted date against the current system date to verify validity.
- Output / Outcome:
    - The user uploads their Business License to fulfill the legal requirements of their pitch.
    - The system uses the OCR engine to scan the document and locate the specific "Expiry Date" field.
    - The system compares this extracted date against the current calendar date.
    - The system identifies that the license has expired and is no longer legally valid.

    - The system marks the document status as "Expired" and blocks the pitch from proceeding to matching.
    - The system notifies the user that a renewed license must be provided to continue.
    - The user uploads a current, valid license, and the system clears the block after re-verifying the date.
- Resulting State: Pitch remains in Incomplete/Blocked status until a document with a valid, future-dated expiration is verified.

Scenario 09: Document Readability Failure (SC-09)

- Context / Condition: An Entrepreneur uploads a low-quality or blurry scan that prevents the system from reliably extracting data.
- Input: A low-resolution or skewed document; OCR Confidence Score falling below the defined threshold (e.g., < 50%).
- Event / Action:
    - The user uploads a document that is blurry or has been scanned at a very low resolution.
    - The system attempts to extract text from the file using the Optical Character Recognition (OCR) engine.
    - The system receives a low confidence score from the engine, indicating the text is not reliably readable.
    - The system flags the document as "Unreadable" and triggers a notification for the user.
    - The system explains to the user that a clearer, high-resolution scan is required for validation.
    - The user uploads a high-quality replacement file that is easily readable by the system.
    - The system successfully extracts the data and updates the document status to "Verified."
- Output / Outcome: System identifies the file as "Unreadable"; generates a request for a high-quality replacement; triggers a fallback alert for administrative logging.
- Resulting State: Document status set to Unreadable; pitch submission remains locked until a high-confidence scan is uploaded and verified.

Scenario 10: Suspicious Content Detection & Manual Review (SC-10)

- Context / Condition: An Entrepreneur uploads a document containing structural anomalies, digital alterations, or data inconsistencies (potential fraud).
- Input: A digitally edited document or one where extracted data (e.g., Business Name) contradicts existing profile records.
- Event / Action:
    - The user uploads a business document that has been digitally altered or contains inconsistent data fields.
    - The system uses the OCR engine to extract the text and immediately identifies structural anomalies, such as irregular font alignments or suspicious formatting.
    - The system cross-references the extracted text with the user's profile and detects a mismatch in the business name or registration details.
    - The system assigns the document a status of "Flagged - Suspect Content" and halts any further automated processing.
    - The system sends an urgent notification to the Administrator's dashboard, providing a detailed report of the detected anomalies.
    - The administrator reviews the suspicious document alongside the AI's findings to determine the severity of the issue.
    - The administrator decides to either request a clean resubmission for minor errors or reject the account entirely if fraudulent intent is confirmed.
- Output / Outcome: System flags the content as "Suspect"; notifies the Administrator for manual audit; provides an anomaly report for human decision-making.
- Resulting State: Submission status set to Flagged - Pending Investigation; account remains blocked until an Administrator issues a final approval or rejection [FR 8.1].

Scenario 11: Partial Completion and Draft Saving (SC-11)

- Context / Condition: An Entrepreneur begins a pitch submission but lacks certain required information or documents to finish the process in one session.
- Input: Partially completed pitch forms and a subset of mandatory documents (e.g., missing Financial Statements).

- Event / Action:
    - The user begins their pitch submission but realizes they do not have the final financial statements ready for upload.
    - The user attempts to navigate away from the submission interface to gather more data.
    - The system recognizes that the mandatory legal documents are saved but the overall checklist is incomplete.
    - The system prevents the pitch from being submitted for review and informs the user that only fully completed pitches can reach investors.
    - The system automatically saves all current progress and stores the pitch in a "Draft" status on the user's dashboard.
    - The user returns to the platform several days later and reopens the draft to resume exactly where they left off.
    - The user uploads the remaining financial files and clicks the "Finalize" button.
    - The system confirms that the data is now 100% complete and triggers the final evaluation pipeline.
- Output / Outcome: System prevents routing to investors; saves existing data to the database; provides a "Draft" access point on the user dashboard.
- Resulting State: Pitch status set to Draft; submission remains invisible to the AI Matching Engine until the Entrepreneur provides the missing data and clicks "Finalize."

Scenario 12: Large File Handling & Asynchronous Processing (SC-12)

- Context / Condition: An Entrepreneur attempts to upload high-resolution media (e.g., a >50 MB business video) that exceeds standard synchronous processing limits.
- Input: Large binary file; chunked data packets transmitted via external storage API (e.g., Cloudinary).
- Event / Action:
    - The user selects a high-resolution business video or a massive pitch deck file for upload.
    - The system initiates the upload process and provides a real-time progress bar to keep the user informed.

    - The system leverages an external cloud storage integration to handle the file in smaller, manageable segments through a "chunked upload" process.
    - The system successfully completes the transfer and displays a confirmation message to the user while moving the file to a background queue.
    - The system passes the file to an asynchronous worker that performs virus scanning and indexing without slowing down the user's browser.
    - The system updates the project record with the final media URL once the background processing is finished.
- Output / Outcome: File successfully persisted in external storage; virus scan and indexing completed in the background; media URL returned to the database.
- Resulting State: Project record updated with media references; system resources remain optimized for other users while background processing continues.

Scenario 13: Cross-Document Data Conflict (SC-13)

- Context / Condition: An Entrepreneur uploads multiple documents (e.g., TIN Certificate and Business License) that contain overlapping but contradictory data fields.
- Input: OCR-extracted strings from different files (e.g., "Mamo Retail P.L.C." vs. "Mamo Retailers P.L.C.").
- Event / Action:
    - The user uploads a TIN Certificate and a Business License as part of their compliance checklist.
    - The system uses the OCR engine to extract the business name from the first document and the second document independently.
    - The system compares the two strings and detects a conflict, such as "Mamo Retail P.L.C." versus "Mamo Retailers P.L.C."
    - The system flags the document set as "Conflict Detected" and alerts the user that the names do not match across their paperwork.
    - The system routes the submission to the Administrator to determine if the mismatch is a simple typo or a legal inconsistency.
    - The administrator reviews the case and either approves the minor discrepancy or requests official supporting evidence from the user.

- Output / Outcome: System identifies a data mismatch; triggers a "Conflict Detected" alert for the user; automatically routes the case to the Administrator [FR 8.2].
- Resulting State: Submission status set to Conflict/Blocked; pitch is excluded from the matching engine until an Admin manually verifies or clears the discrepancy.

Scenario 14: Admin Override of AI Flag (SC-14)

- Context / Condition: A pitch document is flagged as "Suspect" by the AI, requiring a human expert to resolve a potential false positive.
- Input: Flagged document, AI reasoning notes (e.g., low confidence score), and Administrator credentials.
- Event / Action:
    - The administrator logs into the dashboard and navigates to the queue of documents flagged by the AI as "Suspect."
    - The system displays the specific document alongside the AI's reasoning, such as a low confidence score or minor formatting concern.
    - The administrator performs a manual inspection and determines that the AI has produced a false positive and the document is valid.
    - The administrator selects the "Override AI Decision" option to clear the flag.
    - The system updates the document status to "Verified by Admin" and logs the human justification for the change in the audit trail.
    - The system releases the pitch into the next stage of the matching workflow, ensuring the user is not unfairly blocked by an automated error.
- Output / Outcome: Document status updated to "Verified by Admin"; action and justification recorded in the Audit Log [FR 7.2]; pitch released to the matching engine.
- Resulting State: Pitch status set to Ready for Matching; automated block removed by human authorization.

Scenario 15: Multi-Entity Document Conflict (SC-15)

- Context / Condition: An Entrepreneur submits documents belonging to two distinct legal entities (e.g., Company A and Company B), creating a compliance violation.
- Input: Business License for "Company A" and MoA/AoA for "Company B" uploaded to the same pitch record.

- Event / Action:
    - The user uploads a Business License for "Company A" but mistakenly attaches a Memorandum of Association for "Company B."
    - The system extracts the entity names and identifies that the documents belong to two entirely different legal businesses.
    - The system halts the process and notifies the user that an "Entity Conflict" has been detected.
    - The system asks the user to clarify which specific business this pitch is intended for.
    - The user selects "Company A" to resolve the confusion.
    - The system generates a new, targeted document checklist specifically for "Company A" and prompts the user to replace the mismatched files.
- Output / Outcome: Submission halted; system generates a targeted checklist for the selected entity; conflicting documents are marked for mandatory replacement.
- Resulting State: Pitch status set to Entity Conflict; submission remains locked until all documents are updated to reflect a single, consistent legal entity.

Scenario 16: AI Confidence-Based Fallback (SC-16)

- Context / Condition: The internal, lightweight classifier returns a low-confidence result during document analysis.
- Input: Redacted document text and a confidence score below the defined threshold (e.g., < 75%).
- Event / Action:
    - The system's internal classifier processes a complex financial statement but returns a confidence score below the required threshold.
    - The system automatically redacts all sensitive personal and intellectual property data from the text.
    - The system triggers a call to a more powerful external AI API, such as Gemini, to perform a deeper analysis of the document content.
    - The external AI returns a high-confidence classification, identifying the exact nature of the document.
    - The system merges this external insight with the project record and updates the completeness score.

    - The system logs the API call for cost auditing while ensuring the pitch proceeds with 100% accuracy.
- Output / Outcome: High-accuracy classification data returned; results merged with internal records; API usage logged for cost auditing.
- Resulting State: Document status updated to Verified; data integrity maintained while optimizing processing costs.

Scenario 17: Final Administrative Approval (SC-17)

- Context / Condition: A pitch has cleared all automated AI and technical validation checks and awaits a final human quality gate.
- Input: Fully validated business documents, AI completeness scores, and the pitch executive summary.
- Event / Action:
    - The administrator navigates to the "Pending Submissions" queue and prioritizes projects that have cleared all automated AI checks.
    - The system displays the full project summary, the verified documents, and the final AI completeness score.
    - The administrator conducts a final sanity check on the pitch content and the quality of the executive summary.
    - The administrator clicks the "Approve Submission" button to finalize the process.
    - The system updates the pitch status to "Verified & Active," making it visible to the investor community.
    - The system immediately sends a push notification to the user to celebrate that their pitch is now live.
- Output / Outcome: Audit log updated [FR 7.2]; status changed to "Verified & Active"; push notification sent to the Entrepreneur [FR 9.2].
- Resulting State: Pitch status set to Verified & Active; project is now live and visible in the investor matching pipeline.

Scenario 18: Semantic Analysis and Ranking (SC-18)

- Context / Condition: A pitch is officially active and must be converted into machine-readable data for the recommendation engine.

- Input: Verified pitch text (Problem, Solution, Business Model) and supporting financial data.
- Event / Action:
    - The system triggers an asynchronous worker to begin a deep semantic analysis of the newly approved pitch.
    - The system generates complex mathematical embeddings for the problem statement, solution, and market description.
    - The system runs a final classification check for compliance, spam, and overall relevance.
    - The system calculates quantifiable metrics, including a Relevance Score for investors and a Risk Indicator for the platform.
- Output / Outcome: AI-assisted summary generated [FR 4.3]; semantic data and embeddings stored in the Vector Index for similarity search.
- Resulting State: Pitch record is fully Enriched & Indexed; ready for real-time matching against investor profiles.

Scenario 19: Recommendation Consumption & Voice Output (SC-19)

- Context / Condition: A logged-in Investor seeks personalized project recommendations based on their pre-set preferences.
- Input: Investor profile embeddings and real-time similarity search results from the vector database [FR 4.2].
- Event / Action:
    - The system runs the matching engine by comparing the investor’s set preferences against the active pitch embeddings.
    - The system generates a prioritized list of recommendations and sends a mobile notification to the investor.
    - The investor logs in and views their personalized dashboard, filtered by relevance and risk scores.
    - The investor selects a specific pitch to view the detailed report and match explanation.
    - The investor chooses the "Speech Output" function to listen to the summary in their preferred language.

    - The system uses the Text-to-Speech component to narrate the key findings, allowing the investor to consume the data hands-free.
- Output / Outcome: Personalized dashboard displayed; matching rationale provided via explainability features [FR 3.3]; audio summary generated [FR 5.3].
- Resulting State: Investor is Informed and ready to initiate a connection with the Entrepreneur.

Scenario 20: Connection and Coordination (SC-20)

- Context / Condition: An Investor initiates contact with an Entrepreneur to discuss a potential deal.
- Input: Connection request and availability data from the Investor's integrated calendar.
- Event / Action:
    - The investor reviews the pitch and clicks the "Request Connection" button to open a dialogue.
    - The system establishes a secure, encrypted messaging channel dedicated to that specific investor-entrepreneur pair.
    - The user and the investor exchange messages, which the system translates in real-time if their preferred languages differ.
    - The investor initiates the meeting scheduling tool to move the discussion to a formal call.
    - The system displays the investor's available time slots by syncing with their external calendar.
    - The user selects a suitable time, and the system automatically sends calendar invites and meeting links to both parties.
- Output / Outcome: Encrypted message thread established; automated translation provided if language settings differ; calendar invites dispatched.
- Resulting State: Communication is Secured & Logged; meeting scheduled between both parties.

Scenario 21: Post-Investment Milestone Tracking (SC-21)

- Context / Condition: An investment deal has been finalized, and the parties need to track progress and release funds based on performance.

- Input: Evidence of milestone completion (e.g., prototype docs) and Investor verification.
- Event / Action:
    - The user completes the first major phase of their project, such as finishing a prototype.
    - The user marks the milestone as "Complete" in the tracker and uploads supporting evidence for review.
    - The system notifies the investor that a milestone is ready for inspection.
    - The investor reviews the submitted reports and documentation to verify the progress.
    - The investor clicks the "Verify and Simulate Payment" button to acknowledge the success.
    - The system updates the milestone to "Paid/Verified" and logs a simulated funding release in the transaction history for total transparency.
- Output / Outcome: Milestone status updated to "Paid/Verified"; funding tranche release simulated; activity recorded in the Transaction Log [FR 7.2].
- Resulting State: Both dashboards reflect updated financial progress and project maturity.

Scenario 22: Negative Feedback & AI Learning (SC-22)

- Context / Condition: An Investor reviews a recommended pitch but decides it is not a fit for their current strategy.
- Input: "Not Interested" signal and an optional rejection reason (e.g., "Valuation too High").
- Event / Action:
    - The investor reviews the full details of a recommended pitch on their dashboard but decides it is not a suitable match for their portfolio.
    - The investor clicks the "Not Interested" button to decline the proposal.
    - The system prompts the investor to provide a reason for the rejection, such as the sector being a mismatch or the valuation being too high.
    - The system securely logs the rejection decision along with the timestamp and the specific feedback provided by the investor.

    - The system immediately feeds this negative signal into the AI Matching Engine to update the investor's unique preference profile.
    - The system removes the rejected pitch from the investor’s active list to ensure their dashboard remains clean and relevant.
- Output / Outcome: Investor’s vector profile is updated to refine future matching accuracy; the rejected pitch is removed from their active list.
- Resulting State: Improved recommendation relevance for the Investor; pitch remains active for other potential matches.

Scenario 23: Content Enforcement & Pitch Removal (SC-23)

- Context / Condition: An Administrator identifies a pitch that violates platform policy or contains fraudulent data.
- Input: Violation report, evidence, and Administrative justification.
- Event / Action:
    - The administrator logs into the dashboard and reviews a specific pitch that has been flagged for potential policy violations.
    - The administrator confirms that the content is either fraudulent or severely non-compliant based on the evidence provided.
    - The administrator selects the necessary enforcement action, choosing between a temporary suspension or a permanent removal.
    - The administrator enters a detailed justification for the action to maintain a transparent audit trail.
    - The system updates the pitch status to "Suspended" or "Removed" and immediately locks the record to prevent any further matching.
    - The system sends a formal notification to the entrepreneur explaining the violation and detailing any required corrective steps.
    - The system logs the entire enforcement event in the transaction module for future reference.
- Output / Outcome: Pitch unpublished from the marketplace; audit trail updated with justification; Entrepreneur receive corrective instructions.
- Resulting State: Platform integrity preserved; non-compliant content is neutralized.

Scenario 24: Entrepreneur Account Suspension (SC-24)

- Context / Condition: An Administrator identifies a pattern of suspicious behavior, repeated document fraud, or receives multiple complaints regarding a specific Entrepreneur.
- Input: Evidence of violations (e.g., mismatched business details, flagged AI alerts) and Administrative justification.
- Event / Action:
    - The administrator identifies a pattern of suspicious activity, such as repeated document fraud or verified investor complaints, linked to an entrepreneur.
    - The administrator accesses the Entrepreneur Management section to review all collected evidence and AI alerts.
    - The administrator selects the specific user profile and clicks the "Suspend Account" action.
    - The system executes a security sequence that immediately disables the entrepreneur's login credentials in the authentication provider.
    - The system automatically unpublishes all pitches associated with that account and freezes every active messaging thread.
    - The system logs the suspension details, including the justification and timestamp, in the security audit log.
    - The system sends a notification to the entrepreneur informing them that their access has been restricted pending a full review.
- Output / Outcome: System disables login credentials, unpublishes all active pitches, freezes messaging threads [FR 6.1], and logs the action for audit [FR 7.2].
- Resulting State: Entrepreneur account set to Suspended; all platform activity is immediately halted.

Scenario 25: Investor Account Suspension (SC-25)

- Context / Condition: An Investor is flagged for misconduct, such as harassment, inappropriate behavior, or fraudulent documentation.
- Input: Formal reports from Entrepreneurs or system-monitored policy violations.
- Event / Action:
    - The administrator receives formal reports from multiple entrepreneurs regarding suspicious or inappropriate behavior from a specific investor.

    - The administrator reviews the complaint logs and inspects the secure communication transcripts to verify the misconduct.
    - The administrator confirms a violation of platform terms and selects the "Suspend Investor Account" option.
    - The system immediately blocks the investor's login credentials and removes them from the matching algorithm.
    - The system cancels all future scheduled meetings and unlinks every active messaging thread the investor had with entrepreneurs.
    - The system logs the suspension and sends a notification email to the investor regarding the loss of their privileges.
    - The system alerts all entrepreneurs currently interacting with that investor, advising them of the enforcement action for their protection.
- Output / Outcome: Credentials blocked; future meetings canceled [FR 6.2]; active threads unlinked [FR 6.1]; all affected Entrepreneurs are notified.
- Resulting State: Investor account set to Suspended; platform integrity is maintained by protecting the Entrepreneur community.

Scenario 26: Investor Reports Suspicious Entrepreneur (SC-26)

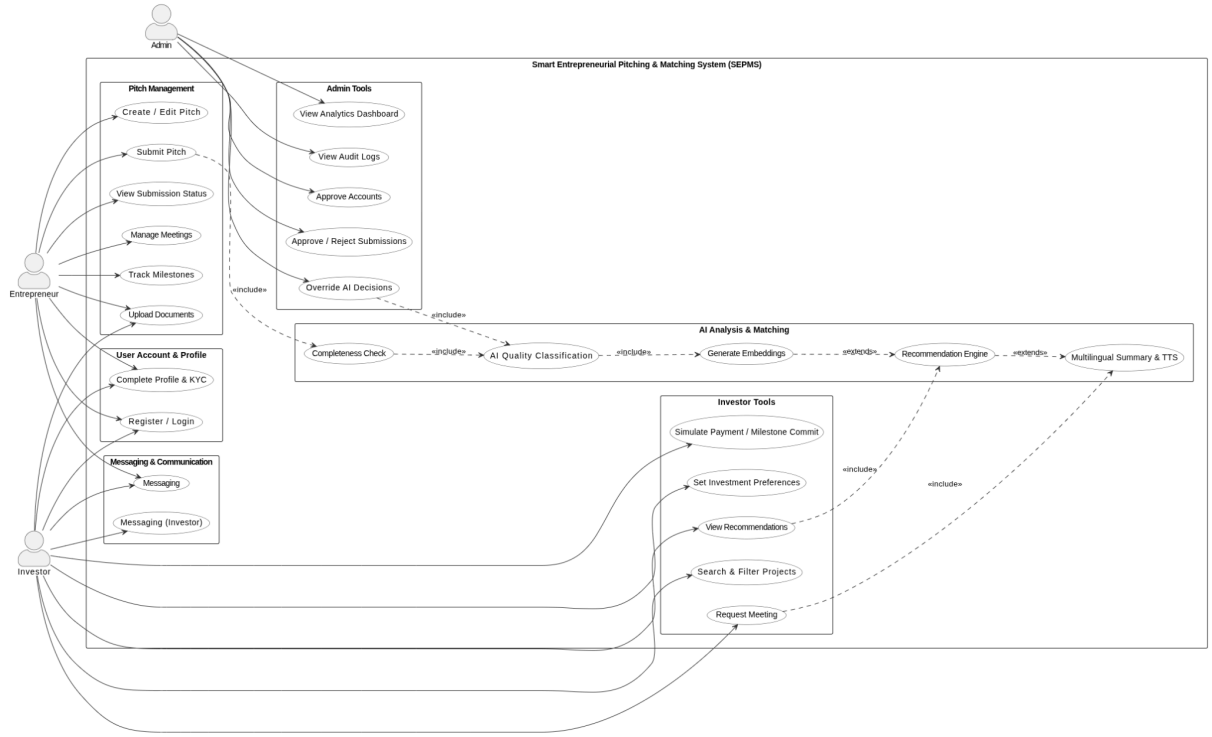
- Context / Condition: During a connection, an investor detects suspicious behavior, such as a demand for fees outside the platform or fraudulent claims.
- Input: "Report Suspicious Activity" trigger within a message thread or pitch view; supporting evidence (screenshots/notes).
- Event / Action:
    - The investor detects suspicious behavior during a conversation, such as the entrepreneur demanding payments outside the platform.
    - The investor navigates to the reporting tool within the messaging thread and clicks "Report Suspicious Activity."
    - The system prompts the investor to provide details and allows them to upload supporting evidence like screenshots.
    - The system securely logs the complaint and triggers an urgent alert for the administrator to investigate.
    - The administrator reviews the reported communication transcripts and the entrepreneur’s historical data.

    - The administrator initiates an internal investigation, which may lead to the pitch being locked or the account being suspended.
- Output / Outcome: Communication transcripts are flagged for review [FR 6.1]; the Entrepreneur's profile is prioritized for audit.
- Resulting State: Incident logged; potential for immediate account locking pending the outcome of the Admin investigation.

Scenario 27: Entrepreneur Reports Suspicious Investor (SC-27)

- Context / Condition: An Entrepreneur receives inappropriate requests or experiences harassment from an Investor.
- Input: "Report Investor Misconduct" selection within the secure messaging interface.
- Event / Action:
    - The entrepreneur receives inappropriate messages or faces harassment from an investor through the secure chat.
    - The entrepreneur utilizes the "Report Investor Misconduct" function directly within the communication interface.
    - The system logs the report and automatically places a temporary freeze on the chat thread to protect the entrepreneur.
    - The system sends an urgent notification to the administrator, flagging the case as a high-priority safety concern.
    - The administrator prioritizes the review of the message transcripts to evaluate the severity of the misconduct.
    - The administrator takes decisive action, such as issuing a formal warning or permanently revoking the investor's platform access.
- Output / Outcome: Immediate protective block established; Admin reviews the interaction history for decisive enforcement action.
- Resulting State: Entrepreneur is protected from further contact; Investor is flagged for potential permanent suspension [SC-25].

3.4.2 Use case model



3.4.2.1 Use Case Description

UseCase 1: Entrepreneur Sign Up & Initial Verification (UC-01)

This scenario covers the first critical step: a new user registering and the process required to move from an anonymous visitor to a verified system actor.

Field	Description
Use Case ID	UC-01 (Entrepreneur Account Creation & Verification)
Primary Actor	Entrepreneur
Supporting Actor	External Services (Authentication Provider)
Goal	For an individual to register and obtain a verified status with Role-Based Access (FR 1.2).
Pre-conditions	User is an unauthenticated visitor to the SEPMS website or mobile app.

Flow of Events (Main Success Path)	
1.	The user selects "Sign Up" and chooses the Entrepreneur role.
2.	The user provides an email or phone number and creates a password.
3.	The system uses the External Authentication Provider to verify the email/phone via an OTP/Link [FR 1.1].
4.	The system creates a basic User Profile record with status "Registered/Unverified."
5.	The user is logged into a restricted "Pre-Verification" dashboard.
6.	Users are immediately prompted to complete the KYC/Document Verification step.
7.	User uploads the required KYC/Business documents (see Section 3).
8.	The system runs automated file validation (type, size).
9.	The system sets the user status to "Pending Admin Review" [FR 1.3].
Outcome	The Entrepreneur's profile is created, the documents are awaiting human review, and they have restricted access.

Table 6 :UseCase 1 - Entrepreneur Sign Up & Initial Verification (UC-01)

UseCase 2: Investor Sign Up & Financial Verification (UC02)

Field	Description
Use case ID	UC-02 (Investor Account Creation & Verification)
Primary Actor	Investor (Domestic or Foreign)

Supporting Actor	External Services (Authentication Provider)
Goal	For an investor to register, prove financial standing, and gain role-based access.
Pre-conditions	The user is an unauthenticated visitor to the SEPMS.
Flow of Events (Main Success Path)	
1.	The user selects "Sign Up" and chooses the Investor role.
2.	The user provides an email or phone number and creates a password.
3.	The system verifies the email/phone via an OTP/Link [FR 1.1].
4.	The system creates a basic User Profile record with status "Registered/Unverified."
5.	The user is logged into a restricted "Pre-Verification" dashboard.
6.	Users are immediately prompted to complete the Financial/KYC Verification step.
7.	The user uploads the required verification documents (see Section 2).
8.	The system runs file validation and sets the user status to "Pending Admin Review" [FR 1.3].
9.	The administrator reviews the financial standing and credentials on the dashboard.
10.	The administrator approves the account.
Outcome	The Investor's profile is created and verified; they can now access the full Recommendation Engine and Matching features.

Table 7: UseCase 3: Successful User Authentication (Login)

This scenario details the basic, successful path for any verified user (Entrepreneur, Investor, or Administrator) to gain access to their role-specific dashboard.

Field	Description
Use case ID	UC-03 (Successful Login)
Primary Actor	User (Entrepreneur, Investor, or Administrator)
Supporting Actor	External Services (Authentication Provider)
Goal	To securely authenticate a user and grant access to the system based on their role.
Pre-conditions	The User has an account with the status "Active/Verified" (Account created and documents approved).
Flow of Events (Main Success Path)	
1.	The user navigates to the SEPMS login page (Web or Mobile).
2.	Users enter their registered Username (Email or Phone) and Password.
3.	The system sends the credentials to the Authentication Provider for verification [FR 1.1].
4.	Authentication Provider confirms the credentials match.
5.	System retrieves the User's role and profile data (e.g., Entrepreneur, Investor, Admin).
6.	System redirects the User to their designated Role-Based Dashboard [FR 1.2].

Outcome	The User is successfully logged in and can access all features relevant to their role.

Table 8: UseCase 4: Password Recovery (Forgot Password)

This scenario details the secure process for a verified user to reset their password when they cannot successfully authenticate. It relies on a second factor (OTP) sent to a verified communication channel.

Field	**Description**
Use case ID	UC-04 (Password Recovery Workflow)
Primary Actor	User (Entrepreneur, Investor, or Administrator)
Supporting Actor	External Services (Notification/Authentication Provider)
Goal	To securely reset a user's password using a verified second factor (OTP) without granting access to the old password.
Pre-conditions	The user has an active account but has forgotten their password.
Flow of Events (Main Success Path)	
1.	**User** navigates to the login page and clicks **"Forgot Password?"**.
2.	**System** prompts the user to enter their registered **Username** (Email or Phone number).
3.	**System** verifies the username exists and retrieves the linked contact information.
4.	**System** generates a **Time-based One-Time Password (OTP)**.

5.	**System** sends the **OTP** via the Notification Provider to the verified email or phone number.
6.	**User** retrieves the OTP and enters it into the recovery form within the time limit.
7.	**System** validates the OTP. If valid, the system prompts the user to enter and confirm a **New Password**.
8.	**System** securely hashes the new password and updates the user's credentials in the Authentication Provider.
9.	**System** displays a confirmation message and redirects the user to the Login page.
Outcome	The user's password is reset, and they can now log in using the new credentials.

Table 9 : UseCase 5: Full Pitch Creation and AI Submission (UC02)

This scenario details the complete end-to-end workflow for an Entrepreneur submitting their project pitch, leading directly into the system's core automated evaluation and classification processes.

Field	**Description**
Use case ID	UC-05 (Full Pitch Creation and Document Submission)
Primary Actor	Entrepreneur, AI Evaluation Engine
Goal	To create a new project record and attach all validated required documents.
Pre-conditions	Entrepreneur has an approved account and is logged in.

Flow of Events	
1.	**Entrepreneur** navigates to "Start New Pitch" and completes the initial **Guided Templates** (Title, Funding Goal, Summary).
2.	**System** stores this structured data in the database and creates the **Project Record**.
3.	**Entrepreneur** opens the “Business Document Upload” section linked to the new Project Record.
4.	**System** displays the required checklist (License, TIN, MoA, Financials, Pitch Deck, etc.).
5.	**Entrepreneur** uploads all required documents.
6.	**System** validates file type/size **[FR 2.2]**.
7.	**OCR** extracts text; the **local classifier** identifies and validates the content of each document.
8.	**AI Completeness Checker** (FR 2.3) scores the submission as **Complete**.
9.	**System** moves the submission status to “Pending Admin Review” (for initial compliance check).
10.	**Administrator** verifies the submission on the dashboard and approves.
Outcome	Pitch submission proceeds to the comprehensive **AI evaluation (embeddings/matching)** and recommendation stage.

Table 10: UseCase 6: Missing Required Documents (Submission Pre-Validation)

This scenario documents the system's ability to halt a submission and provide actionable feedback when mandatory business documents are missing, thus enforcing data completeness and quality before deeper AI processing.

Field	Description
Use case ID	UC-06 (Submission Data Validation Failure)
Primary Actor	Entrepreneur
Goal	To ensure an incomplete pitch submission is corrected and completed before it can proceed to the evaluation pipeline.
Pre-conditions	Entrepreneur has completed the structured pitch forms and initiated the Document Upload step.
Flow of Events (Alternative Path)	
1.	**Entrepreneur** navigates to the "Business Document Upload" checklist.
2.	**Entrepreneur** uploads several documents but omits the **TIN Certificate** and **Financial Statements**.
3.	**Entrepreneur** attempts to finalize the submission.
4.	**System** runs a synchronous **Pre-Validation Check** against the required document checklist.
5.	**System** detects the missing mandatory items.
6.	**System** halts the submission process and displays specific alerts to the Entrepreneur: "TIN Certificate missing," "Financial Statements required."
7.	**Entrepreneur** uploads the missing **TIN Certificate** and **Financial Statements**.
8.	**System** re-runs the Pre-Validation check and finds the submission complete.
Outcome	The submission status is updated to **"Complete"**, and the system automatically triggers the **AI Evaluation Engine** for deep processing (as per UC02).

Table 11: Usecase 7: Wrong Document Uploaded (Content Mismatch)

This scenario details the system's ability to use its automated classification tools to detect incorrect documents (a content mismatch) and enforce the correct upload, preventing bad data from entering the deep AI evaluation pipeline.

Field	**Description**
Use case ID	**UC-07** (Document Content Mismatch Failure)
Primary Actor	Entrepreneur
Goal	To reject a document whose content does not match the required type (e.g., uploading a photo instead of a legal PDF).
Pre-conditions	Entrepreneur has initiated the Document Upload step linked to their Pitch Record.
Flow of Events (Alternative Path)	
1.	**Entrepreneur** navigates to the upload slot for **MoA & AoA (Memorandum and Articles of Association)**.
2.	**Entrepreneur** mistakenly uploads a non-text document (e.g., a **product photo**).
3.	**System** accepts the file type (e.g., JPG) but immediately passes the content to the **OCR** and **Local Classifier**.
4.	**OCR** extracts minimal or irrelevant text; the **Classifier** detects a mismatch (e.g., identifying the content as "Image Data" or "Non-Legal Document").
5.	**System** sets the document status to **"Incorrect Upload"**.
6.	**System** displays a clear alert to the Entrepreneur: "The uploaded file does not match the required document type (Expected: MoA/AoA)."

7.	**Entrepreneur** uploads the correct MoA & AoA document.
8.	**System** validates the correct content and updates the document status to **"Verified"**.
Outcome	The Pitch submission checklist is completed only after the mandatory, correctly identified MoA & AoA document is present.

Table 12: UseCase 8: Expired or Outdated Document Submission

This scenario details the system's ability to automatically check for the validity (based on dates) of time-sensitive legal documents like a Business License, ensuring only compliant and current pitches proceed to the investor matching phase.

Field	**Description**
Use case ID	**UC-08** (Document Expiration Failure)
Primary Actor	Entrepreneur
Goal	To block the submission of a pitch until all mandatory legal documents are current and valid.
Pre-conditions	Entrepreneur has successfully uploaded a document (e.g., Business License).
Flow of Events (Alternative Path)	
1.	**Entrepreneur** uploads a required legal document, such as a **Business License**.
2.	**System** validates the file type and size.
3.	**OCR** (Optical Character Recognition) extracts key text, specifically the **Issue Date** and **Expiry Date** from the document.

4.	**Rule-Based Logic** compares the extracted Expiry Date against the system's **current date**.
5.	**System** identifies that the Expiry Date is in the past.
6.	**System** assigns the document status as **"Expired"** and displays a specific, non-blocking alert: "Your business license is expired. Please upload the renewed version."
7.	**System** keeps the overall Pitch Submission status as **"Incomplete/Blocked"**.
8.	**Entrepreneur** obtains and uploads the renewed license.
9.	**System** repeats steps 3-6 and confirms the new date is valid.
Outcome	The document status is updated to **"Verified"**, and the pitch proceeds to the next stage (AI Evaluation).

Table 13: **UseCase 9: Low-Quality/Unreadable Document Scenario**

This scenario details the system's ability to detect and reject documents that are physically unreadable (e.g., blurry, skewed, or low-resolution scans) based on the confidence score of the Optical Character Recognition (OCR) engine.

Field	**Description**
Use case ID	**UC-09** (Document Readability Failure)
Primary Actor	Entrepreneur
Goal	To prevent unreadable documents from proceeding to the AI evaluation pipeline, ensuring data quality at the source.
Pre-conditions	Entrepreneur has successfully uploaded a file to a document slot.

Flow of Events (Alternative Path)	
1.	**Entrepreneur** uploads a document (e.g., a **blurry PDF scan** of a business license).
2.	**System** passes the document to the **OCR Engine** for text extraction.
3.	The **OCR Engine** attempts extraction but returns a **confidence score below the threshold** (e.g., Confidence < 50%).
4.	**Classifier** (FR 3.2) cannot reliably identify the document type or content due to low OCR score.
5.	**System** flags the document status as **"Unreadable"** and temporarily routes the event for a **fallback check** (a lightweight administrative alert if confidence is extremely low).
6.	**System** displays a clear alert to the Entrepreneur: "Document is unreadable. Please upload a clearer version."
7.	**Entrepreneur** replaces the blurry document with a high-quality scan.
8.	**System** repeats steps 2-4 and achieves a high confidence score.
Outcome	The document status is updated to **"Verified"**, and the pitch proceeds to the next stage (AI Evaluation and Matching).

Table 14: UseCase 10: Fraud or Suspicious Document Scenario

This scenario details the system's ability to detect complex anomalies and inconsistencies within documents suggesting potential fraud and the necessary escalation path to the Administrator for a manual, legally-sound decision.

Field	**Description**
Use case ID	**UC-10** (Suspicious Content Detection and Manual Review)

Primary Actor	Entrepreneur (Submitter)
Supporting Actor	AI Evaluation Engine, Administrator
Goal	To detect documents with structural anomalies or mismatched data fields and ensure they do not proceed without human authorization.
Pre-conditions	Entrepreneur has uploaded a required document (e.g., Business License or MoA).
Flow of Events (Exception Path)	
1.	**Entrepreneur** uploads a document that has been digitally altered (e.g., edited license) or contains inconsistent data.
2.	**OCR** successfully extracts text but identifies **inconsistencies** in formatting (e.g., font changes, alignment issues).
3.	**AI Engine** performs an anomaly check, flagging two key issues: **unusual formatting** and **mismatched business name** (e.g., the name on the license doesn't match the Entrepreneur's profile name).
4.	**System** immediately assigns the document status as **"Flagged - Suspect Content"**.
5.	**System** sends an urgent notification to the **Administrator** for manual review **[FR 8.2]**.
6.	**Administrator** reviews the document against the known profile data and the AI anomaly report.
7.	**Administrator** either:
	a) Requests **resubmission** if the error is minor (e.g., user uploaded the wrong corporate entity's license).
	b) **Rejects the account** entirely if clear fraudulent intent is found (FR 8.1).

Outcome	The document is flagged; the submission is blocked, and the **Administrator** makes a final, auditable decision regarding the account's status.

Table 15: UseCase 11: Partial Submission and Draft Saving

This scenario details the system's ability to allow an entrepreneur to save an incomplete pitch as a draft, recognizing that not all required information or documents may be available immediately. This ensures a seamless user experience while maintaining the rule that only **complete** submissions are routed for review and matching.

Field	**Description**
Use case ID	**UC-11** (Partial Completion and Draft Saving)
Primary Actor	Entrepreneur
Goal	To allow the Entrepreneur to save partially completed work without losing progress, while preventing incomplete data from entering the matching pipeline.
Pre-conditions	Entrepreneur has initiated a Pitch Submission (UC02, Step 1).
Flow of Events (Alternative Path)	
1.	**Entrepreneur** uploads all mandatory **Legal Documents** (License, MoA, etc.) but leaves the **Financial Statements** section empty.
2.	**Entrepreneur** attempts to exit the submission interface.
3.	**System** runs a quick check and recognizes **partial compliance** (e.g., scoring text 60% against the required checklist).
4.	**System** does **not** allow the pitch to be marked "Submitted" for review, displaying a notice: "Submission is incomplete. Only complete pitches are routed to investors."

5.	**System** automatically or manually (via a "Save Draft" button) saves the current data and sets the Pitch status to **"Draft"**.
6.	**Entrepreneur** successfully exits the application and logs out.
7.	**Entrepreneur** logs in a few days later and selects the **Draft Pitch** from their dashboard.
8.	**Entrepreneur** uploads the remaining Financial Statements and clicks **"Finalize Submission"**.
9.	**System** confirms 100\% completeness and triggers the **AI Evaluation Pipeline**.
Outcome	The Pitch is successfully saved in Draft status until the missing data is provided, after which it proceeds to the matching engine.

Table 16: UseCase 12: Multistep Upload (Handling Large Media Files)

This scenario details the system's ability to handle large media files (such as high-resolution pitch decks or long business videos) efficiently by using **chunked upload** and asynchronous processing, thereby avoiding timeouts and ensuring a reliable user experience. This showcases the design's architectural robustness.

Field	**Description**
Use case ID	**UC-12** (Large File Handling and Asynchronous Processing)
Primary Actor	Entrepreneur
Supporting Actor	External Services (Cloud Storage/Cloudinary)
Goal	To successfully upload large files without browser timeouts or session failure, leveraging external scalable media handling.

Pre-conditions	Entrepreneur is logged in and is on the Document/Media Upload interface.
Flow of Events (Main Success Path)	
1.	**Entrepreneur** selects a large file (e.g., a business video > 50 MB) for upload.
2.	**System** initiates the upload process, displaying a **progress bar** to the Entrepreneur.
3.	The **Cloudinary** (or similar external storage) integration handles the file using **chunked upload**, sending data in smaller, manageable segments **[FR 10.2]**.
4.	**System** constantly updates the progress bar until the upload is 100% complete.
5.	Once the upload finishes, the **System** displays the message: **"Upload successful"**.
6.	**System** then displays **"Processing document..."** as the file is passed to an **asynchronous worker** for indexing and virus scanning.
7.	The **Completeness Checker** runs only *after* the asynchronous processing is finished and the final URL is returned to the database.
Outcome	The large file is stored reliably in external services, preventing internal API timeouts, and the pitch submission record is updated with the media URL.

Table 17: UseCase 13: Document Conflict (Name Mismatch)

This scenario details the system's ability to cross-reference data extracted from two different submitted documents, detecting a conflict (e.g., mismatched business names on a TIN Certificate and a Business License), and escalating the issue for human resolution.

Field	Description

Use case ID	**UC-13** (Cross-Document Data Conflict)
Primary Actor	Entrepreneur (Submitter)
Supporting Actor	AI Evaluation Engine, Administrator
Goal	To detect and flag potential conflicts between essential legal documents, ensuring data integrity across the submission.
Pre-conditions	Entrepreneur has uploaded two or more documents that contain overlapping data fields (e.g., Business Name, Registration Number).
Flow of Events (Exception Path)	
1.	**Entrepreneur** uploads the **TIN Certificate** and the **Business License**.
2.	**OCR Engine** extracts the designated entity name from the **TIN Certificate** (e.g., "Mamo Retail P.L.C.").
3.	**OCR Engine** extracts the designated entity name from the **Business License** (e.g., "Mamo Retailers P.L.C.").
4.	**System Logic** compares the two extracted strings and detects a **mismatch** (minor difference or major conflict).
5.	**System** flags the document set status as **"Conflict Detected"** and provides a specific alert to the Entrepreneur: "Document names differ. Please provide supporting evidence or correct the upload."
6.	**System** automatically routes the submission to the **Administrator** for manual review **[FR 8.2]**.
7.	**Administrator** reviews the conflict and either:
	a) Approves the conflict if it is a minor, known discrepancy (e.g., a simple spelling mistake).

	b) Requests **additional documents** (e.g., a letter from the business registry) to resolve the conflict.
Outcome	The submission is blocked from matching until the Administrator has verified the conflict and manually approved or requested clarification.

Table 18: UseCase 14: Administrator Correction and AI Override

This scenario details the system's function for an Administrator to review a submission that the AI has incorrectly flagged, allowing human judgment to override the automated decision and justifying the hybrid system architecture.

Field	**Description**
Use case ID	**UC-14** (Admin Override of AI Flag)
Primary Actor	Administrator
Supporting Actor	AI Evaluation Engine
Goal	To allow a human expert (Administrator) to correct a potential false positive flagged by the AI, ensuring valid submissions are not blocked.
Pre-conditions	A pitch document (e.g., MoA) has been flagged by the AI (e.g., in UC10) and its status is "Pending Admin Review."
Flow of Events (System Correction Path)	
1.	**Administrator** logs into the Admin Dashboard **[FR 8.1]** and navigates to the "Flagged Submissions" queue **[FR 8.2]**.
2.	**System** displays the flagged document alongside the **AI notes** explaining *why* it was flagged (e.g., low confidence score, minor formatting inconsistency).

3.	**Administrator** reviews the document and determines the flag is a **false positive** (the document is legally valid).
4.	**Administrator** clicks the action button **“Override AI Decision (Mark as Valid)”**.
5.	**System** records the Administrator's action, the timestamp, and the reason (if provided) in the **Audit Log [FR 7.2]**.
6.	**System** updates the document status to **"Verified by Admin"**.
7.	**System** changes the overall pitch status to **"Ready for Matching"** (or triggers the next step in the workflow).
Outcome	Human judgment prevails; the submission continues through the pipeline, and the override action is logged for accountability and future AI model training.

Table 19: UseCase 15: Multi-Business Entrepreneur (Conflict of Entities)

This scenario details the system's ability to detect and resolve a conflict where documents uploaded by a single user belong to different, distinct legal entities (e.g., Company A vs. Company B). This enforces strict data hygiene for legal compliance.

Field	**Description**
Use case ID	**UC-15** (Multi-Entity Document Conflict)
Primary Actor	Entrepreneur
Supporting Actor	AI Evaluation Engine

Goal	To identify which legal entity the pitch is for and ensure all supporting documents consistently belong to that single entity.
Pre-conditions	Entrepreneur is submitting a new pitch (UC02) and likely owns multiple businesses.
Flow of Events (Exception Path)	
1.	**Entrepreneur** uploads the **Business License** for **Company A**.
2.	**Entrepreneur** mistakenly uploads the **MoA & AoA** for **Company B**.
3.	**System** extracts the entity names from both documents (UC13 logic).
4.	**AI Logic** detects a significant **mismatch** between the core legal documents.
5.	**System** flags the submission as **"Entity Conflict"** and halts processing.
6.	**System** asks the Entrepreneur: **“We detected a name conflict: [Company A] vs. [Company B]. Which business is this pitch for?”**
7.	**Entrepreneur** selects the correct entity, **Company A**, confirming the intent.
8.	**System** uses the selection to generate a new, targeted checklist for Company A's required documents.
9.	**System** alerts the Entrepreneur that all conflicting documents (Company B's MoA) must be **replaced** with Company A's documents.
Outcome	The submission is locked to a single legal entity, and the Entrepreneur must upload a consistent set of documents before proceeding.

Table 20: UseCase 16: AI Fallback Scenario (Hybrid Checker)

This scenario documents the system's strategy for handling low-confidence predictions from its internal, lightweight classifiers by triggering a higher-cost, more powerful **external AI**

API (like Gemini or GPT) as a structured fallback. This optimizes processing costs while maximizing accuracy.

Field	Description
Use case ID	**UC-16** (AI Confidence-Based Fallback)
Primary Actor	AI Evaluation Engine
Supporting Actor	External AI APIs (Gemini/GPT)
Goal	To achieve high-accuracy document classification and analysis only when the internal classifier fails to meet a necessary confidence threshold, controlling costs and improving robustness.
Pre-conditions	● A document has been successfully uploaded (UC02), and the internal classification process has begun. ● The user has explicitly consented to external AI processing during upload. ● Sensitive identifiers (PII and IP data) have been automatically redacted before any external API call.
Security & Compliance Preconditions	● User consent stored in audit log for external processing. ● Data minimization applied only required text sent. ● Redaction completed successfully. ● External API provider listed in Privacy Policy.
Flow of Events (System Enhancement Path)	
1.	**Internal Classifier** processes a pitch document (e.g., Financial Statement).
2.	The **Internal Classifier** returns a **low confidence score** (e.g., 45%) for classifying the document type or extracting key data.
3.	**System Logic** detects the score is below the predefined threshold (e.g., 75%).

4.	**System** triggers an **asynchronous worker** to call the **External AI API** (e.g., Gemini) for structured analysis of the document's text/content **[FR 10.3]**.
5.	The **External API** returns a result (e.g., 95% confidence that it is a 'Balance Sheet' and extracts a key metric).
6.	**System** merges the result from the External API with the internal data and updates the document status and the overall **Completeness Score**.
7.	**System** logs the API call for cost auditing purposes.
Outcome	The document is accurately classified despite the internal model's uncertainty, allowing the pitch to proceed with high data integrity while minimizing overall API usage.

Table 21: UseCase 17: Admin Approves Entrepreneur Submission (Final Gate)

This scenario details the final mandatory human step where the Administrator reviews the fully validated pitch and all supporting documents before the project is released into the active investor matching pipeline.

Field	**Description**
Use case ID	**UC-17** (Final Administrative Approval)
Primary Actor	Administrator
Supporting Actor	Entrepreneur, Notification Service
Goal	To grant final permission for a pitch to become active and visible to Investors, utilizing all previous validation steps.
Pre-conditions	The Pitch Status is **"Awaiting Final Admin Approval"** (meaning UC02 and all necessary validation checks, UC06 through UC16, are complete and resolved).

Flow of Events (Main Success Path)	
1.	**Administrator** logs into the Admin Dashboard **[FR 8.1]** and navigates to the **“Pending Submissions”** queue.
2.	**System** displays the list, prioritizing submissions that have cleared all automated checks.
3.	**Administrator** selects the specific pitch and reviews the key data points:
	* The original **Business Documents** (License, MoA).
	* The final **AI Completeness Score** and **Classification**.
	* The pitch content itself (Executive Summary, Vision).
4.	**Administrator** confirms all details meet the platform's final criteria and clicks **“Approve Submission”**.
5.	**System** securely logs the Administrator's action (FR 7.2) and marks the pitch status as **“Verified & Active”**.
6.	**System** immediately sends a **Push Notification** to the Entrepreneur: **“Your pitch has been approved and is now active!”** (FR 9.2).
Outcome	The pitch record is active, fully verified, and now becomes eligible for evaluation and recommendation to potential investors.

Table 22 :UseCase 18: AI Evaluation and Scoring Pipeline

This scenario details the critical, resource-intensive process where the system analyzes the fully verified pitch content using multiple semantic and classification models to generate the data required for investor matching.

Field	**Description**
Use case ID	**UC-18** (Semantic Analysis and Ranking)

Primary Actor	AI Evaluation Engine
Supporting Actor	Vector Index Storage, Document Database
Goal	To transform the submitted pitch text into structured, quantifiable data (embeddings, scores, summaries) for efficient investor matching.
Pre-conditions	The Pitch Status is **"Verified & Active"** (UC17 complete).
Flow of Events (System Core Process)	
1.	**System** triggers an **asynchronous worker** to fetch the verified pitch data and documents.
2.	**AI Engine** performs **semantic modeling** (FR 3.1) by generating **transformer-based sentence embeddings** for key textual components:
	* Problem statement
	* Solution
	* Business model
	* Market description
3.	**AI Engine** executes the **Classification Pipeline** (FR 3.2), checking for:
	* **Completeness** (Final check post-admin approval).
	* **Compliance** (Adherence to platform rules).
	* **Spam-likelihood** (Final filter).
4.	**System** generates quantifiable metrics:
	* A **Relevance Score** (for initial ranking).

	* A **Risk Indicator** (based on classification results).
5.	**System** produces an **AI-assisted textual summary** of the project for quick investor consumption (FR 4.3).
6.	The final **embeddings** and scoring results are stored in the **vector index** for fast similarity search **[FR 3.1, 10.2]**.
Outcome	The Pitch record is fully analyzed, enriched with semantic data, and becomes **searchable and matchable** via vector similarity methods (FR 4.2).

Table 22: UseCase 19: Investor Recommendation, Review, and Voice Output

This scenario details the Investor's experience, from viewing the personalized recommendation dashboard to reviewing a pitch using AI-generated summaries and multilingual features.

Field	**Description**
Use case ID	**UC-19** (Recommendation Consumption)
Primary Actor	Investor
Supporting Actor	System, Notification Service, Text-to-Speech Component
Goal	To present relevant, prioritized pitches to the Investor and facilitate quick, informed decision-making.
Pre-conditions	The Pitch is **"Verified & Active"** (UC17) and fully analyzed (UC18). The Investor is logged in and has set their preferences **[FR 4.1]**.
Flow of Events (Main Success Path)	

1.	**System** runs the **Matching Engine** (vector similarity search) using the Pitch embeddings against the Investor's profile embeddings **[FR 4.2]**.
2.	**System** generates a list of highly-ranked recommendations and sends a **Push Notification** to the Investor's mobile device **[FR 9.2]**.
3.	**Investor** logs in and views the prioritized dashboard.
4.	**Investor** filters the list based on risk indicator and relevance score.
5.	**Investor** selects a recommended pitch to view the detailed report.
6.	**System** displays the pitch details, including the **AI-assisted summary** (FR 4.3) and the **Explainability Features** (FR 3.3) for why the pitch was matched.
7.	**Investor** selects the **Speech Output** function and chooses a language (e.g., English or Amharic) **[FR 5.1]**.
8.	**System** uses the **Text-to-Speech Component** to generate an audio summary of the key findings **[FR 5.3]**.
Outcome	The Investor has quickly consumed all necessary data and is ready to decide whether to connect with the Entrepreneur (leading to UC20).

Table 23: UseCase 20: Communication and Meeting Scheduling

This scenario details the system's function for facilitating direct, secure communication between the Investor and Entrepreneur, culminating in the scheduling of a formal meeting.

Field	**Description**
Use case ID	**UC-20** (Connection and Coordination)
Primary Actor	Investor (Initiator), Entrepreneur (Responder)

Goal	To establish a secure communication channel and finalize a meeting time using the in-platform tools.
Pre-conditions	The Investor has reviewed the pitch (UC19) and has high interest.
Flow of Events (Main Success Path)	
1.	**Investor** is viewing the approved pitch details and clicks the **"Request Connection"** button.
2.	**System** opens the **Secure Messaging** channel, specific to the Investor-Entrepreneur pair **[FR 6.1]**.
3.	**Investor** sends an introductory message.
4.	**System** sends an instant **Notification** (web/push) to the **Entrepreneur** alerting them of the new message.
5.	**Entrepreneur** responds to the message. If the Entrepreneur's preferred language differs, the **System** translates the text based on language settings **[FR 5.1]**.
6.	**Investor** initiates the **Meeting Scheduling** function **[FR 6.2]**.
7.	**System** displays the Investor's available time slots (integrated with their external calendar).
8.	**Entrepreneur** selects an available time slot and confirms the meeting.
9.	**System** logs the scheduled meeting time and sends final calendar invites to both parties.
Outcome	Communication is secured and logged; a formal meeting is scheduled, moving the process closer to a deal.

Table 24: UseCase 21: Milestone Payment Simulation and Tracking

This scenario details the system's function for simulating a phased, milestone-based investment relationship, which allows both the Investor and Entrepreneur to track transaction progress securely and transparently. This maps directly to **Functional Requirement 7 (Payment Simulation and Milestone Tracking)**.

Field	**Description**
Use case ID	**UC-21** (Post-Investment Milestone Tracking)
Primary Actor	Investor, Entrepreneur
Supporting Actor	System
Goal	To securely simulate payment commitment releases based on the successful verification of project milestones.
Pre-conditions	The Investor and Entrepreneur have finalized a deal, and the **Milestone Framework** has been mutually established in the system **[FR 7.1]**.
Flow of Events (Main Success Path)	
1.	**Entrepreneur** completes the first agreed-upon milestone (e.g., "Prototype Completion").
2.	**Entrepreneur** navigates to the Milestone Tracker and marks the milestone as **"Complete and Ready for Review"**.
3.	**System** sends a **Notification** to the **Investor** to review the completed milestone.
4.	**Investor** reviews the evidence (e.g., linked documentation, status report) submitted by the Entrepreneur.
5.	**Investor** approves the milestone by clicking **"Verify Completion and Simulate Payment Release"**.

6.	**System** updates the milestone status to **"Paid/Verified"** and simulates the release of the associated funding tranche.
7.	**System** securely logs the transaction activity in the **Transaction Logging** module, ensuring transparency and traceability **[FR 7.2]**.
8.	Both parties can view the updated **Payment Progress** on their respective dashboards.
Outcome	The milestone is successfully verified; the system simulates the associated payment release, and the transaction is logged for future auditing.

Table 25: UseCase 22: Proposal Rejection (Investor Declines Pitch)

This scenario details the process when an Investor decides a recommended pitch is not suitable for their portfolio, demonstrating how the system captures this essential negative feedback for continuous improvement of the recommendation algorithm.

Field	**Description**
Use case ID	**UC-22** (Negative Feedback and AI Learning)
Primary Actor	Investor
Supporting Actor	AI Evaluation Engine
Goal	To allow the Investor to clearly decline a pitch and to ensure that this signal is captured and used to refine future recommendation models.
Pre-conditions	The Investor has reviewed the pitch (UC19) and has decided against the investment.
Flow of Events (Alternative Path)	
1.	**Investor** is viewing the pitch details on their dashboard.

2.	**Investor** clicks the **“Not Interested”** button.
3.	**System** prompts the Investor to optionally select a **Rejection Reason** (e.g., "Wrong Sector," "Valuation too High," "Early Stage").
4.	**System** securely logs the rejection decision, the timestamp, and the reason provided **[FR 7.2]**.
5.	**System** immediately feeds this negative feedback signal back to the **AI Matching Engine** and **Vector Index** to update the Investor's profile model **[FR 4.2]**.
6.	**System** removes the rejected pitch from the Investor's active recommendation list.
Outcome	The Investor's profile is updated to prevent similar unsuitable pitches in the future, and the pitch remains active and available for matching with other relevant investors.

Table 25: UseCase 23: Admin Removal or Suspension of Pitch

This scenario details the Administrator's final action to enforce platform integrity by suspending or permanently removing a pitch deemed fraudulent, non-compliant, or violating platform policies.

Field	**Description**
Use case ID	**UC-23** (Content Enforcement and Removal)
Primary Actor	Administrator
Supporting Actor	Entrepreneur, Notification Service
Goal	To enforce platform policies, remove harmful content, and notify the entrepreneur of the corrective action required.

Pre-conditions	A pitch has been flagged (e.g., UC10, UC13) and has been escalated for final review, or the Admin finds a violation during routine monitoring (FR 8.3).
Flow of Events (Enforcement Path)	
1.	**Administrator** logs into the Admin Dashboard and reviews the specific pitch and its associated violation reports.
2.	**Administrator** confirms the pitch is either **fraudulent** or severely **violates platform policies**.
3.	**Administrator** selects the appropriate enforcement action:
	a) **Suspension:** Temporary block, usually allowing for correction.
	b) **Removal:** Permanent deletion from public view.
4.	**Administrator** enters a detailed justification for the action (required for audit logging).
5.	**Administrator** clicks **“Apply Enforcement Action”**.
6.	**System** updates the Pitch Status to **"Suspended"** or **"Removed"** and locks the record from matching or editing.
7.	**System** sends a formal **Notification** to the **Entrepreneur** detailing the violation and the action taken, along with any steps for **correction** (if suspended).
8.	**System** securely logs the action in the **Transaction Logging** module.
Outcome	The pitch is removed from active circulation, preserving platform trust and quality, and the enforcement action is fully auditable.

Table 26: **UseCase 24: Entrepreneur Account Suspension**

This scenario details the process where an Administrator identifies a pattern of suspicious behavior or violation by an entrepreneur and takes immediate, system-wide action to suspend the account and block access, ensuring platform integrity.

Field	**Description**
Use case ID	**UC-24** (User Enforcement and Restriction)
Primary Actor	Administrator
Supporting Actor	Entrepreneur, Notification Service, Authentication Provider
Goal	To immediately disable an entrepreneur's access and activity when platform policies are suspected to be violated, while maintaining an audit trail.
Pre-conditions	The Administrator has received alerts or has identified patterns of suspicious activity linked to a specific entrepreneur's profile (e.g., repeated document fraud attempts, investor complaints).
Flow of Events (Enforcement Path)	
1.	**Administrator** logs into the Admin Dashboard and navigates to the **Entrepreneur Management** section **[FR 8.1]**.
2.	**Administrator** reviews the collected evidence: **suspicious document patterns, mismatched business details, flagged AI alerts**, and any **investor complaints**.
3.	**Administrator** selects the specific user and clicks the action **“Suspend Account”**.
4.	**System** immediately performs the security sequence:
	* Disables the entrepreneur's login credentials in the Authentication Provider.

	* Unpublishes all associated pitches (UC24 logic).
	* Freezes any active messaging threads **[FR 6.1]**.
	* Logs the suspension action, timestamp, and justification **[FR 7.2]**.
5.	**System** sends a **Notification Email** to the entrepreneur: *"Your account has been temporarily suspended pending review."*
6.	**Entrepreneur** attempts to log in and receives the message: *"Your account is currently suspended. Please contact support."*
Outcome	The account enters a restricted, frozen state; the entrepreneur cannot access or submit new content until the account is reviewed and potentially reinstated by the Administrator.

Table 27: UseCase 25: Investor Account Suspension

This scenario details the process for an Administrator to act upon reports of investor misconduct, immediately revoking access and interaction privileges to protect the platform's community and integrity.

Field	**Description**
Use case ID	**UC-25** (Investor Enforcement and Restriction)
Primary Actor	Administrator
Supporting Actor	Investor, Entrepreneur, Notification Service
Goal	To immediately suspend an investor's privileges when terms are violated, halting all current and future interactions.

Pre-conditions	The Administrator has received reports, or system monitoring has flagged, an investor for policy violations (e.g., fraudulent documents, inappropriate behavior).
Flow of Events (Enforcement Path)	
1.	**Multiple Entrepreneurs** submit formal reports regarding suspicious investor activity (e.g., inappropriate messages, demanding private data outside the system).
2.	**Administrator** logs into the Admin Dashboard and reviews the **complaint logs** and **secure communication transcripts**.
3.	**Administrator** confirms a policy violation and selects **"Suspend Investor Account"** in the User Management section **[FR 8.1]**.
4.	**System** performs the security sequence immediately:
	* **Blocks login** credentials for the Investor.
	* **Removes the Investor** from the matching algorithm (FR 4.2).
	* **Cancels** all future scheduled meetings (FR 6.2).
	* **Unlinks** all active message threads with entrepreneurs (FR 6.1).
	* Logs the suspension action, timestamp, and justification **[FR 7.2]**.
5.	**System** sends a **Notification Email** to the investor: *"Your investor account has been suspended due to policy violations."*
6.	**System** sends **Notifications** to all entrepreneurs currently interacting with the investor, advising of the suspension.
Outcome	The Investor cannot access or interact with the platform; integrity is maintained, and all affected entrepreneurs are protected.

UseCase 26: Investor Reports Suspicious Entrepreneur

This scenario details the process by which an Investor reports an Entrepreneur for suspicious activity, allowing the Administrator to investigate and potentially enforce penalties.

Field	**Description**
Use case ID	**UC-26** (Investor Conduct Report)
Primary Actor	Investor
Recipient Actor	Entrepreneur (Accused)
Goal	To allow an Investor to report an Entrepreneur for misconduct (e.g., fraudulent claims, inappropriate contact) and trigger administrative review.
Pre-conditions	The Investor and Entrepreneur have established a secure message thread (UC21).
Flow of Events (Report and Investigation Path)	
1.	**Investor** detects suspicious behavior (e.g., the Entrepreneur demands excessive fees outside the platform).
2.	**Investor** navigates to the pitch or message thread and clicks **"Report Suspicious Activity."**
3.	**System** prompts the Investor for details and uploads any supporting evidence (screenshots, etc.).
4.	**System** securely logs the complaint and sends an urgent notification to the **Administrator**.
5.	**Administrator** reviews the complaint details and the **communication transcripts** (FR 6.1).
6.	**Administrator** initiates an **internal investigation**, which may include locking the pitch or suspending the Entrepreneur's account (UC25) pending resolution.

Outcome	The reported activity is logged, and the Administrator takes action to resolve the conflict and enforce platform terms.

Table 28: UseCase 27: Entrepreneur Reports Suspicious Investor

This scenario details the process by which an Entrepreneur reports an Investor for misconduct, triggering immediate action to protect the Entrepreneur from potentially harmful or inappropriate contact.

Field	**Description**
Use case ID	**UC-27** (Entrepreneur Conduct Report)
Primary Actor	Entrepreneur
Recipient Actor	Investor (Accused)
Goal	To allow an Entrepreneur to report an Investor for misconduct (e.g., harassment, inappropriate requests) and trigger immediate protective action.
Pre-conditions	The Investor and Entrepreneur have established a secure message thread (UC21).
Flow of Events (Report and Protective Action Path)	
1.	**Entrepreneur** receives inappropriate or suspicious communication from the Investor.

2.	**Entrepreneur** uses the reporting function within the message thread: **“Report Investor Misconduct.”**
3.	**System** logs the complaint and automatically **temporarily suspends the communication thread** between the two parties for safety.
4.	**System** sends an urgent notification to the **Administrator**.
5.	**Administrator** prioritizes the review of this complaint (due to the potential for immediate harm) and reviews the message transcripts.
6.	**Administrator** immediately takes decisive action, which may include permanent **suspension of the Investor account** (UC26) or a formal warning.
Outcome	The Entrepreneur is protected by the suspension of the communication channel, and the Investor's privileges are reviewed based on the severity of the violation.

3.5. Object Model

3.5.1. Data Dictionary

A. Core Entities

1. Collection: users

Purpose: Stores basic system user accounts.

Field Name	Type	Constraints	Description

_id	ObjectId	Primary Key	Unique identifier generated by MongoDB
fullName	String	Required	Full legal name of the user
email	String	Required, Unique	User email address
phoneNumber	String	Optional	Registered phone number
passwordHash	String	Required	Hashed password
role	String	Required	User role: Admin, Entrepreneur, Investor
status	String	Required	Account status: Active, Inactive, Suspended
lastLogin	Date	Optional	Timestamp of last login
failedLoginAttempts	Number	Default: 0	Failed login count
passwordLastChanged	Date	Optional	Timestamp of last password change
createdAt	Date	Auto-generated	Document creation time
updatedAt	Date	Auto-generated	Last update time

Relationship: One-to-one with entrepreneurProfiles or investorProfiles.

2. Collection: entrepreneurProfiles

Purpose: Stores additional business profile details for entrepreneurs.

Field Name	Type	Constraints	Description
_id	ObjectId	Primary Key	Profile ID
userId	ObjectId	Required, Unique	References users._id

companyName	String	Required	Registered company name
verified	Boolean	Default: false	Verification status
phoneNumber	String	Optional	Company phone number
address	String	Optional	Office address
city	String	Optional	City
country	String	Optional	Country
website	String	Optional	Website URL
verifiedAt	Date	Optional	Verification timestamp
documents	Array<ObjectId>	Optional	List of verification documents
createdAt	Date	Auto-generated	Document creation timestamp
updatedAt	Date	Auto-generated	Last update timestamp

Relationship: One-to-one with users.

3. Collection: investorProfiles

Purpose: Additional profile details for investors.

Field Name	Type	Constraints	Description
_id	ObjectId	Primary Key	

userId	ObjectId	Required, Unique	References users._id
preferredSectors	Array<String>	Optional	Preferred investment focus areas
minInvestment	Decimal128	Optional	Minimum investment budget
maxInvestment	Decimal128	Optional	Maximum investment budget
investmentType	String	Optional	Equity, Debt, etc.
address	String	Optional	Investor address
phoneNumber	String	Optional	Phone contact
createdAt	Date	Auto-generated	
updatedAt	Date	Auto-generated	

Relationship: One-to-one with users.

B. Submission & Tagging

4. Collection: submissions

Purpose: Stores submitted business ideas/pitches.

Field Name	Type	Constraints	Description

_id	ObjectId	Primary Key	
entrepreneurId	ObjectId	Required	References entrepreneurProfiles._id
title	String	Required	Submission title
summary	String	Required	Summary of idea/pitch
sector	String	Required	Business sector
stage	String	Required	Seed, SeriesA, Growth
targetAmount	Decimal128	Required	Funding requested
status	String	Required	Draft, Submitted, UnderReview, Approved, Rejected, Funded
tags	Array<ObjectId>	Optional	References tags._id
documents	Array<ObjectId>	Optional	Submission document IDs
createdAt	Date	Auto-generated	
updatedAt	Date	Auto-generated	

5. Collection: tags

Purpose: Standardized labels for categorizing submissions.

Field Name	Type	Constraints	Description

_id	ObjectId	Primary Key	
name	String	Unique, Required	Tag label (e.g., SaaS, Fintech)
createdAt	Date	Auto-generated	

C. Communication & Meetings

6. Collection: conversations

Purpose: Stores chat threads between users.

Field Name	Type	Constraints	Description
_id	ObjectId	Primary Key	
participantIds	Array<ObjectId>	Required	Array of user IDs
lastMessageAt	Date	Auto	Timestamp of last message
createdAt	Date	Auto	
updatedAt	Date	Auto	

7. Collection: messages

Purpose: Chat messages exchanged between users.

Field Name	Type	Constraints	Description
id	ObjectId	Primary Key	
conversationId	ObjectId	Required	References conversations._id
senderId	ObjectId	Required	References users. id
content	String	Required	Message content
attachments	Array<ObjectId>	Optional	Attached file IDs
sentAt	Date	Required	Timestamp sent
isDeleted	Boolean	Default: false	Soft delete flag

8. Collection: meetings

Purpose: Schedules meetings between entrepreneur and investor.

Field Name	Type	Constraints	Description
id	ObjectId	Primary Key	
submissionId	ObjectId	Required	References submissions._id
participants	Array<ObjectId>	Required	List of user IDs
proposedSlots	Array<Date>	Optional	List of proposed meeting dates
confirmedSlot	Date	Optional	Final agreed time
status	String	Required	Proposed, Confirmed, Cancelled, etc.
createdAt	Date	Auto	

updatedAt	Date	Auto	

D. Financial & Verification

9. Collection: milestones

Purpose: Funding milestones under each submission.

Field Name	Type	Constraints	Description
_id	ObjectId	Primary Key	
submissionId	ObjectId	Required	References submissions._id
description	String	Required	Goal of the milestone
amount	Decimal128	Required	Amount required
dueDate	Date	Required	Deadline
status	String	Required	Pending, Achieved, Funded, Failed
createdAt	Date	Auto	
updatedAt	Date	Auto	

10. Collection: ledgerEntries

Purpose: Records release of funds for milestones.

Field Name	Type	Constraints	Description
_id	ObjectId	Primary Key	
milestoneId	ObjectId	Required	References milestones._id
amountReleased	Decimal128	Required	Amount released
releasedAt	Date	Required	Release timestamp
transactionRef	String	Optional	Transaction reference
notes	String	Optional	Comments

11. Collection: verifications

Purpose: KYC/AML verification of users and profiles.

Field Name	Type	Constraints	Description
_id	ObjectId	Primary Key	
targetType	String	Required	User, EntrepreneurProfile, InvestorProfile
targetId	ObjectId	Required	ID of the target entity
status	String	Required	Pending, Approved, Rejected
kycCompleted	Boolean	Default: false	
amlPassed	Boolean	Default: false	
phoneVerified	Boolean	Default: false	
emailVerified	Boolean	Default: false	

reviewerId	ObjectId	Optional	Admin who reviewed
reviewedAt	Date	Optional	Timestamp
createdAt	Date	Auto	
updatedAt	Date	Auto	

12. Collection: verificationDocuments

Purpose: Uploaded documents for verification.

Field Name	Type	Constraints	Description
_id	ObjectId	Primary Key	
verificationId	ObjectId	Required	References verifications._id
fileUrl	String	Required	Storage URL
fileType	String	Required	ID_PROOF, ADDRESS_PROOF, BUSINESS_REG
uploadedAt	Date	Auto	
notes	String	Optional	

13. Collection: submissionDocuments

Purpose: Business documents supporting a submission.

Field Name	Type	Constraints	Description
_id	ObjectId	Primary Key	
submissionId	ObjectId	Required	References submissions. _id
fileUrl	String	Required	File storage link
fileName	String	Required	Document name
fileType	String	Required	PitchDeck, Financials, BusinessPlan
uploadedBy	ObjectId	Required	References users. _id
uploadedAt	Date	Auto	

E. Audit & Notifications

14. Collection: adminActions

Purpose: Logs administrative actions for auditing.

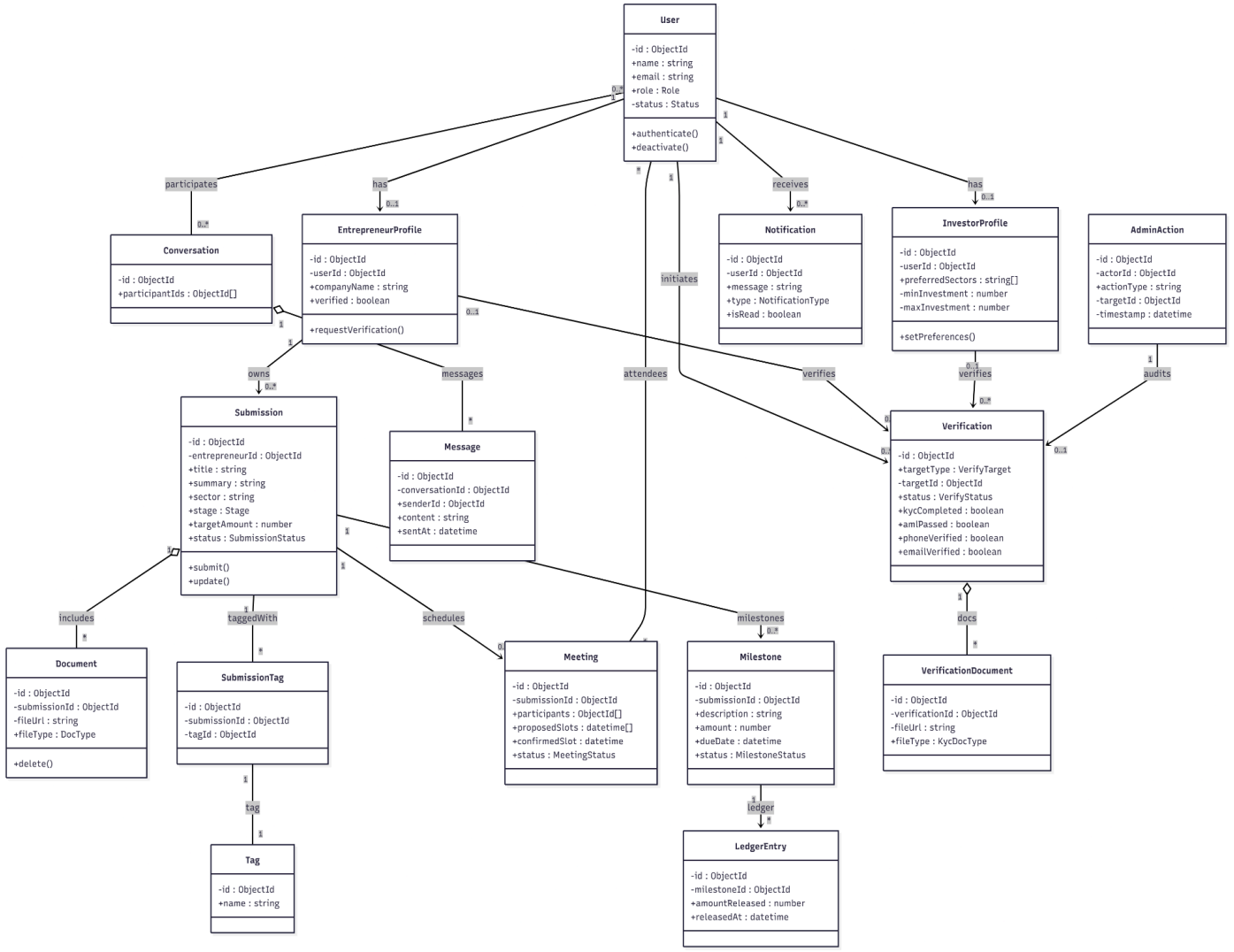
Field Name	Type	Constraints	Description
_id	ObjectId	Primary Key	
actorId	ObjectId	Required	Admin user ID
actionType	String	Required	Description of action
targetId	ObjectId	Optional	Affected entity ID
meta	Object	Optional	Additional structured data
timestamp	Date	Auto	

15. Collection: notifications

Purpose: In-app notification system.

Field Name	Type	Constraints	Description
id	ObjectId	Primary Key	
userId	ObjectId	Required	Recipient user
message	String	Required	Notification body
type	String	Required	Notification type
isRead	Boolean	Default: false	Read status
data	Object	Optional	Action payload
createdAt	Date	Auto	

3.5.2. Class diagram



3.5.3. Dynamic model

The Dynamic Model focuses on the **behavioral aspects** of the Smart Entrepreneurial Pitching and Matching System, illustrating how objects and actors interact in chronological order to achieve the system's core functions. It defines the flow of control, the sequencing of operations, and the life cycle (states) of key system entities, such as a **Pitch** or a **User Profile**. This model is critical for validating the design against the functional requirements and is detailed through the following **Sequence**, **Activity**, and **State Chart** diagrams. In these diagrams the ‘SEPMS Backend' represents the Node.js server and the ‘AI Engine or Asynchronous worker’ represent the Python FastAPI service.

3.5.4.. Sequence diagram

Fig: Sequence diagram: Entrepreneur Sign Up & Initial Verification (UC01)

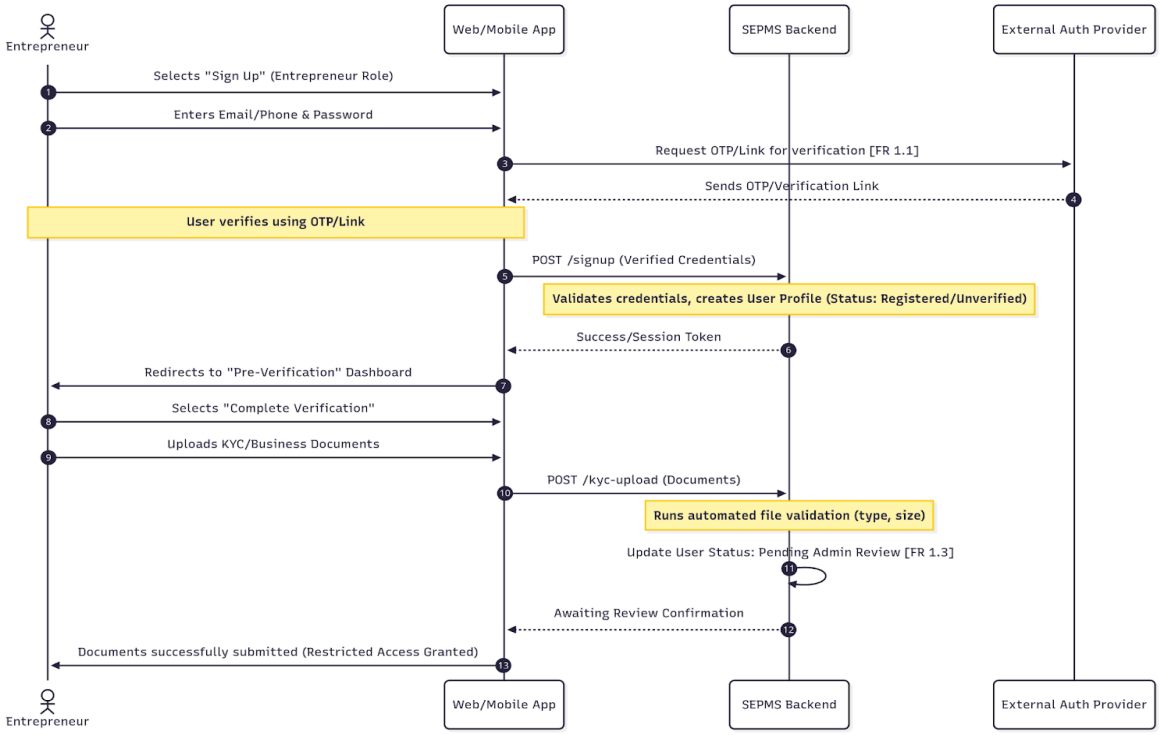


Fig: Sequence diagram: Investor Sign Up & Financial Verification (UC02)

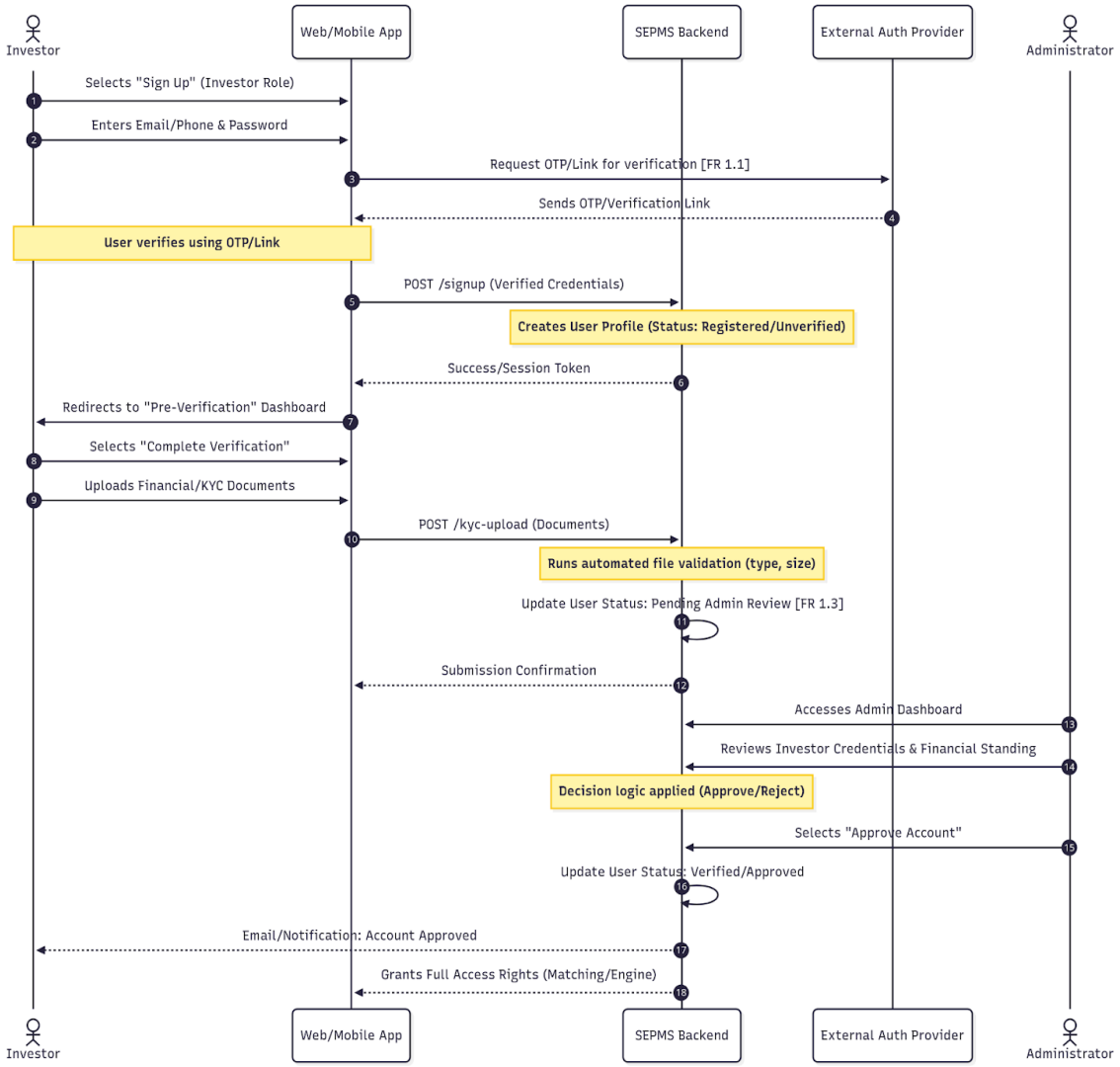


Fig: Sequence diagram: Successful User Authentication (Login)

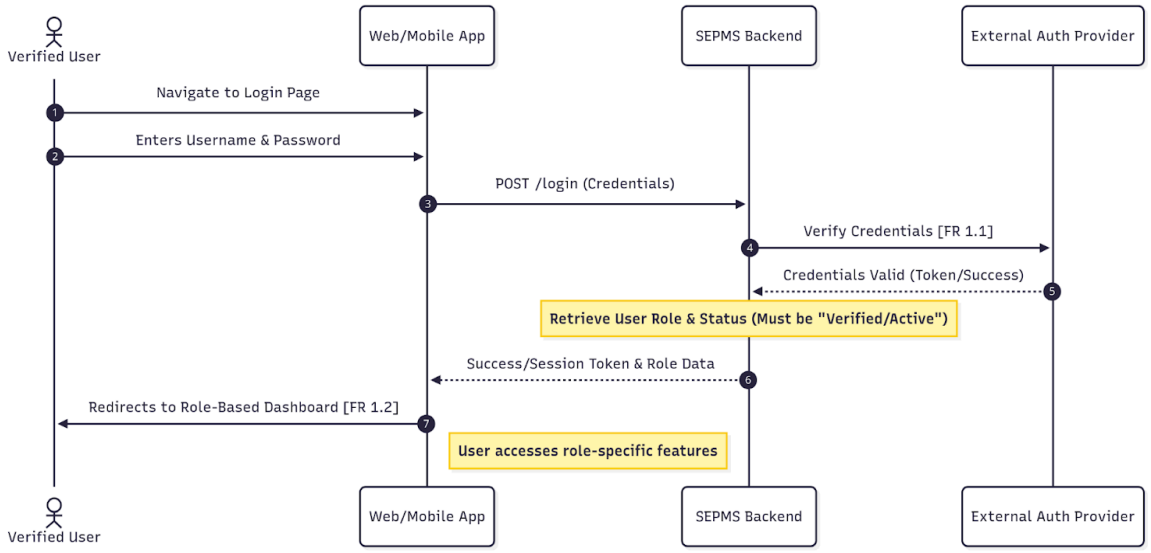


Fig: Sequence diagram: Password Recovery (Forgot Password)

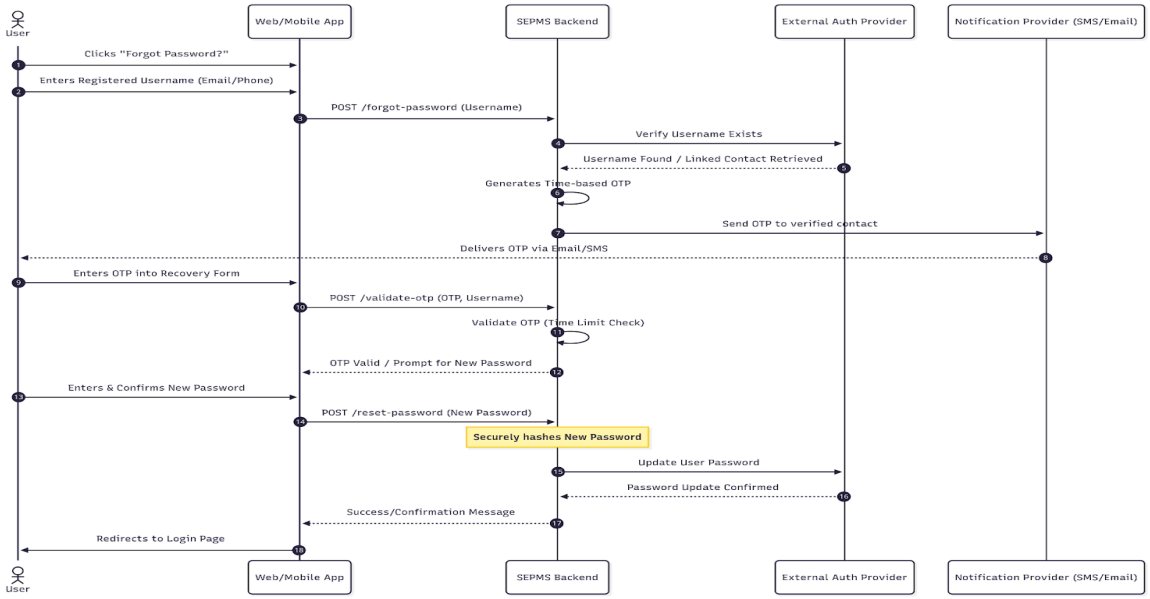


Fig: Sequence diagram: Full Pitch Creation and AI Submission (UC02)

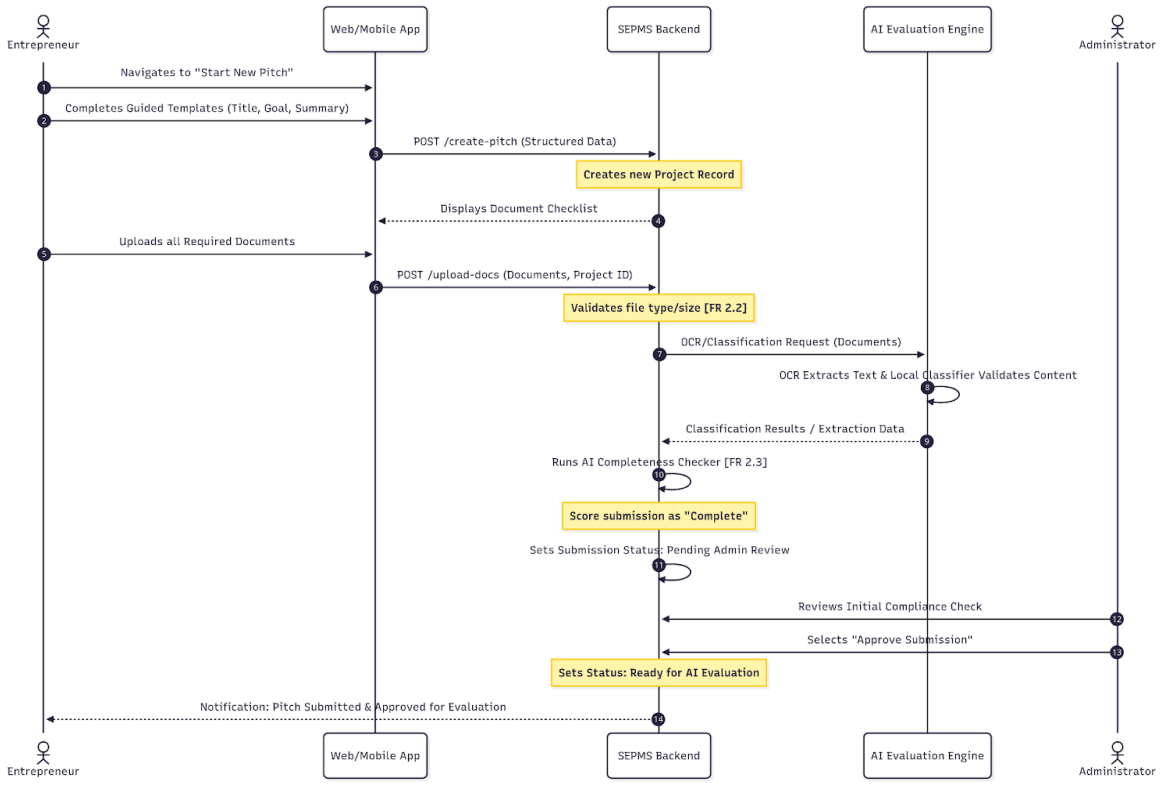


Fig: Sequence diagram: Missing Required Documents (Submission Pre-Validation)

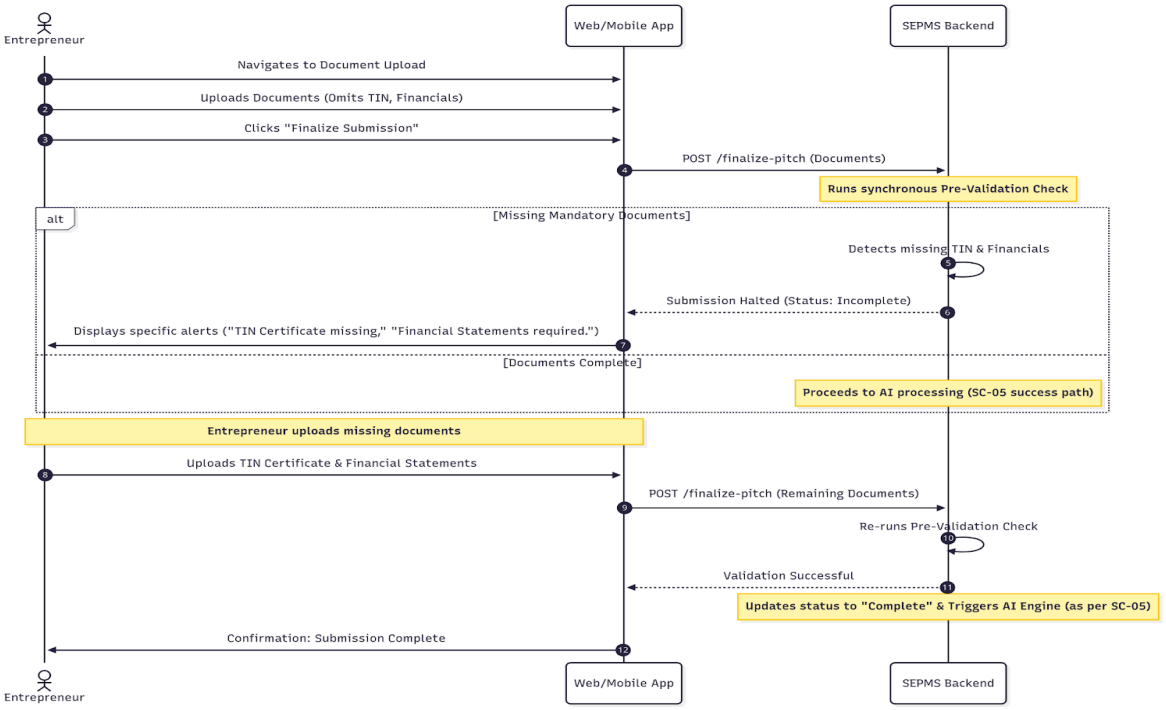


Fig: Sequence diagram: Wrong Document Uploaded (Content Mismatch)

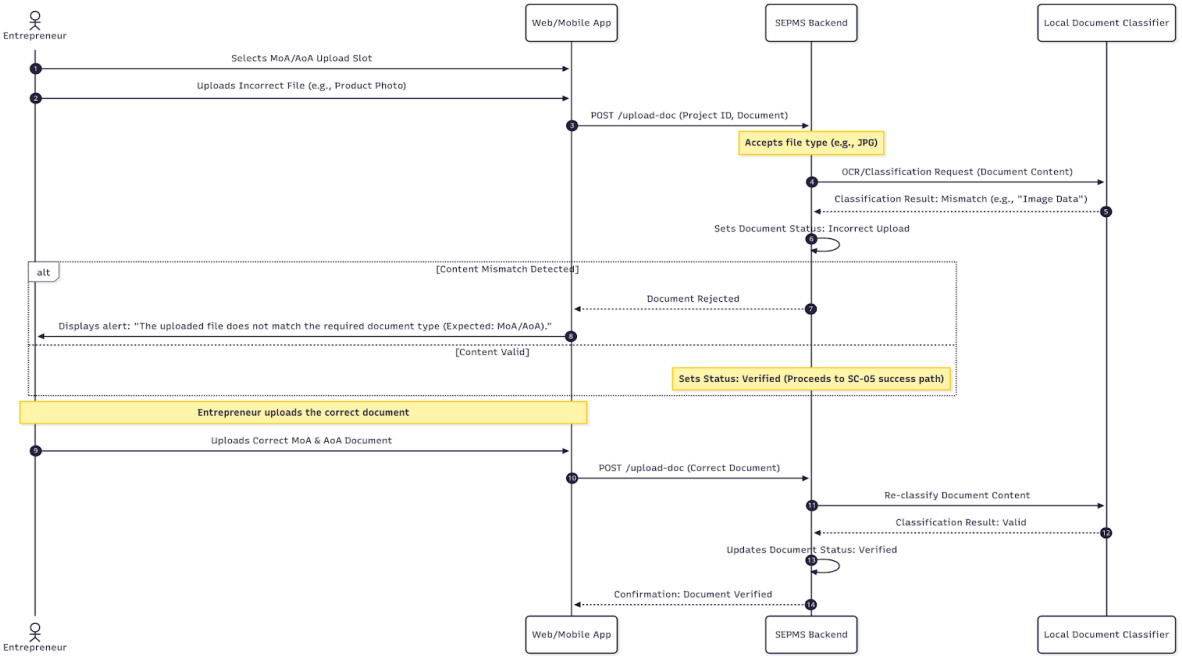


Fig: Sequence diagram: Expired or Outdated Document Submission

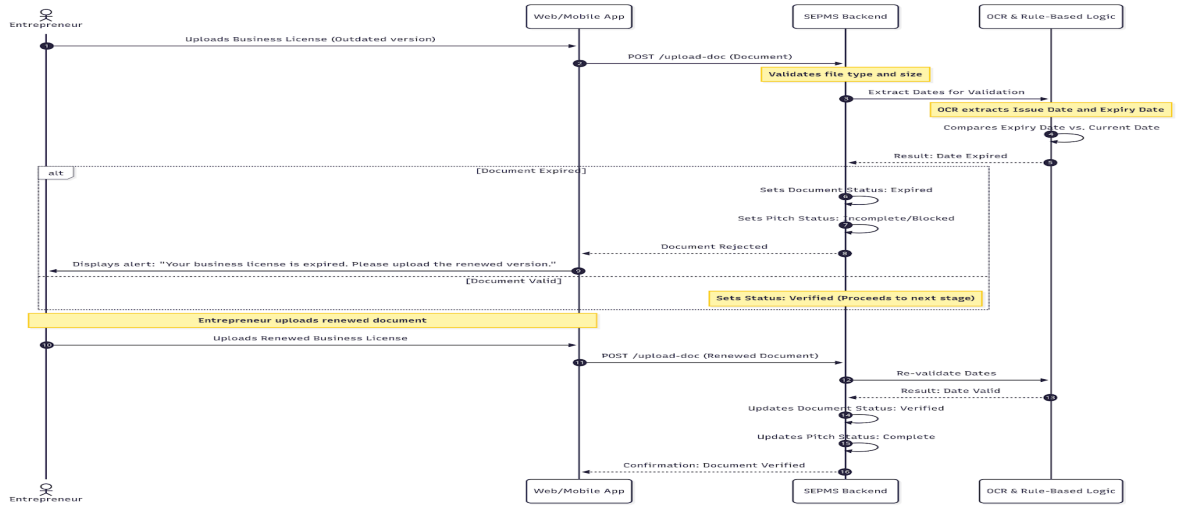


Fig: Sequence diagram: Low-Quality/Unreadable Document Scenario

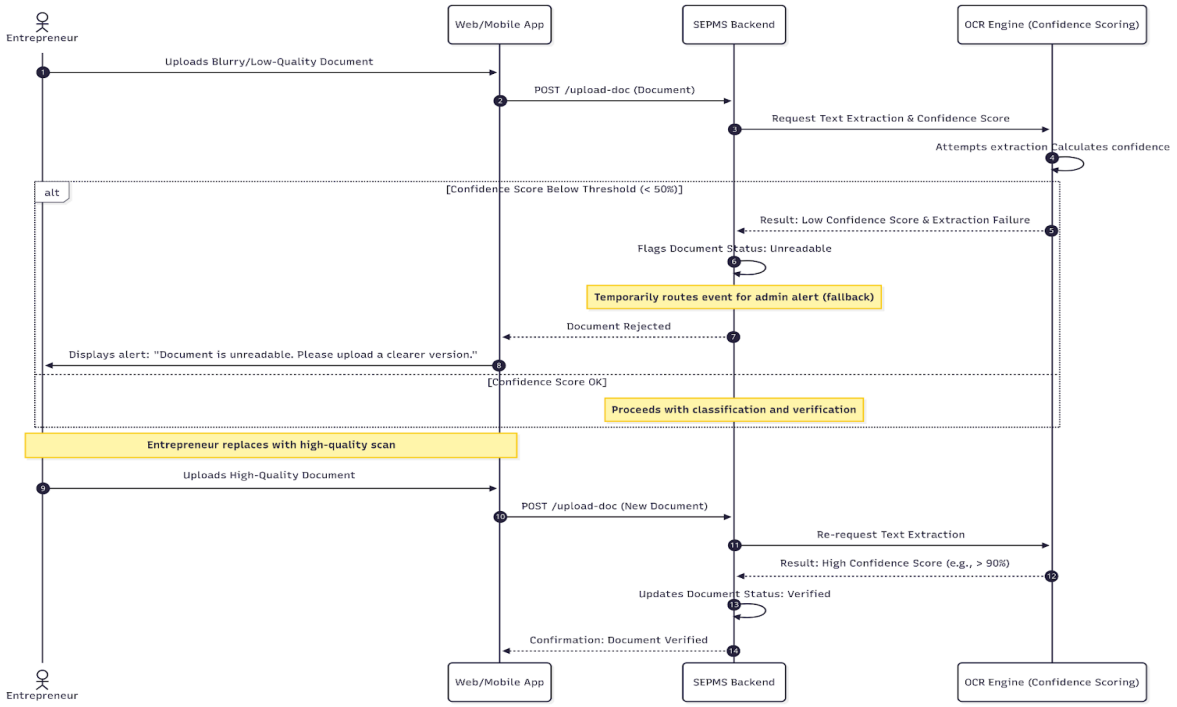


Fig: Sequence diagram: Fraud or Suspicious Document Scenario

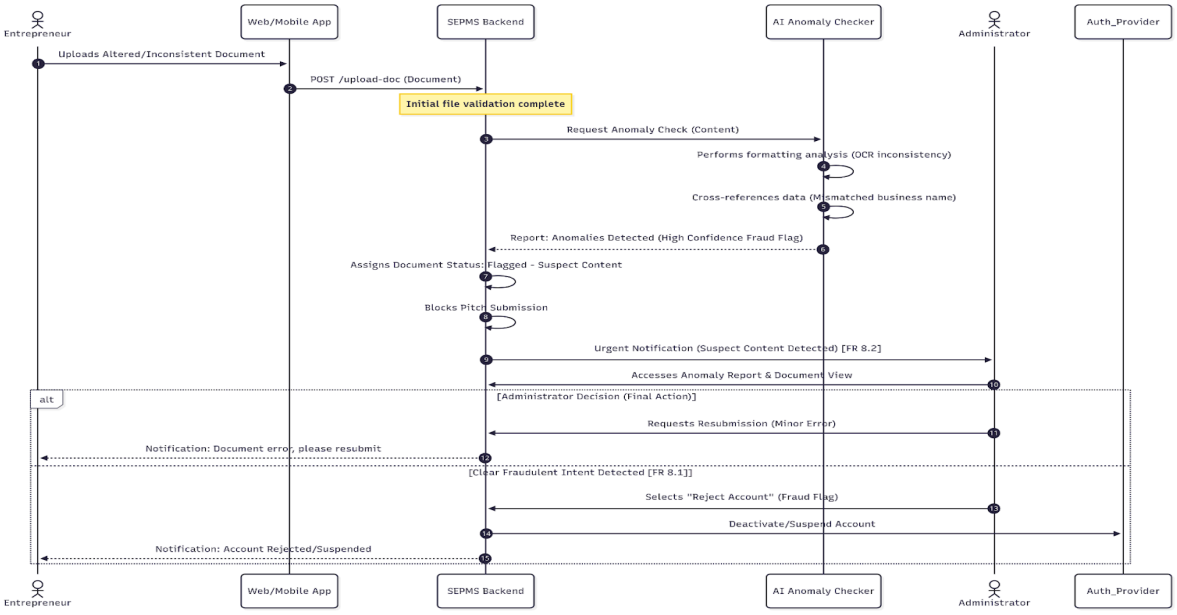


Fig: Sequence diagram: Partial Submission and Draft Saving

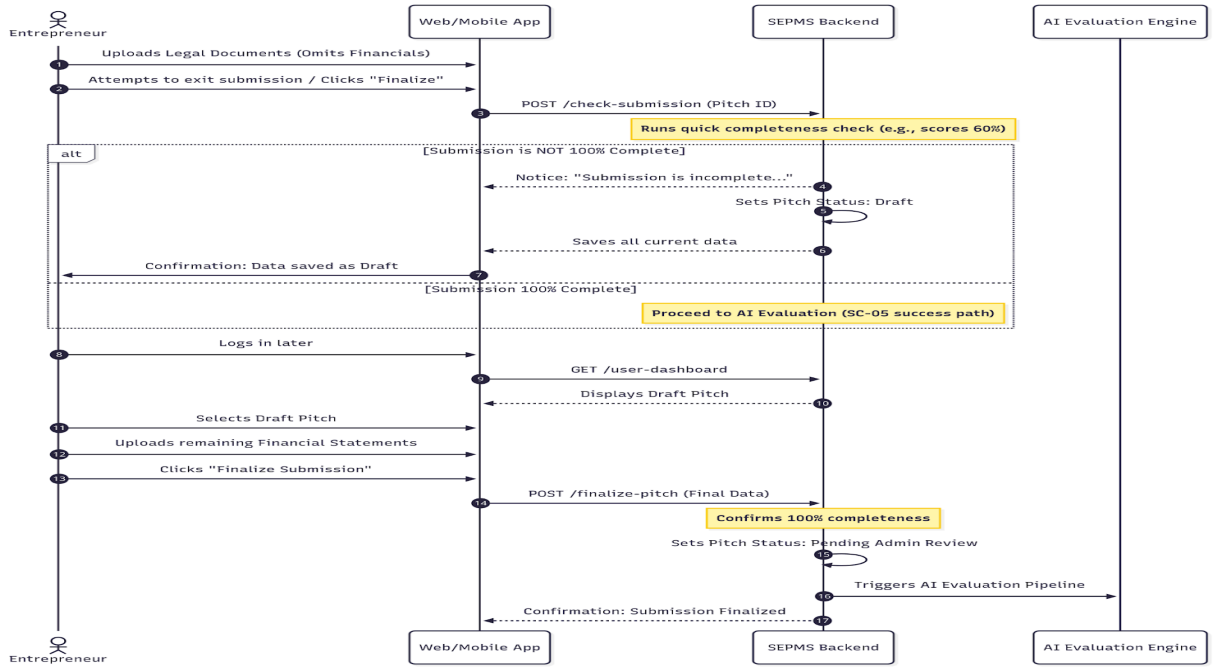


Fig: Sequence diagram: Multistep Upload (Handling Large Media Files)

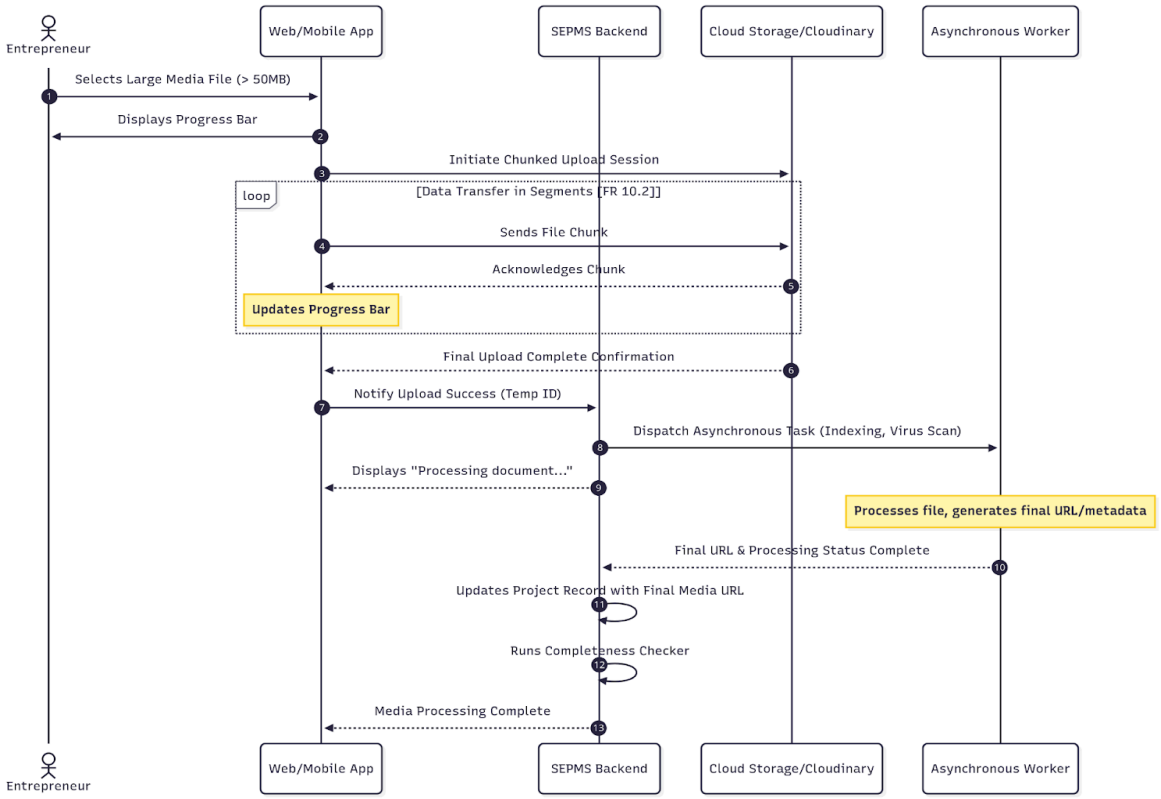


Fig: Sequence diagram: Document Conflict (Name Mismatch)

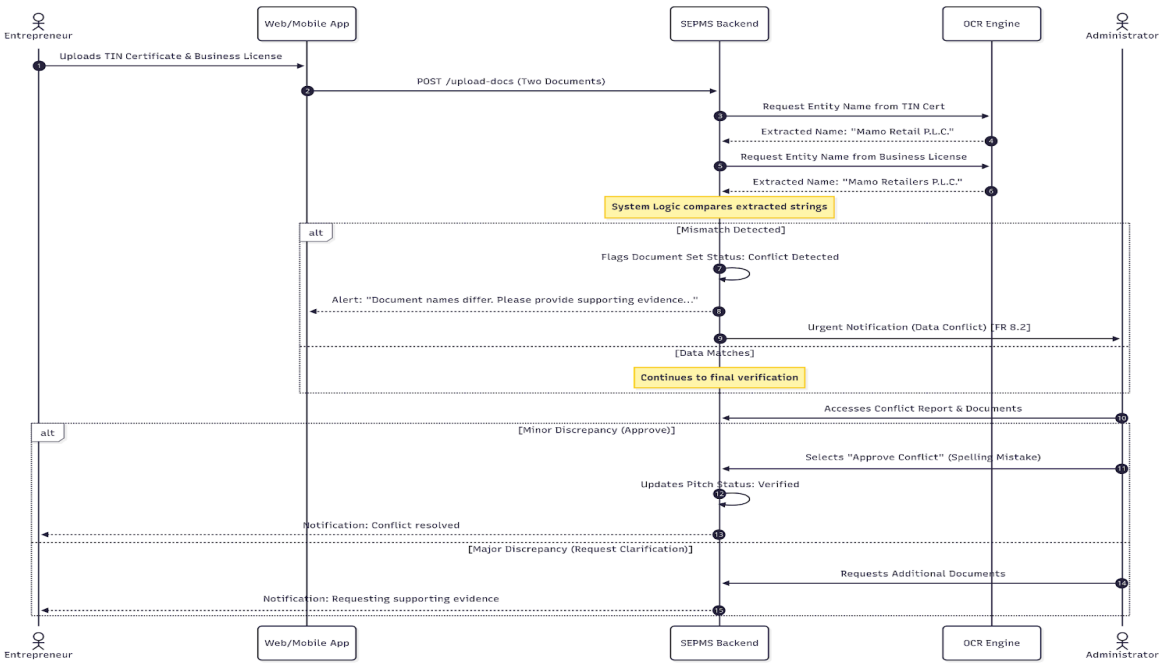


Fig: Sequence diagram: Administrator Correction and AI Override

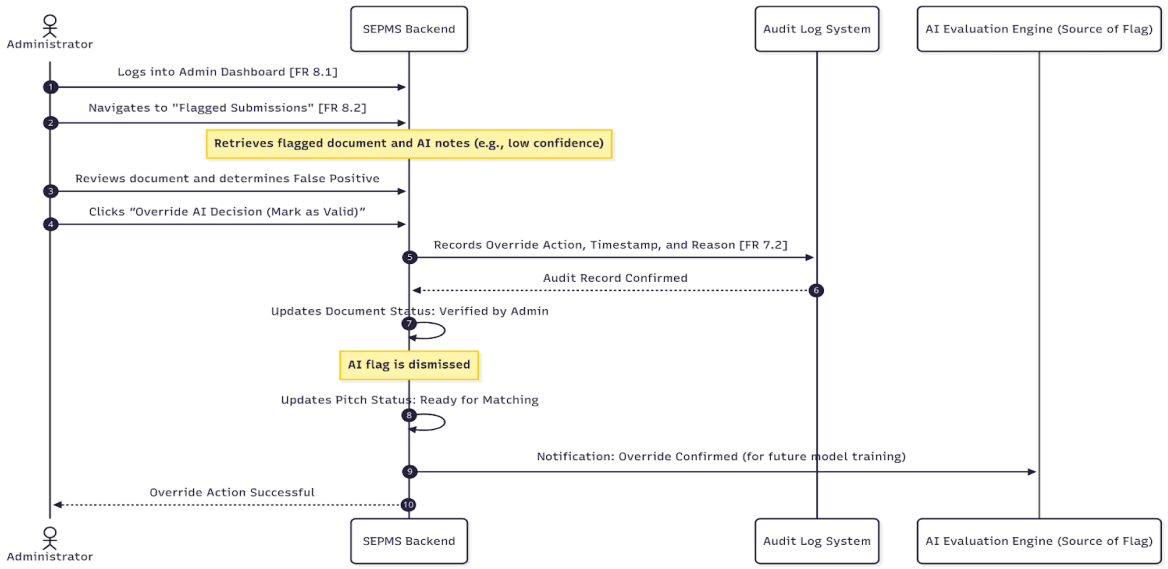


Fig: Sequence diagram: Multi-Business Entrepreneur (Conflict of Entities)

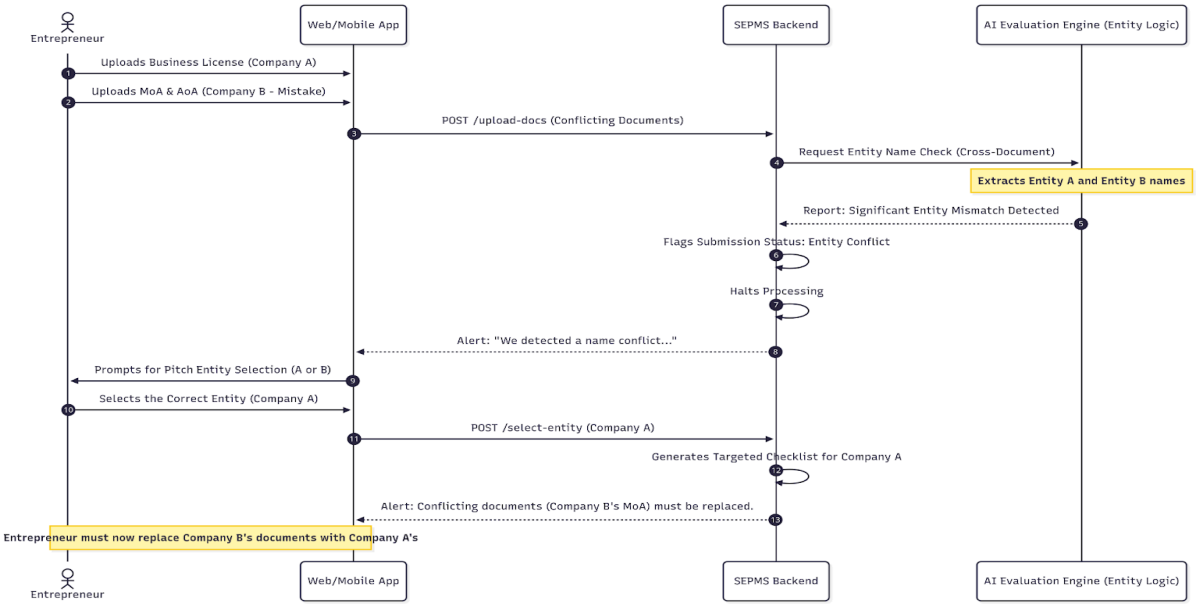


Fig: Sequence diagram: AI Fallback Scenario (Hybrid Checker)

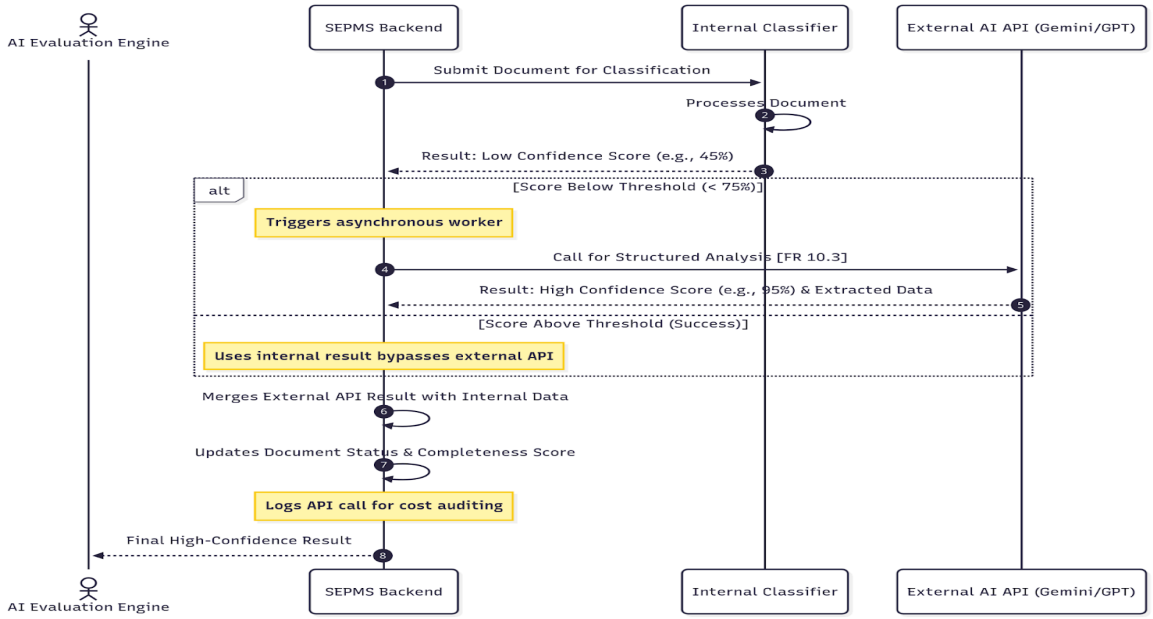


Fig: Sequence diagram: Admin Approves Entrepreneur Submission (Final Gate)

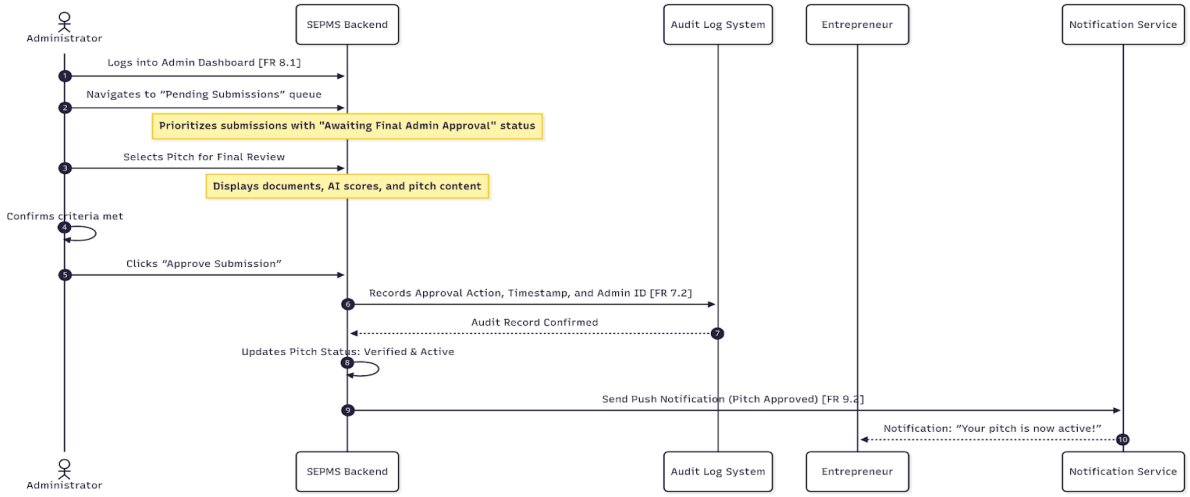


Fig: Sequence diagram: AI Evaluation and Scoring Pipeline

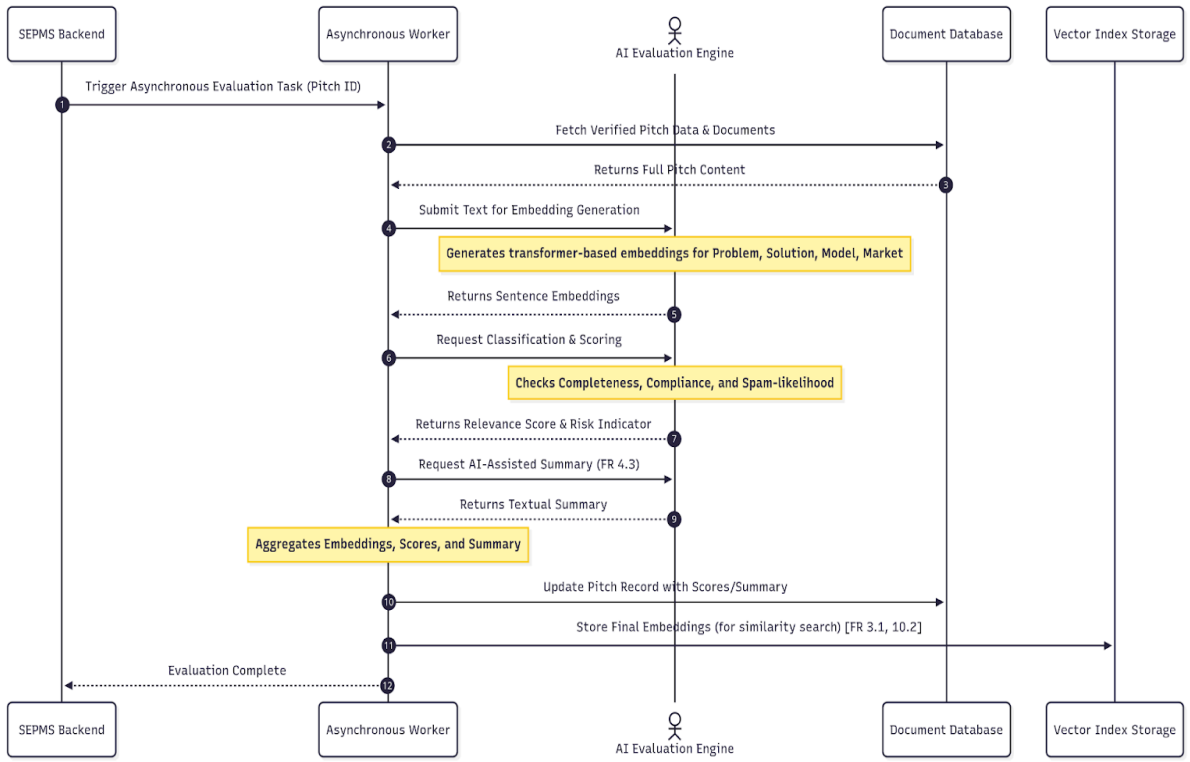


Fig: Sequence diagram: Investor Recommendation, Review, and Voice Output

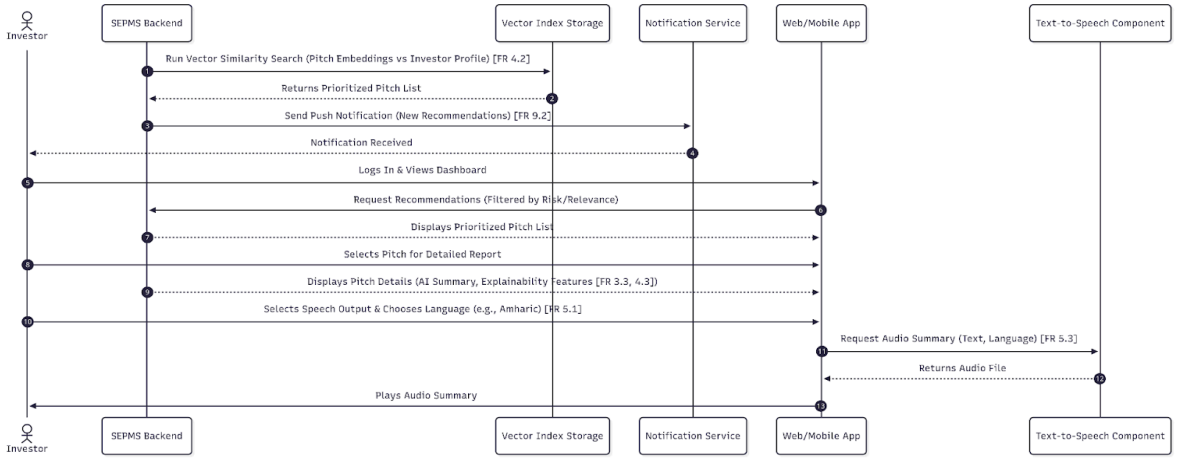
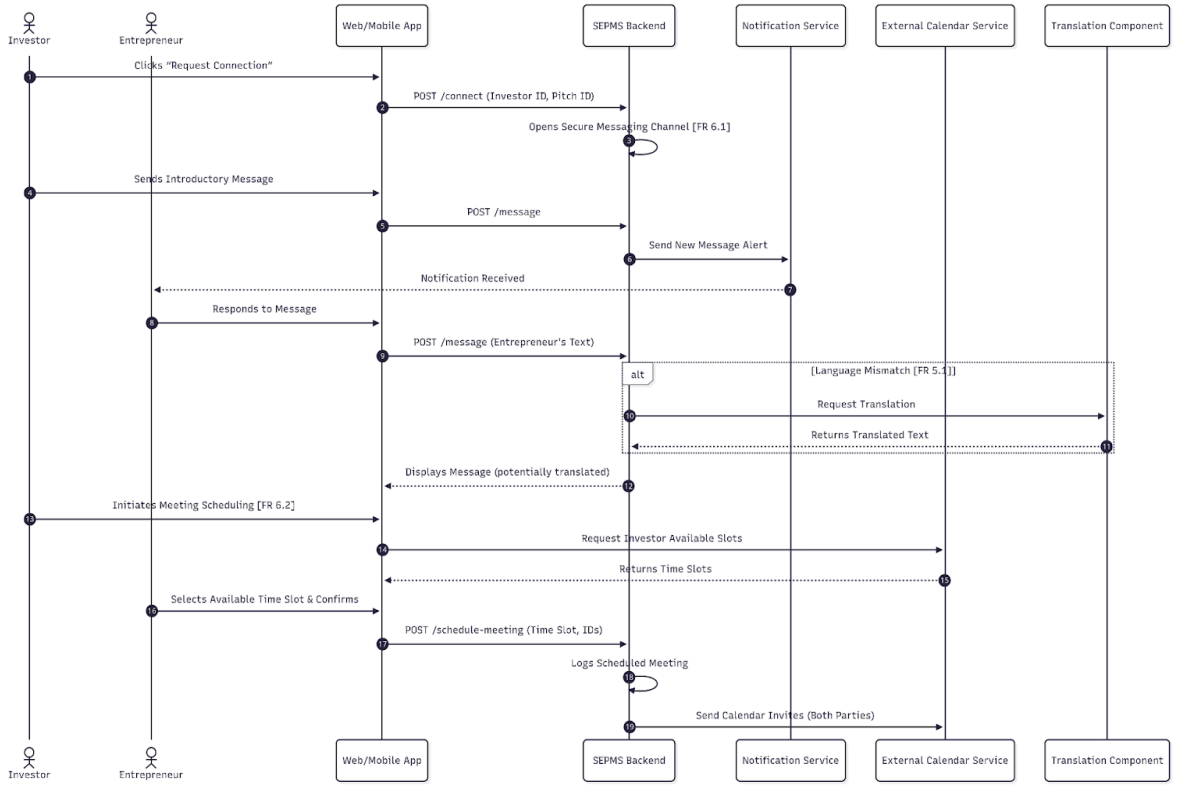


Fig: Sequence diagram: Communication and Meeting Scheduling



Scenario 21

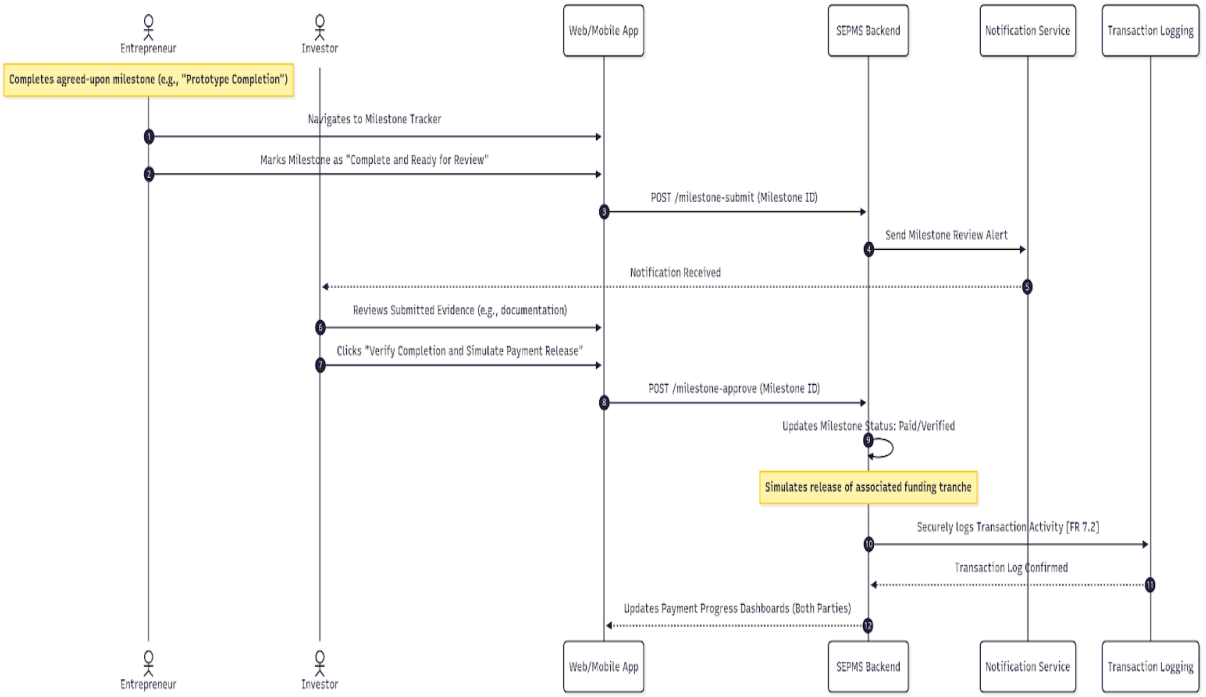


Fig: Sequence diagram: Proposal Rejection (Investor Declines Pitch)

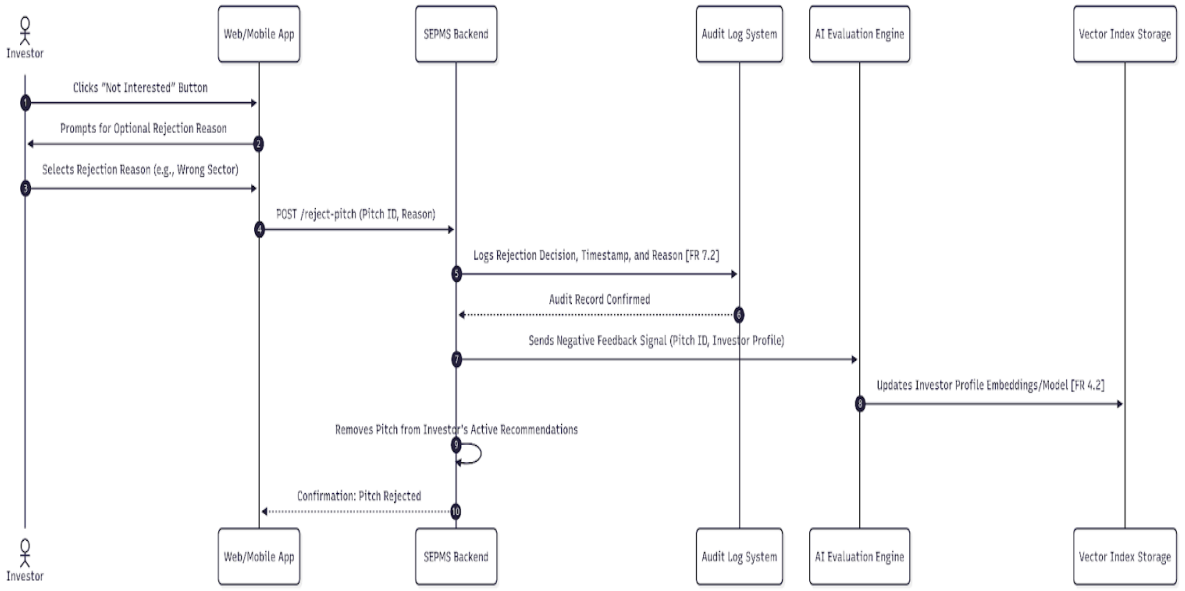


Fig: Sequence diagram: Admin Removal or Suspension of Pitch

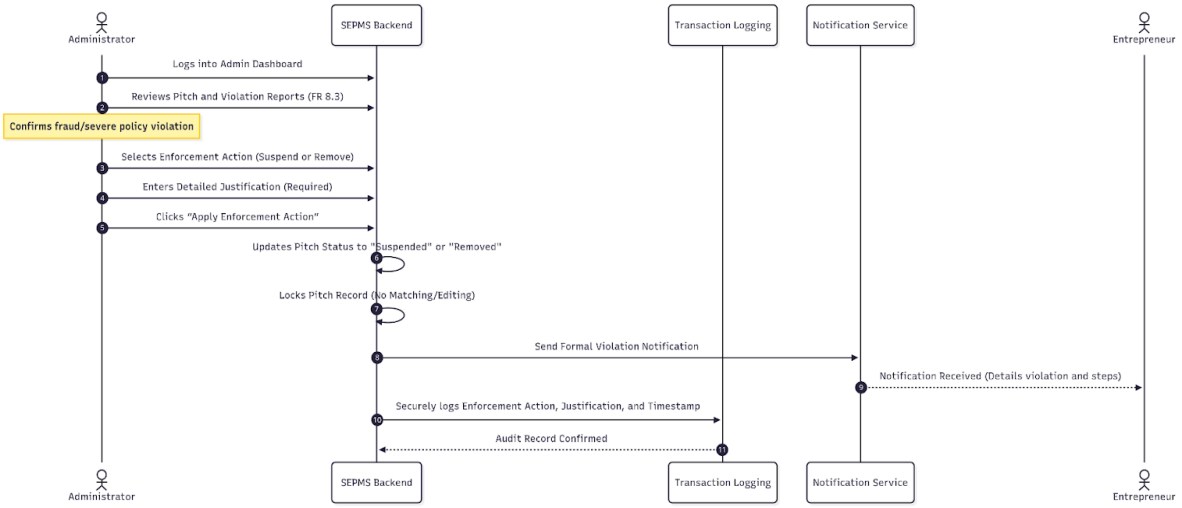


Fig: Sequence diagram: Entrepreneur Account Suspension

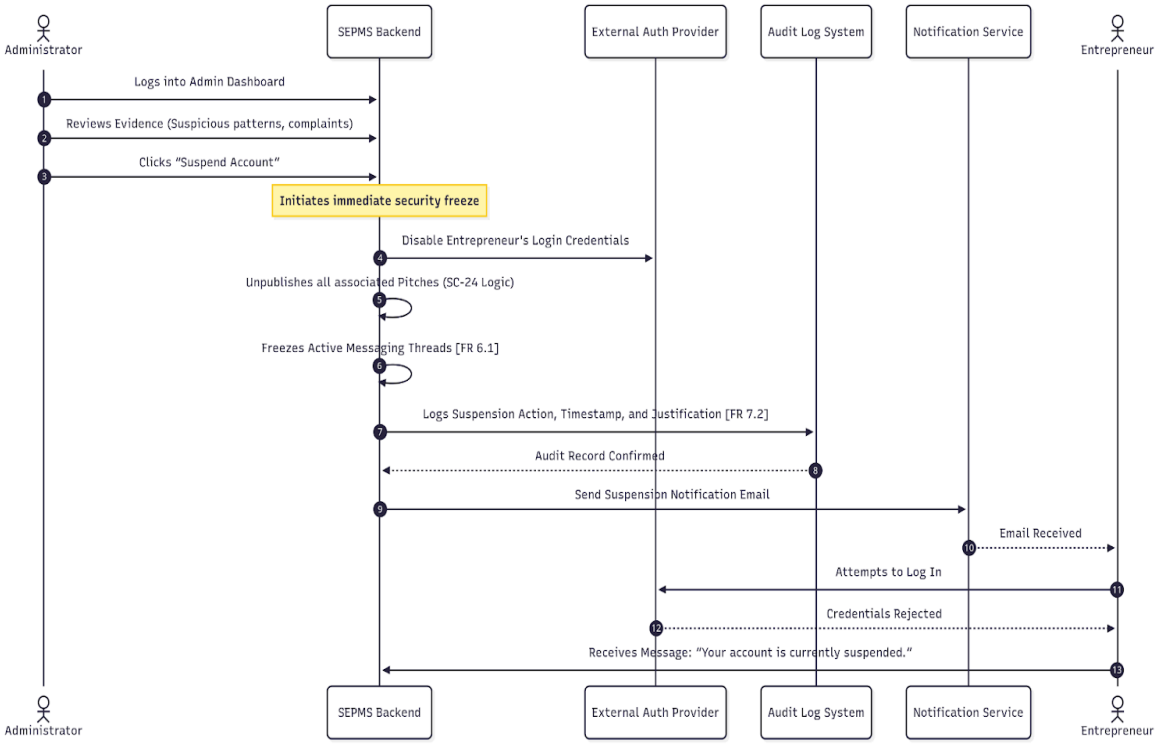


Fig: Sequence diagram: Investor Account Suspension

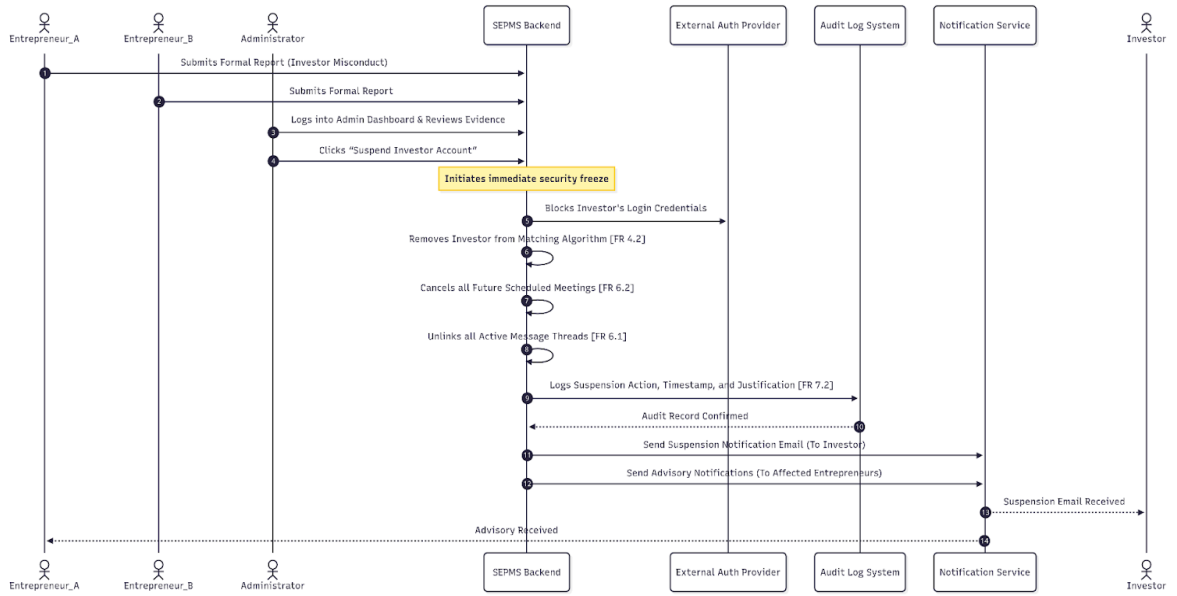


Fig: Sequence diagram: Investor Reports Suspicious Entrepreneur

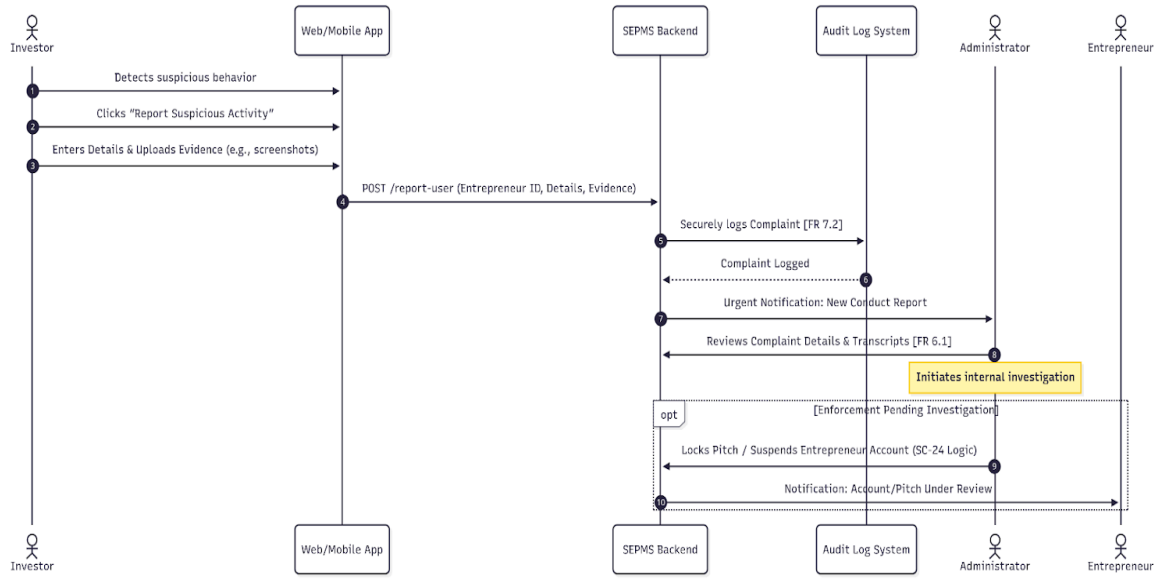
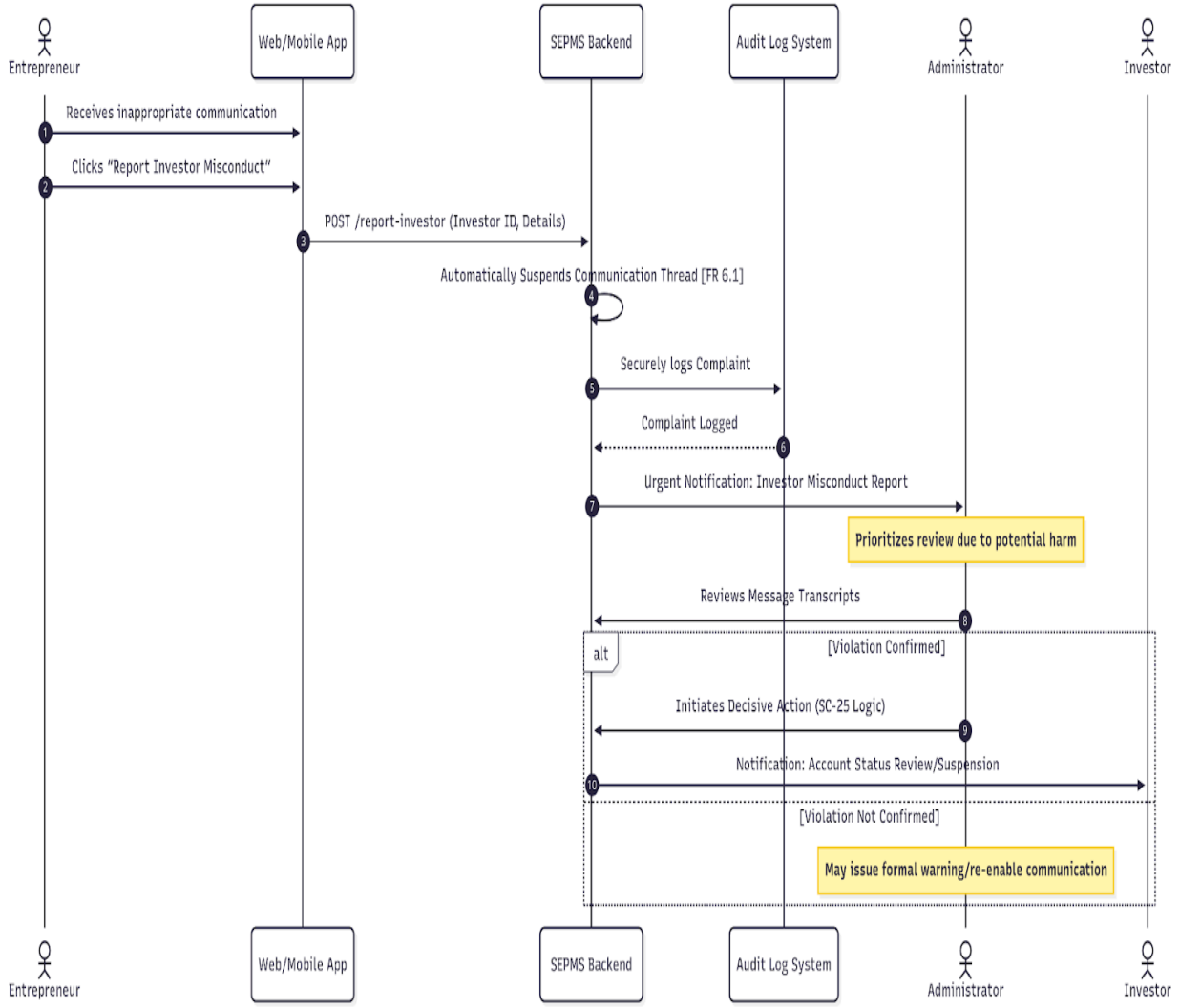


Fig: Sequence diagram: Entrepreneur Reports Suspicious Investor



3.5.5. Fig: Activity diagram

Fig: Activity diagram Scenario 1: Entrepreneur Sign Up & Initial Verification (UC01)

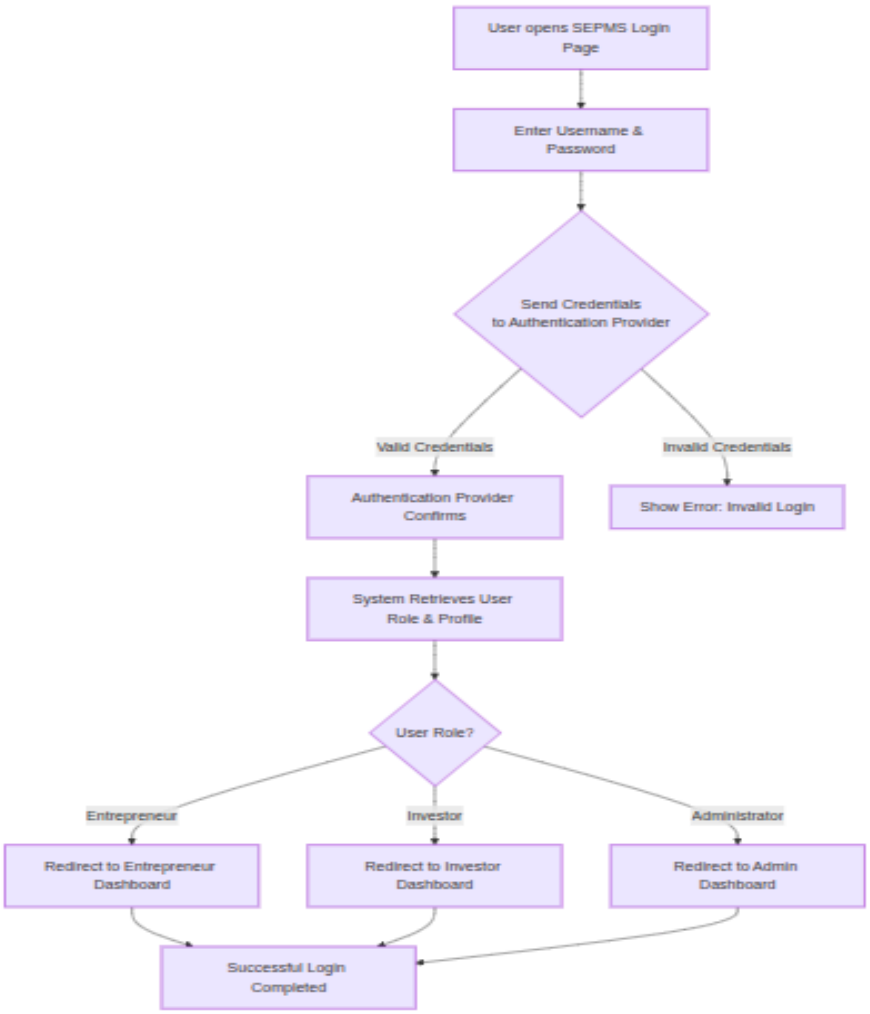


Fig: Activity diagram:- Scenario 2: Investor Sign Up & Financial Verification (UC02)

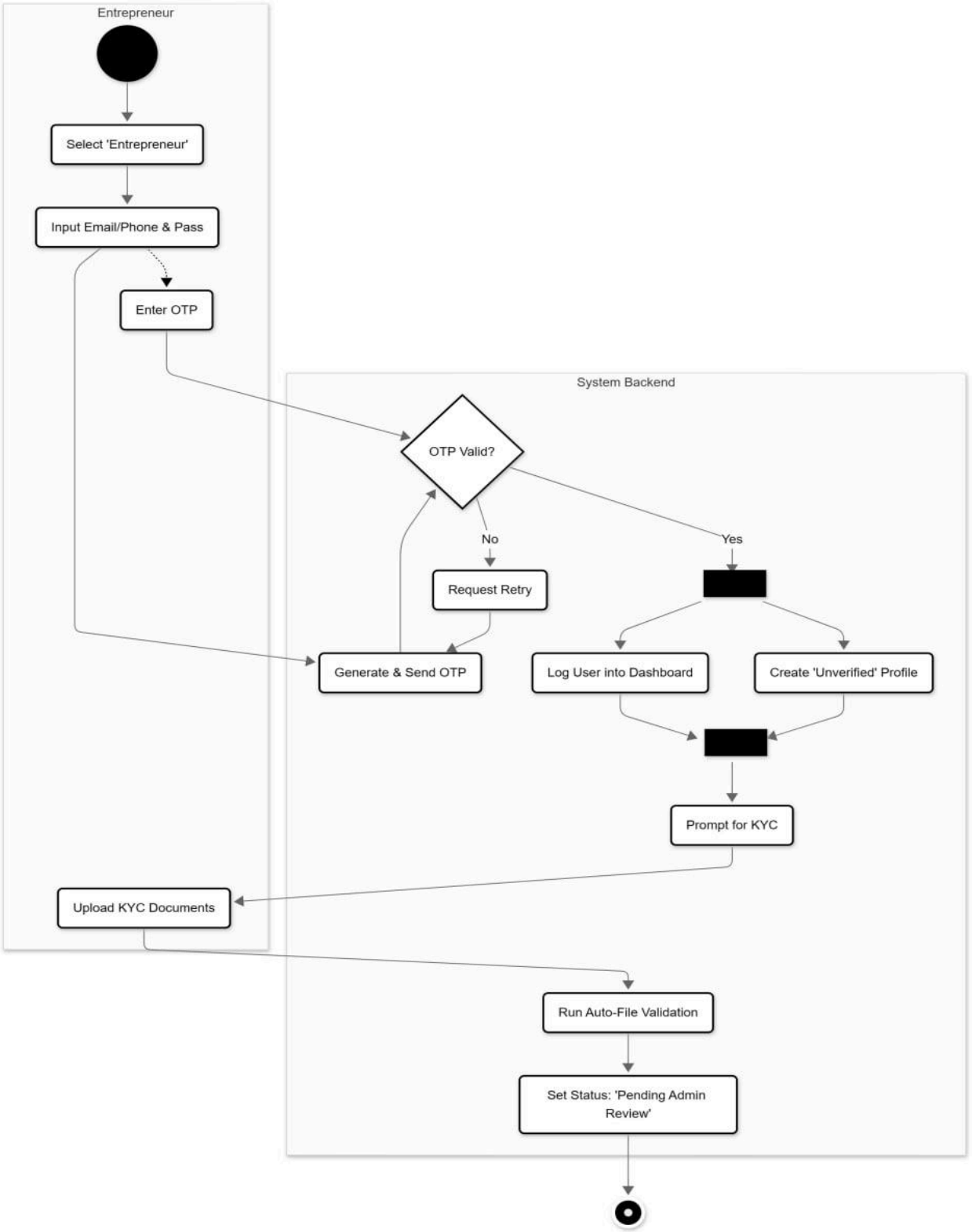


Fig: Activity diagram:- Scenario 3: Successful User Authentication (Login)

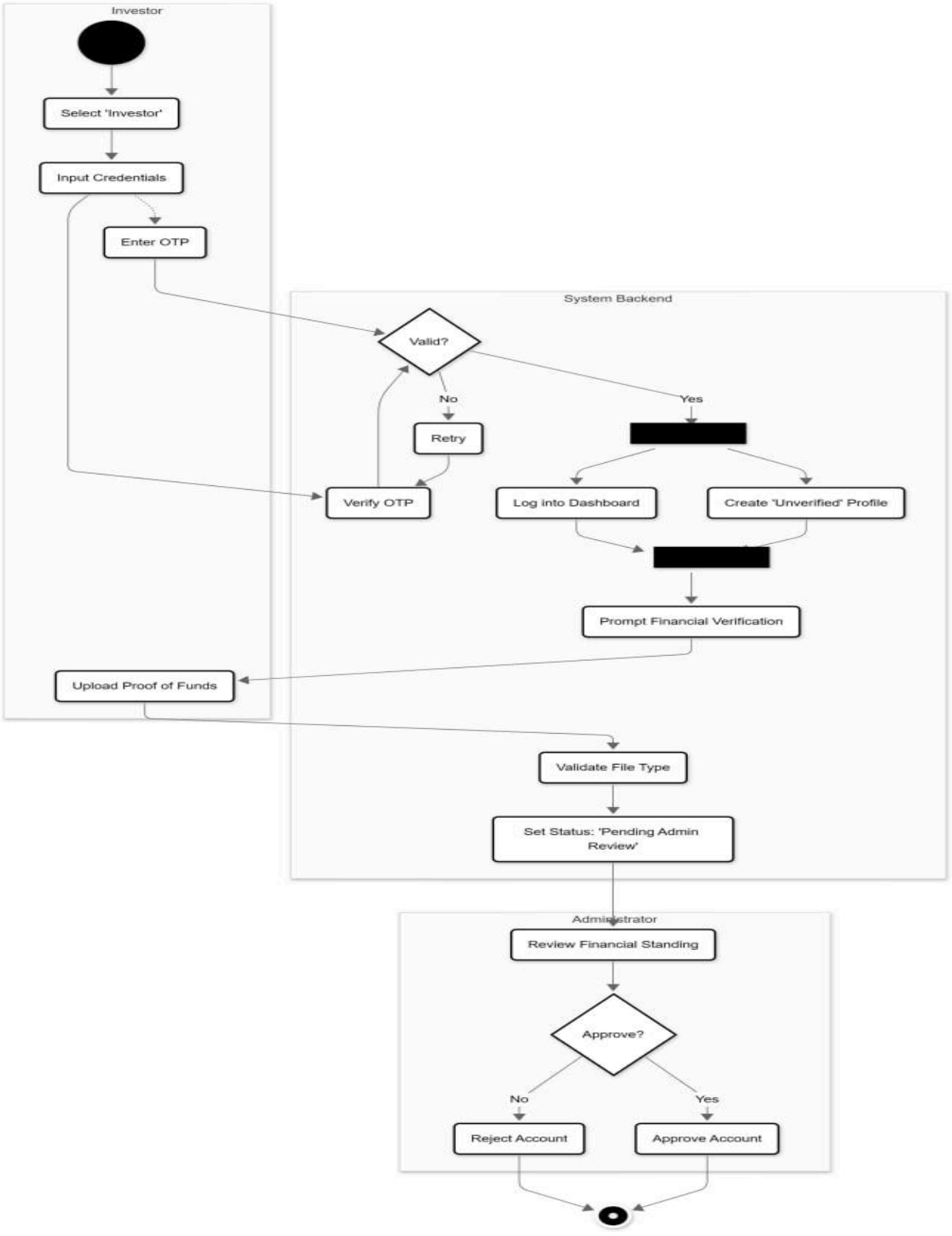


Fig: Activity diagram:- Scenario 4: Password Recovery (Forgot Password)

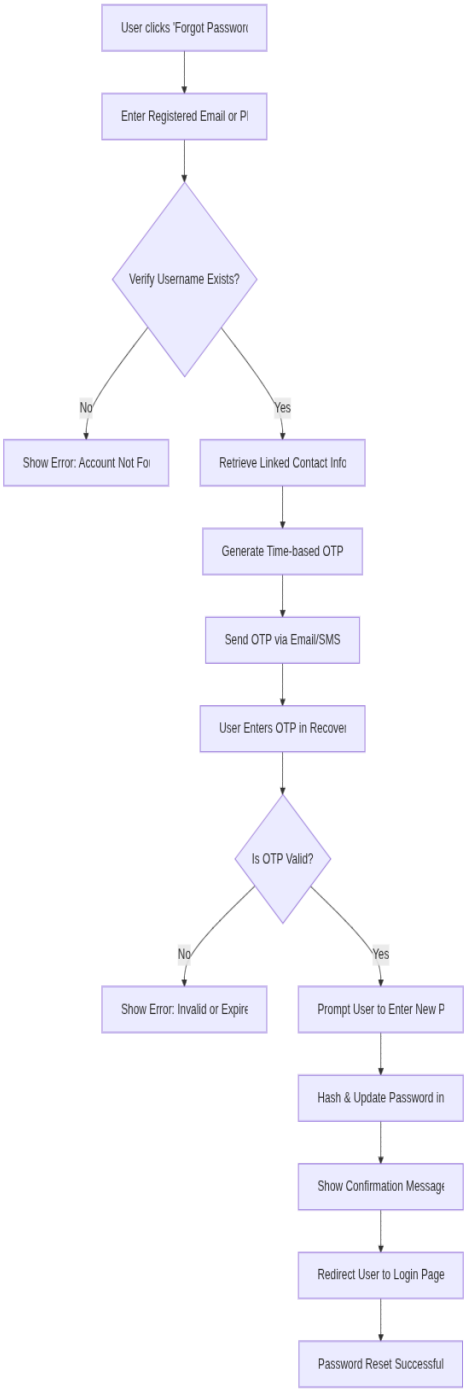


Fig: Activity diagram:- Scenario 5: Full Pitch Creation and AI Submission (UC02)

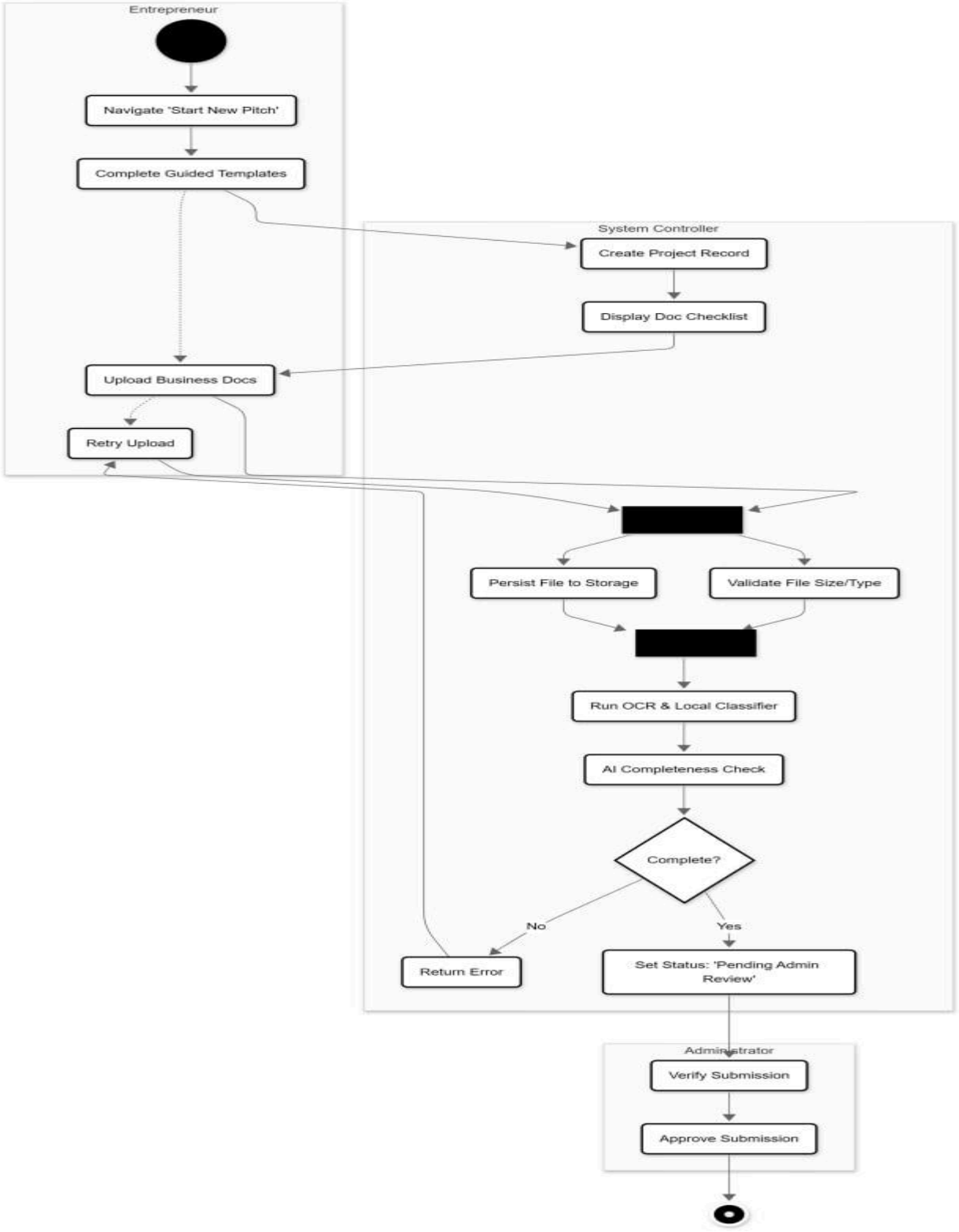


Fig: Activity diagram:- Scenario 6: Missing Required Documents

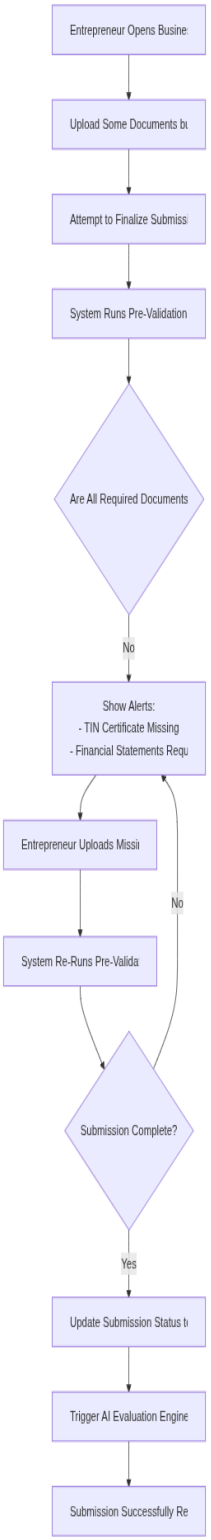


Fig: Activity diagram:- Scenario 7: Wrong Document Uploaded (Content Mismatch)

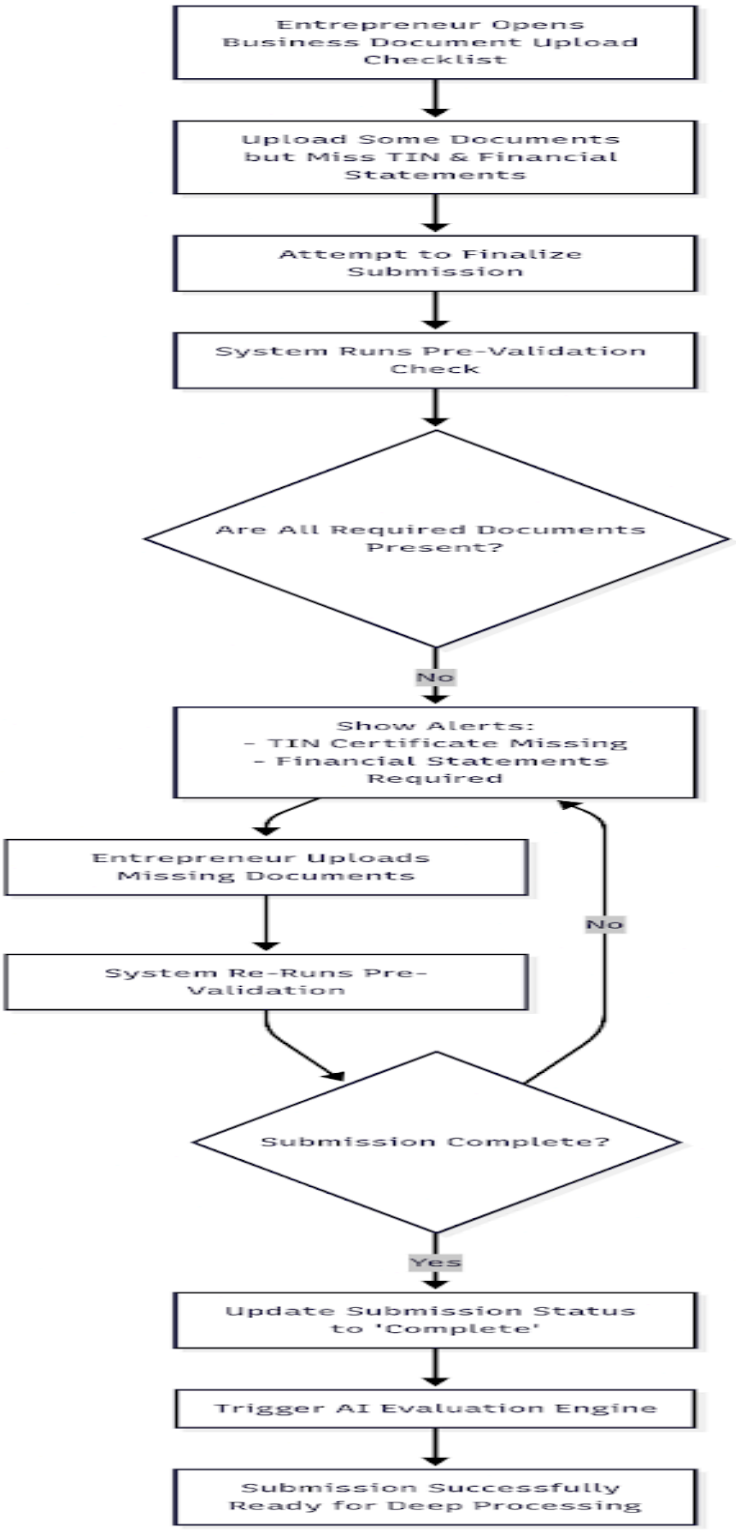


Fig: Activity diagram:- Scenario 8: Expired or Outdated Document Submission

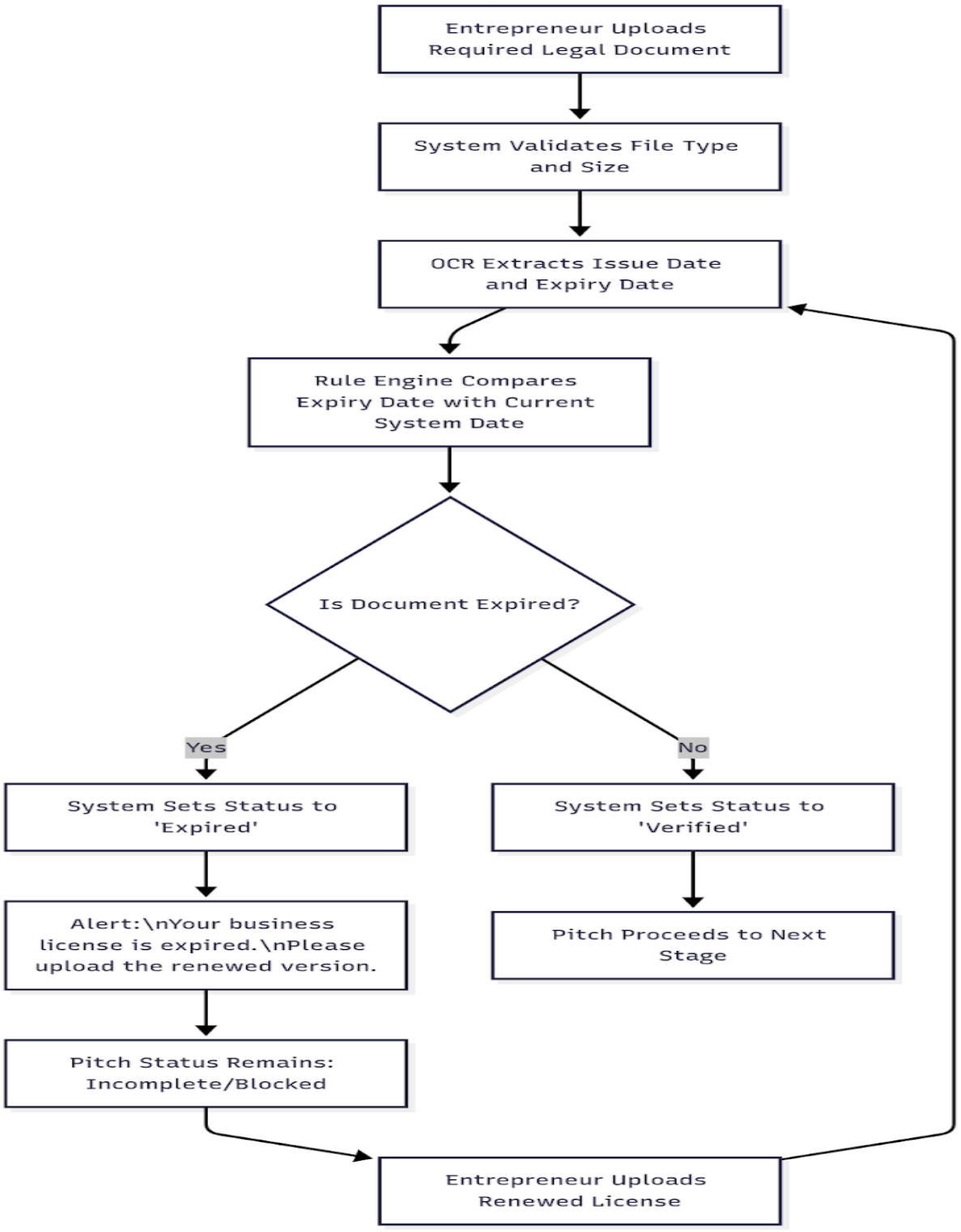

Entrepreneur Uploads Required Legal Document
System Validates File Type and Size
OCR Extracts Issue Date and Expiry Date
Rule Engine Compares Expiry Date with Current System Date
Is Document Expired?
Yes
No
System Sets Status to 'Expired'
Alert:\nYour business license is expired.\nPlease upload the renewed version.
Pitch Status Remains: Incomplete/Blocked
System Sets Status to 'Verified'
Pitch Proceeds to Next Stage
Entrepreneur Uploads Renewed License

Fig: Activity diagram:- Scenario 9: Low-Quality/Unreadable Document Scenario

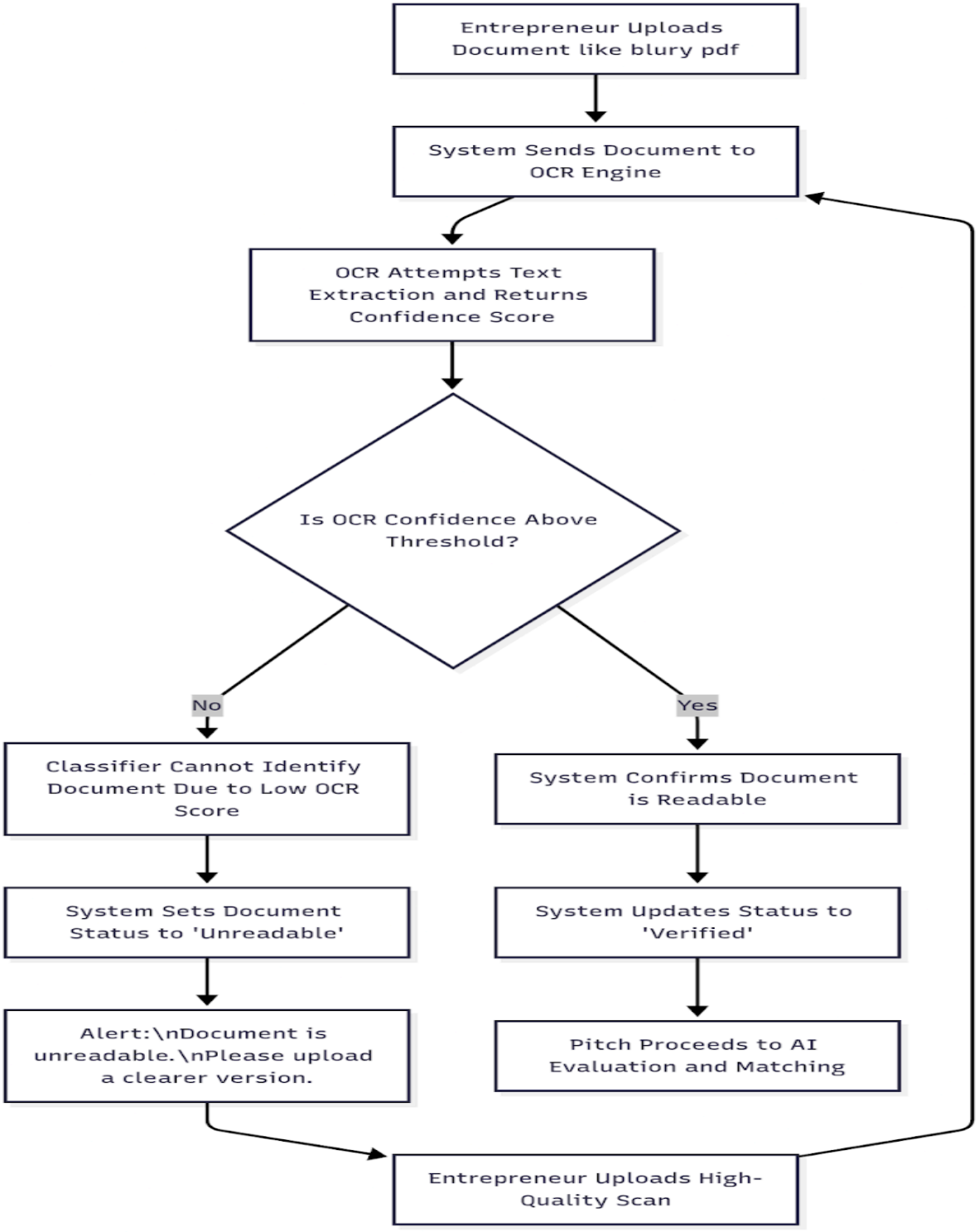


Fig: Activity diagram:- Scenario 10: Fraud or Suspicious Document Scenario

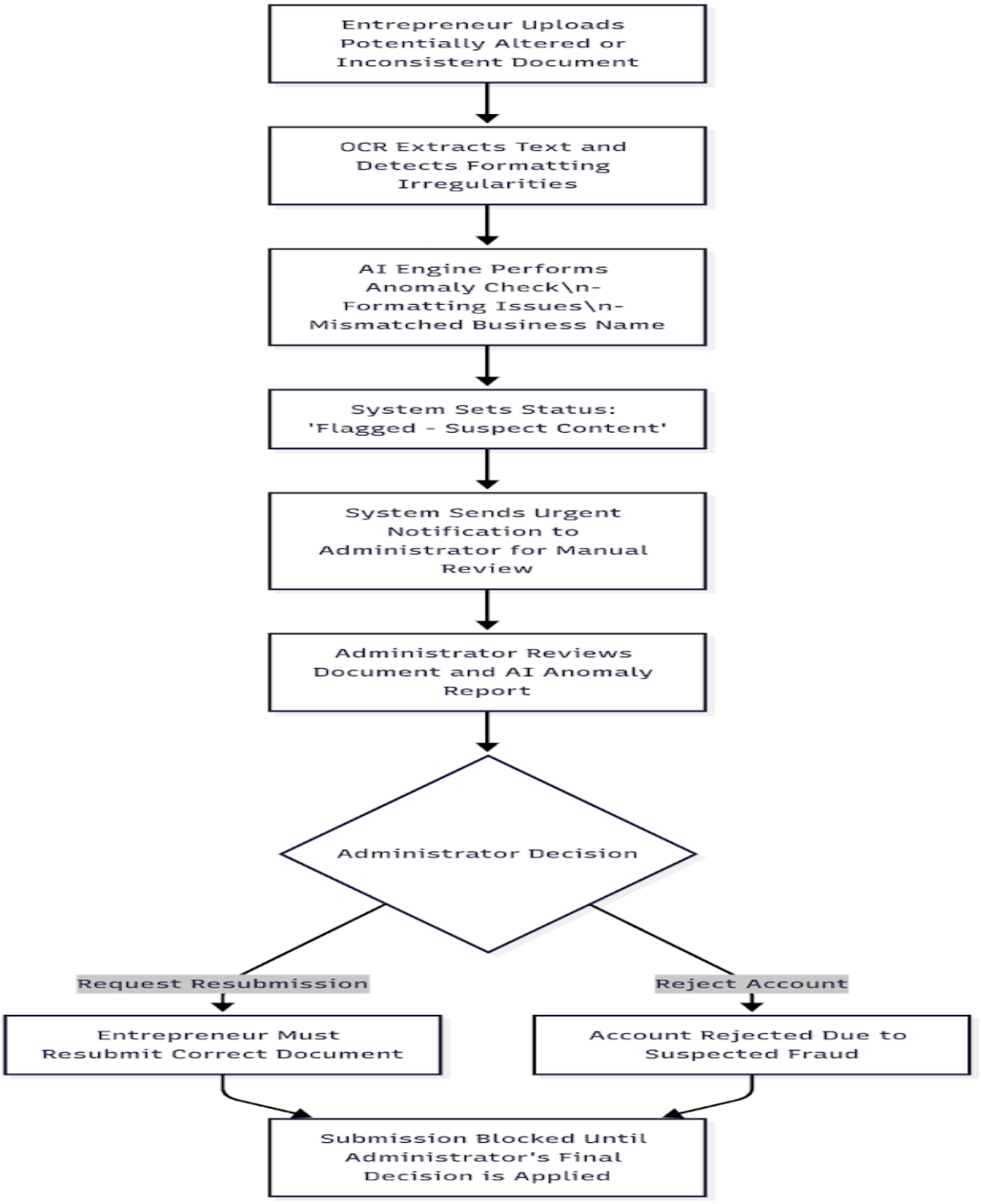


Fig: Activity diagram:- Scenario 11: Partial Submission and Draft Saving

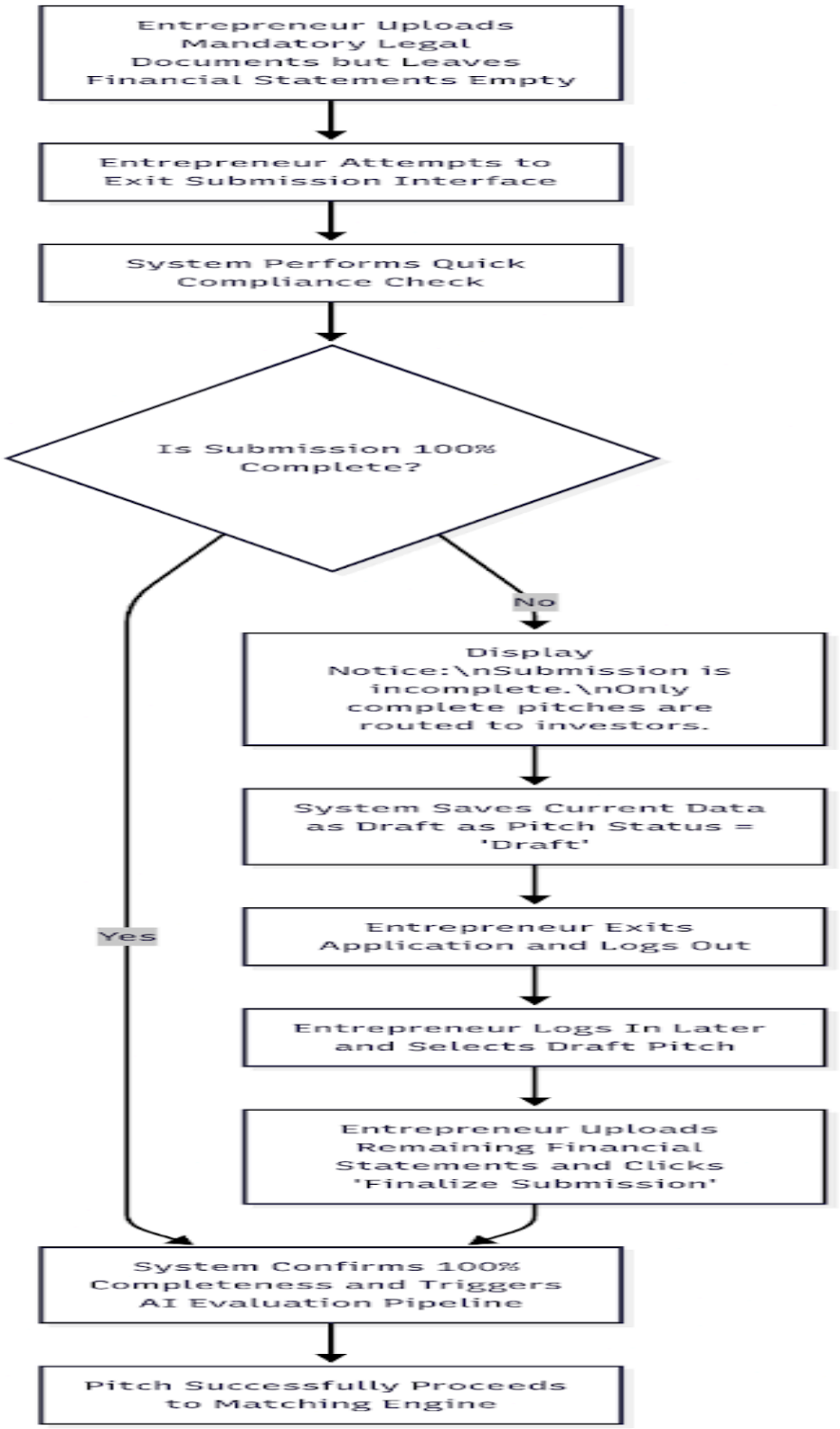

Entrepreneur Uploads Mandatory Legal Documents but Leaves Financial Statements Empty
Entrepreneur Attempts to Exit Submission Interface
System Performs Quick Compliance Check
Is Submission 100% Complete?
No
Yes
Display Notice:\nSubmission is incomplete.\nOnly complete pitches are routed to investors.
System Saves Current Data as Draft as Pitch Status = 'Draft'
Entrepreneur Exits Application and Logs Out
Entrepreneur Logs In Later and Selects Draft Pitch
Entrepreneur Uploads Remaining Financial Statements and Clicks 'Finalize Submission'
System Confirms 100% Completeness and Triggers AI Evaluation Pipeline
Pitch Successfully Proceeds to Matching Engine

Fig: Activity diagram:- Scenario 12: Multistep Upload (Handling Large Media Files)

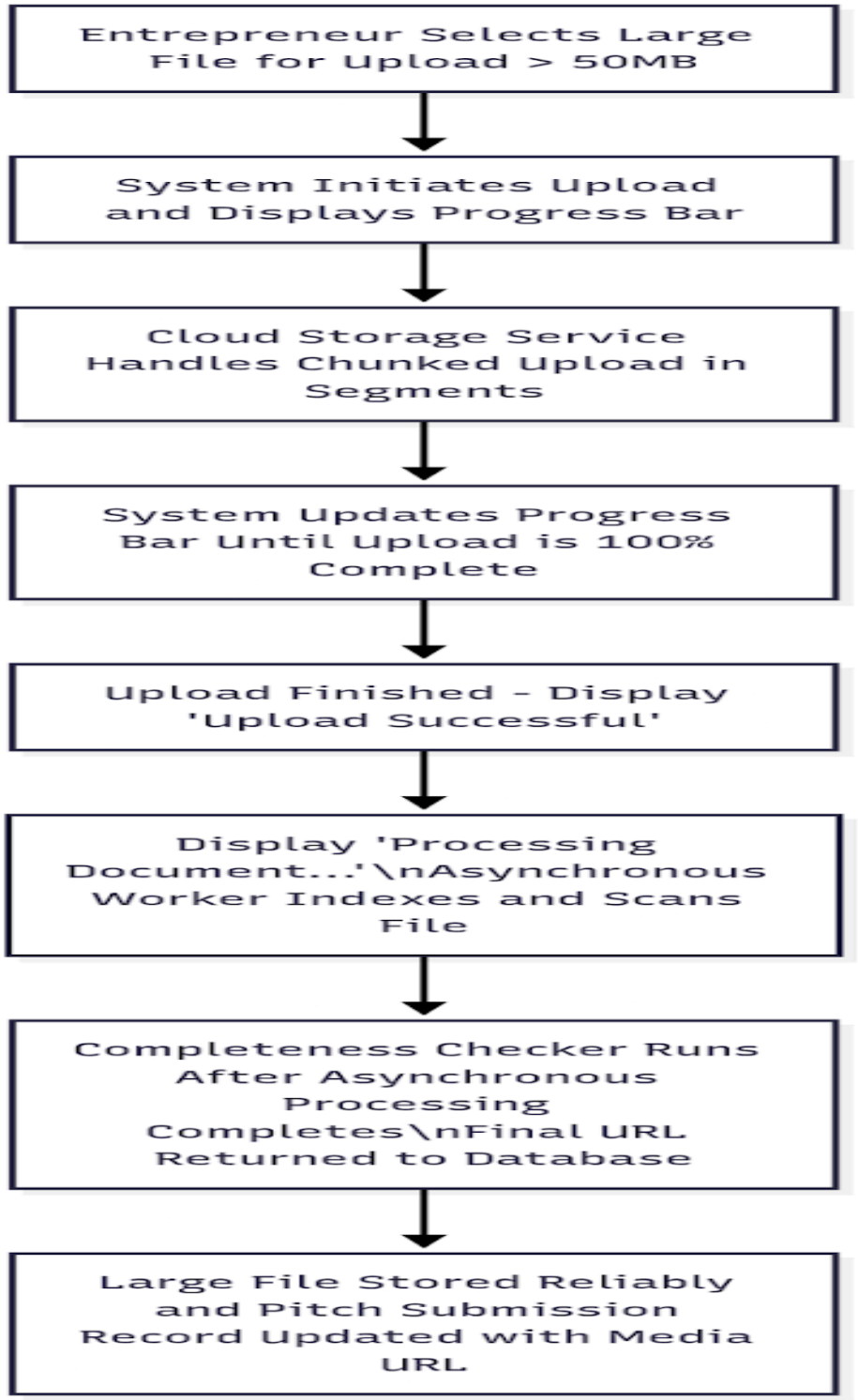


Fig: Activity diagram:- Scenario 13: Document Conflict (Name Mismatch)

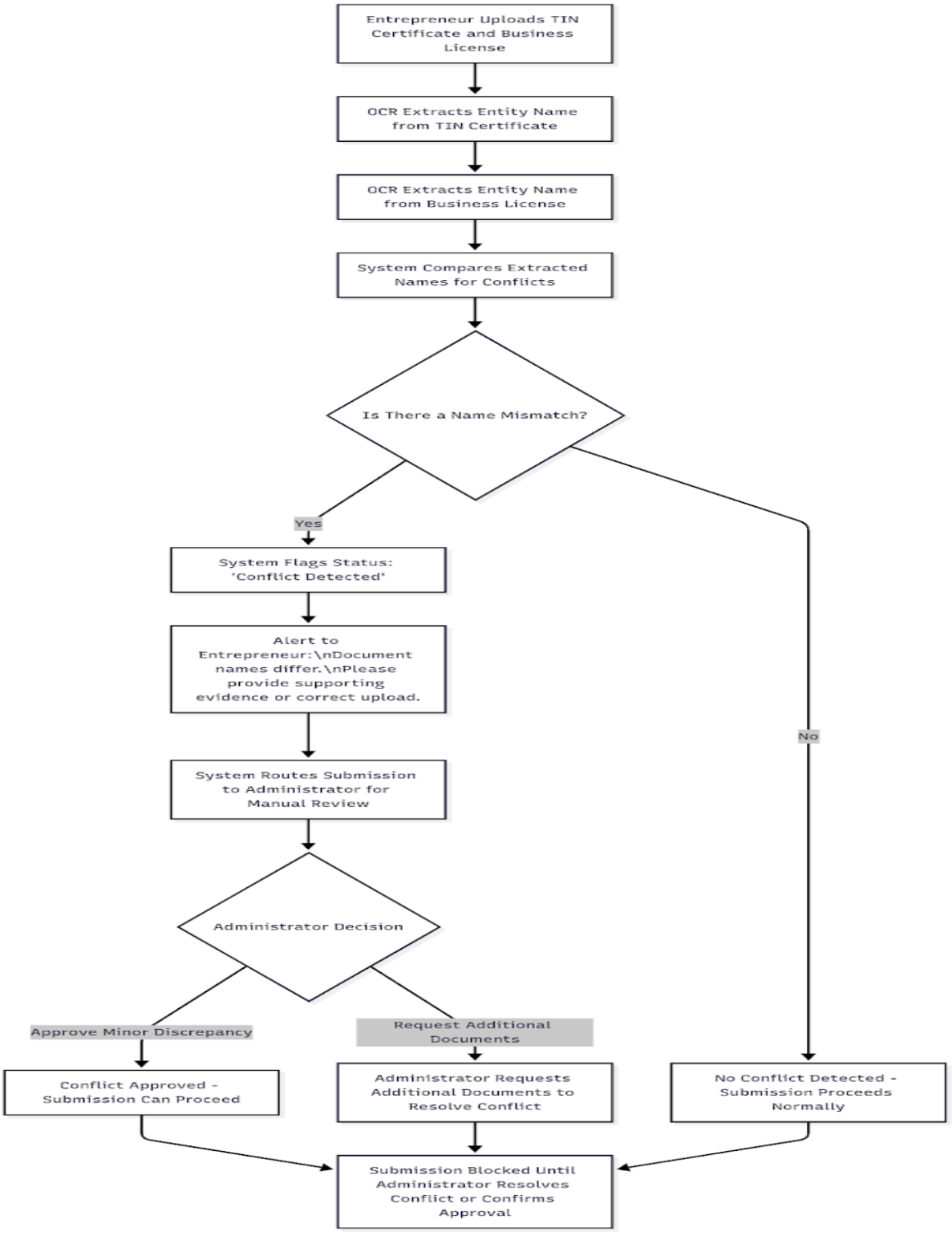

Entrepreneur Uploads TIN Certificate and Business License
OCR Extracts Entity Name from TIN Certificate
OCR Extracts Entity Name from Business License
System Compares Extracted Names for Conflicts
Is There a Name Mismatch?
Yes
No
System Flags Status: 'Conflict Detected'
Alert to Entrepreneur:\nDocument names differ.\nPlease provide supporting evidence or correct upload.
System Routes Submission to Administrator for Manual Review
Administrator Decision
Approve Minor Discrepancy
Request Additional Documents
Conflict Approved - Submission Can Proceed
Administrator Requests Additional Documents to Resolve Conflict
No Conflict Detected - Submission Proceeds Normally
Submission Blocked Until Administrator Resolves Conflict or Confirms Approval

Fig: Activity diagram:- Scenario 14: Administrator Correction and AI Override

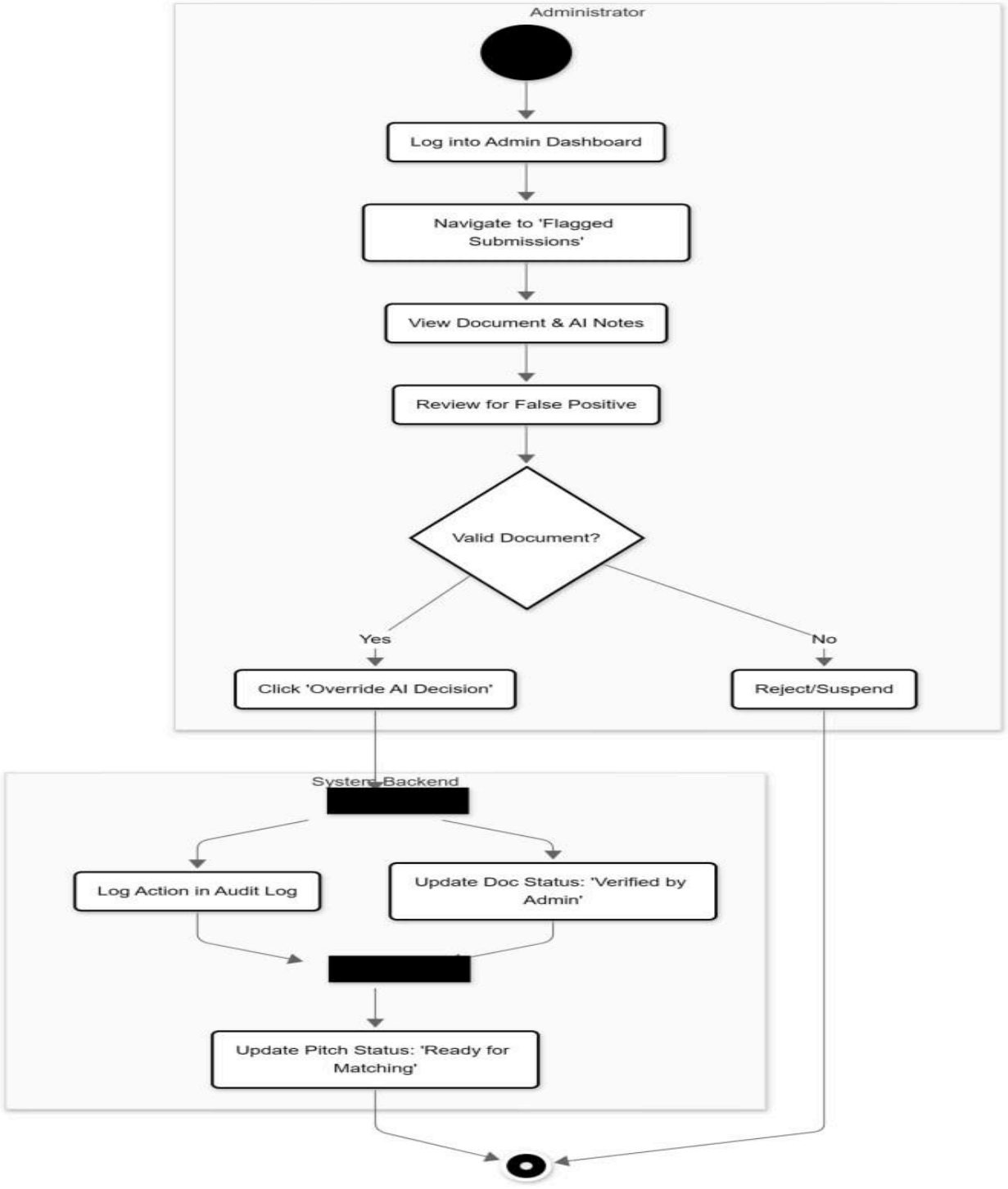


Fig: Activity diagram:- Scenario 15: Multi-Business Entrepreneur (Conflict of Entities)

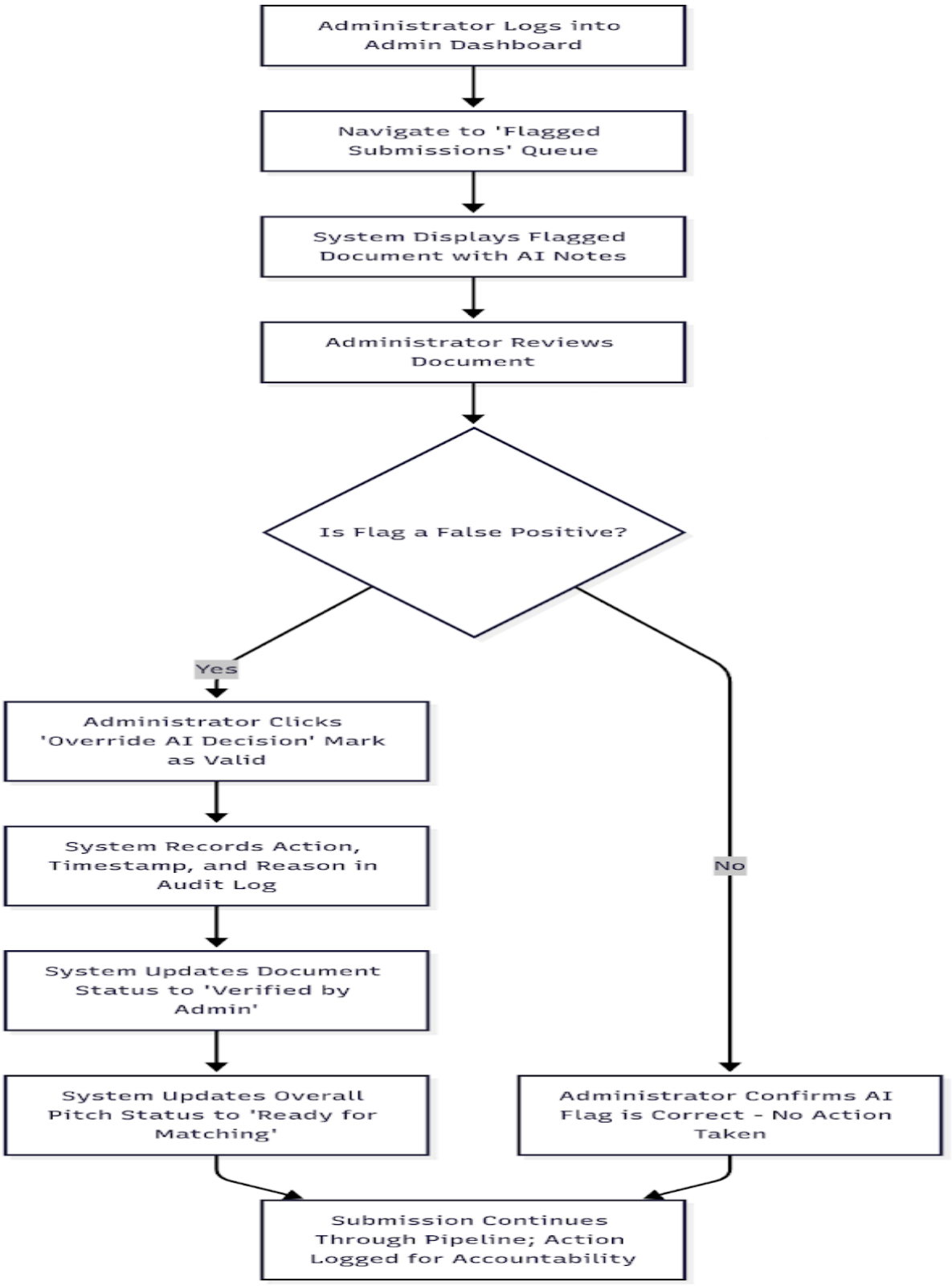


Fig: Activity diagram:- Scenario 16: AI Fallback Scenario (Hybrid Checker)

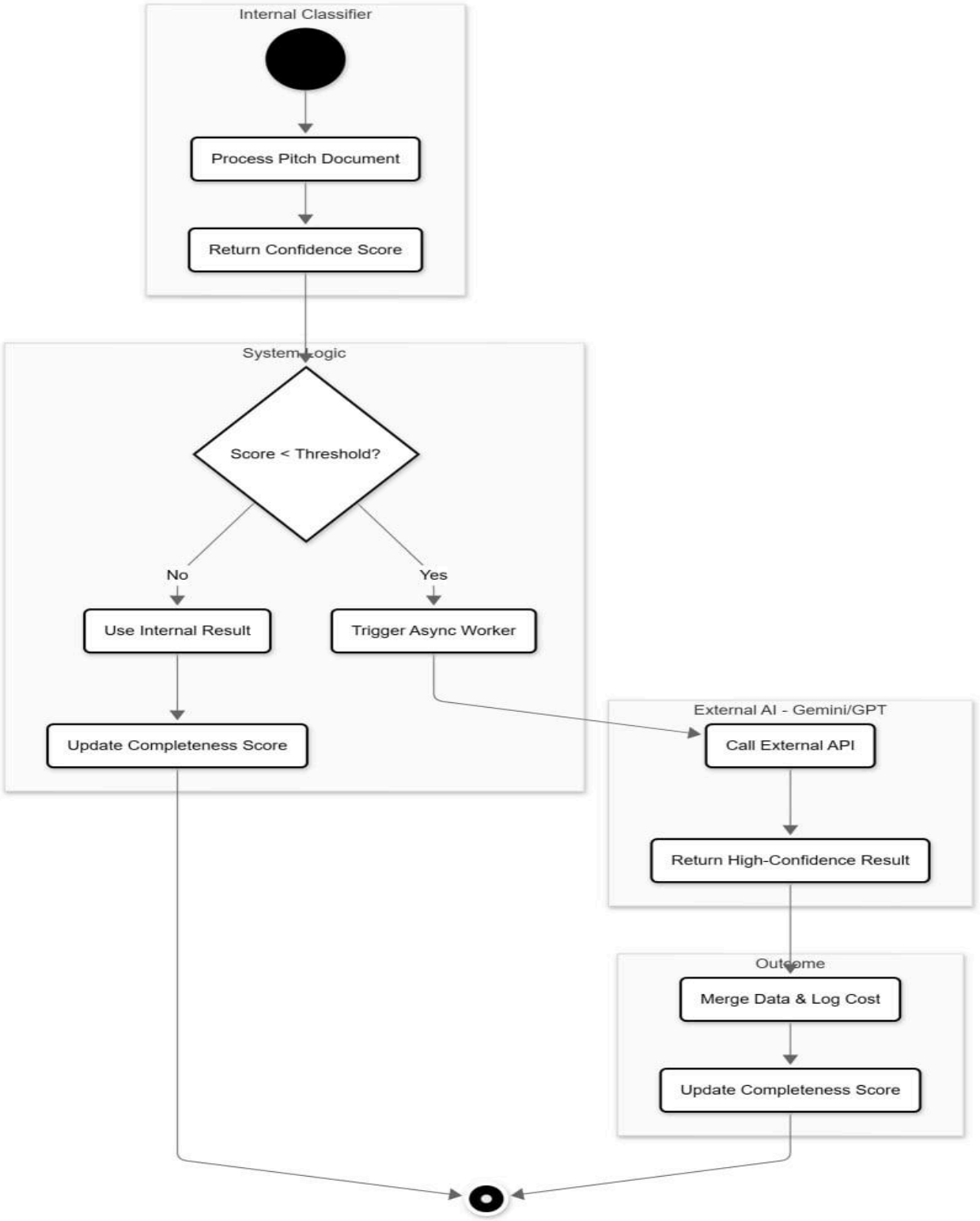


Fig: Activity diagram:- Scenario 17: Admin Approves Entrepreneur Submission

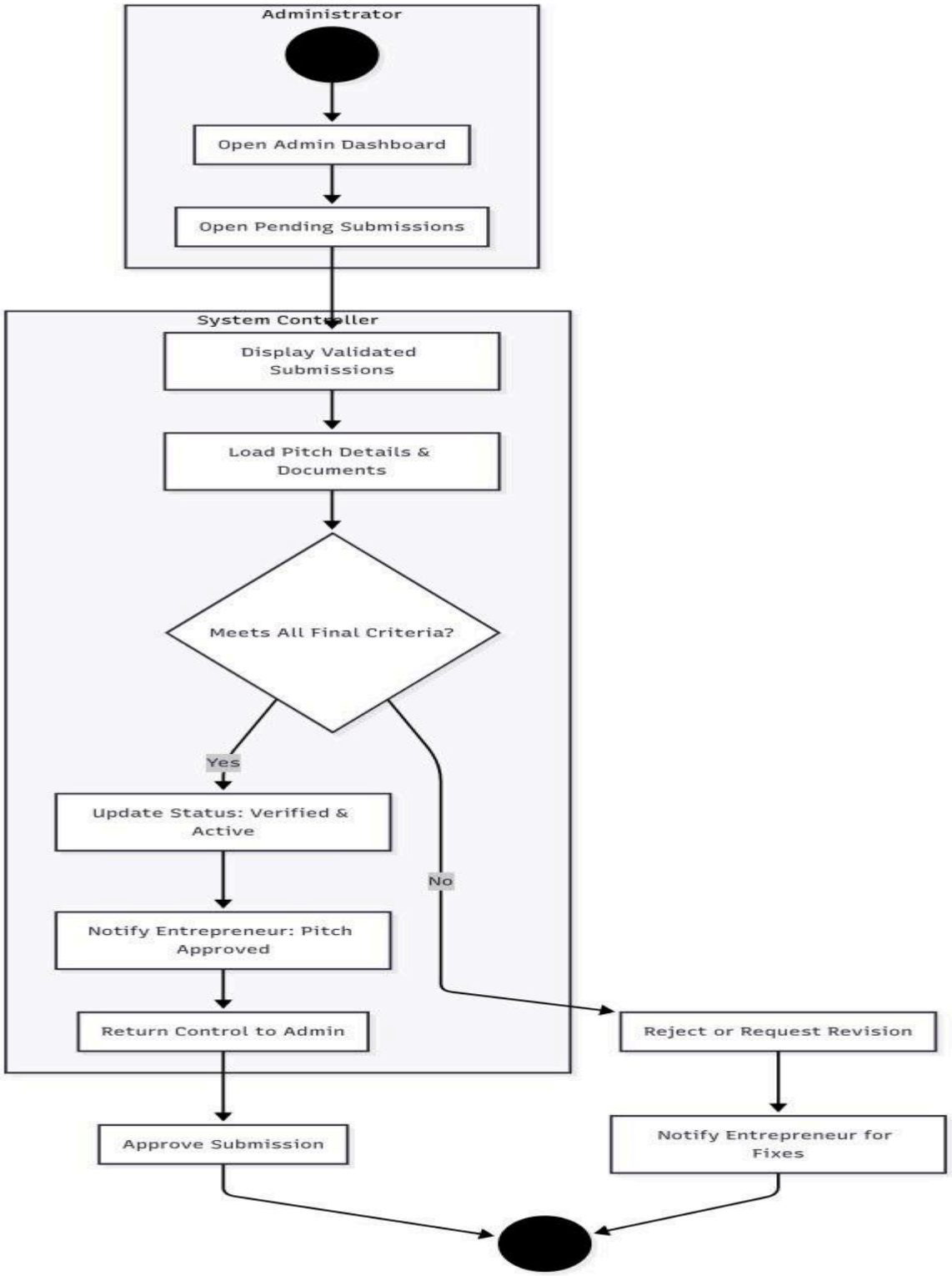


Fig: Activity diagram:- Scenario 18: AI Evaluation and Scoring Pipeline

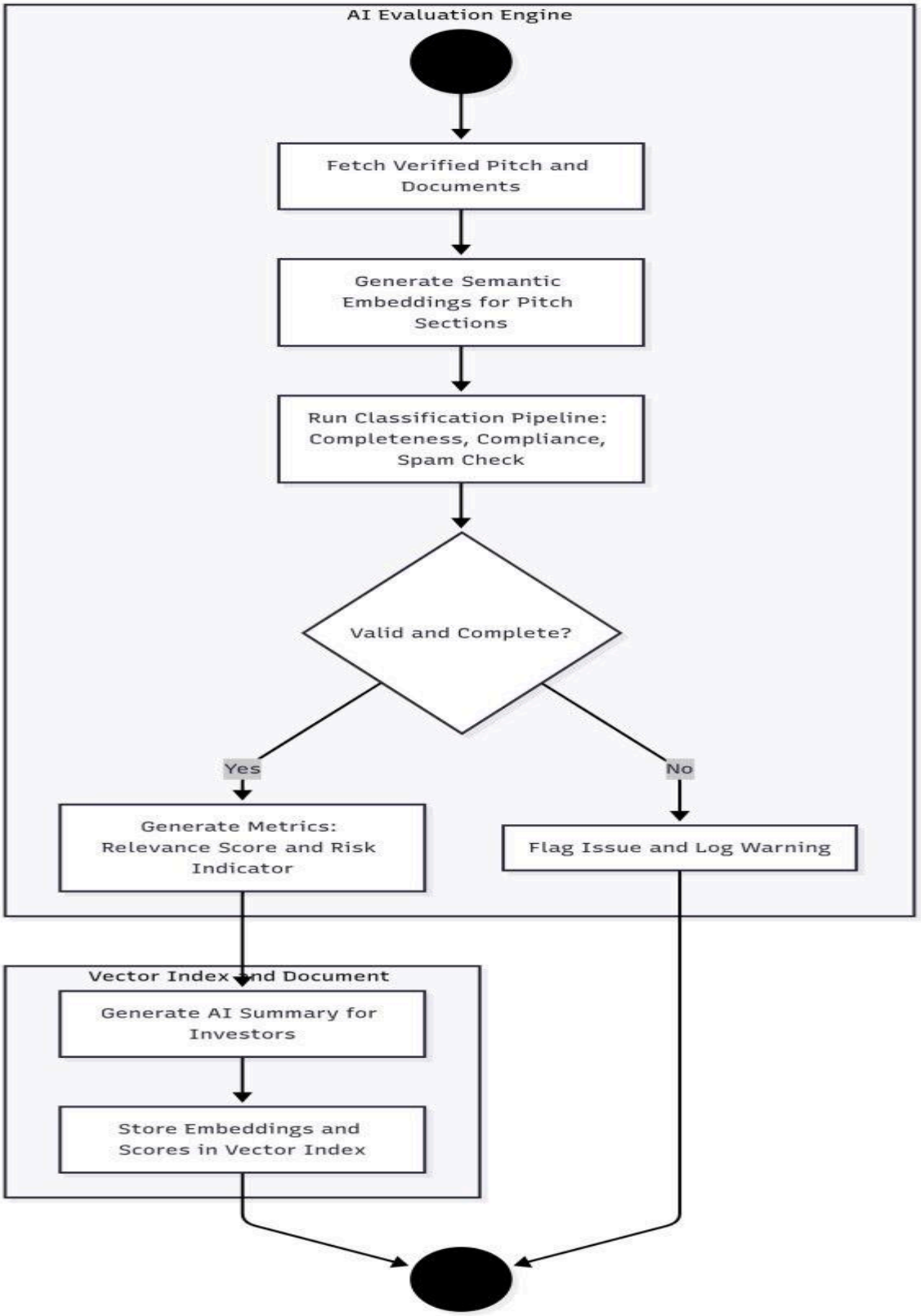


Fig: Activity diagram:- Scenario 19: Investor Recommendation, Review, and Voice Output

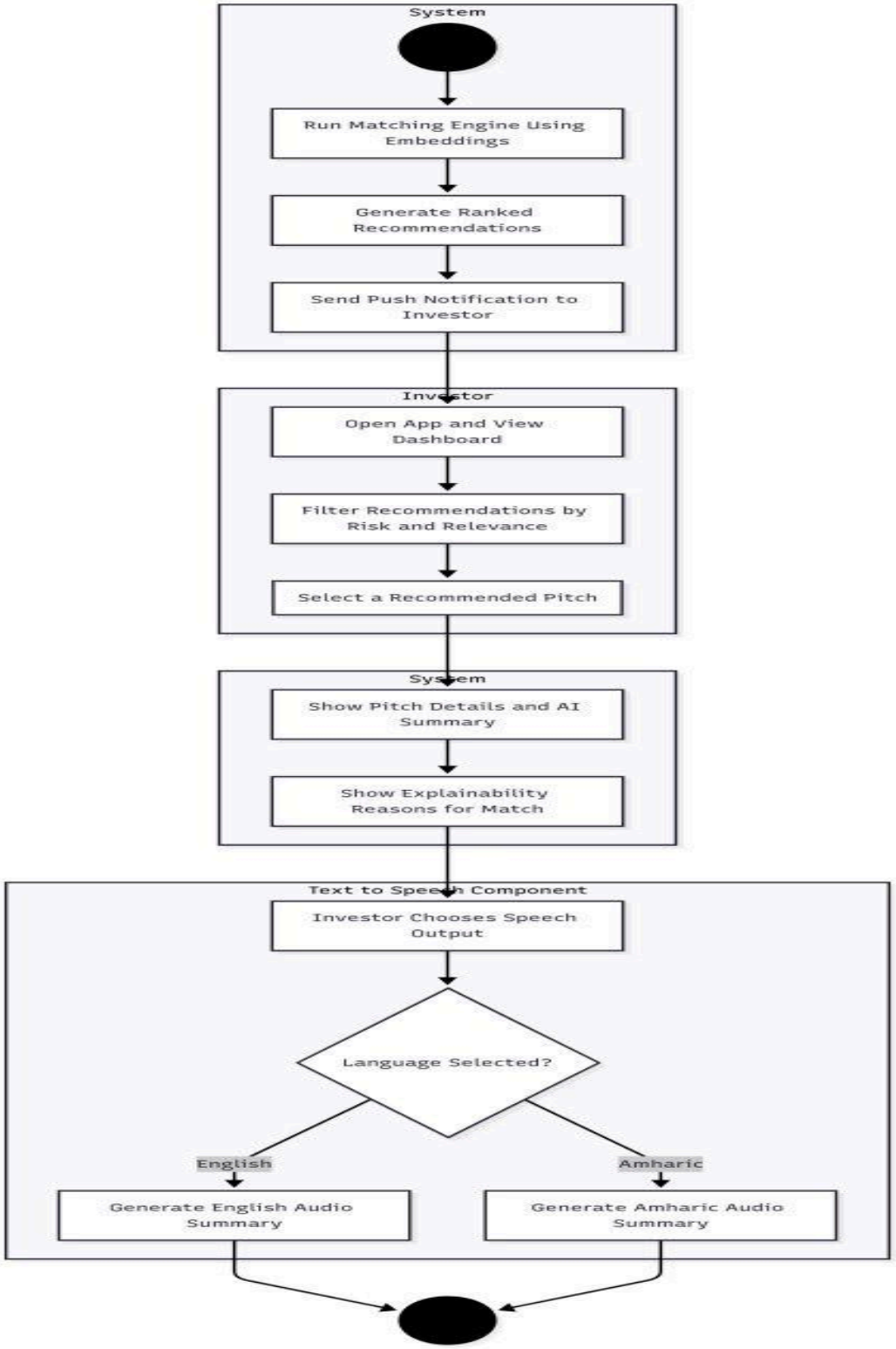


Fig: Activity diagram:- Scenario 20: Communication and Meeting Scheduling

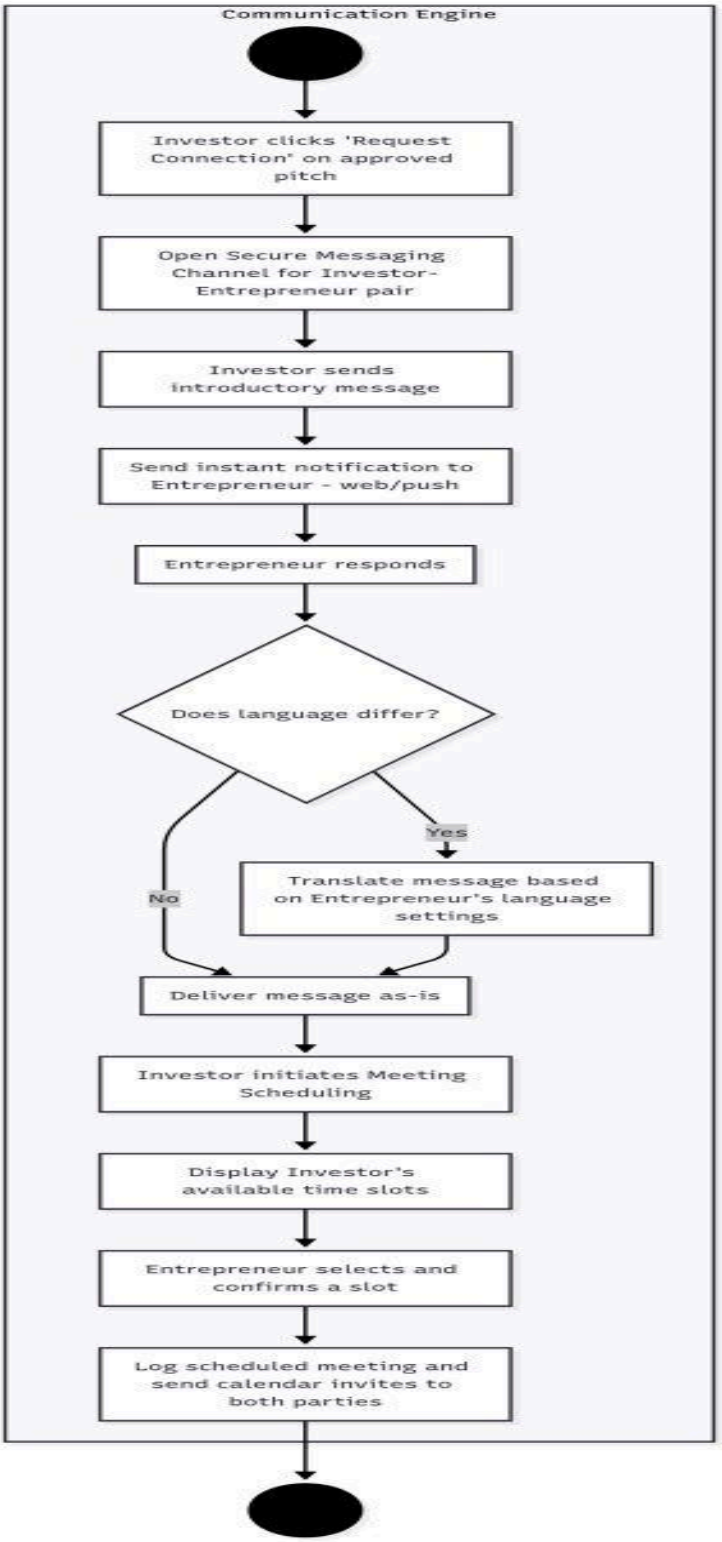


Fig: Activity diagram:- Scenario 21: Milestone Payment Simulation and Tracking

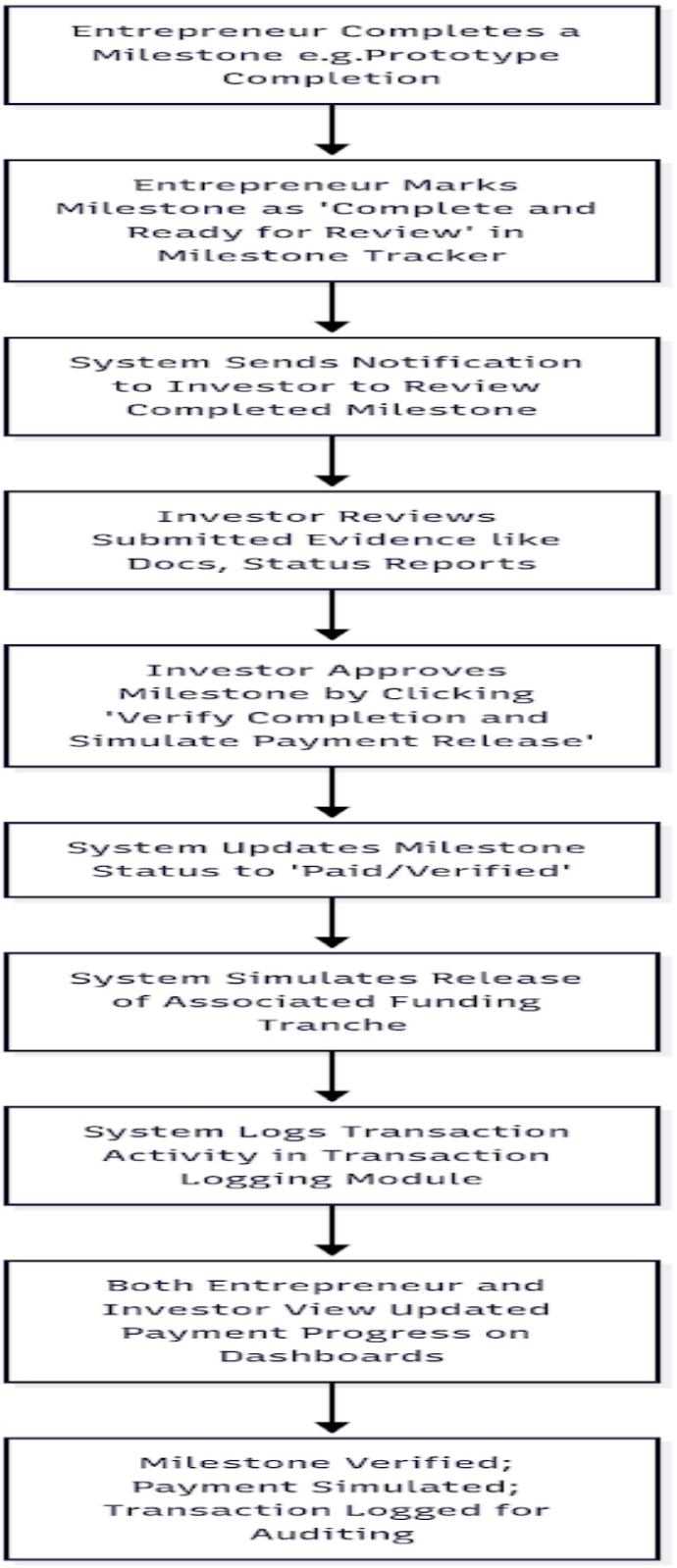

Entrepreneur Completes a Milestone e.g.Prototype Completion
Entrepreneur Marks Milestone as 'Complete and Ready for Review' in Milestone Tracker
System Sends Notification to Investor to Review Completed Milestone
Investor Reviews Submitted Evidence like Docs, Status Reports
Investor Approves Milestone by Clicking 'Verify Completion and Simulate Payment Release'
System Updates Milestone Status to 'Paid/Verified'
System Simulates Release of Associated Funding Tranche
System Logs Transaction Activity in Transaction Logging Module
Both Entrepreneur and Investor View Updated Payment Progress on Dashboards
Milestone Verified; Payment Simulated; Transaction Logged for Auditing

Fig: Activity diagram:- Scenario 22: Proposal Rejection (Investor Declines Pitch)

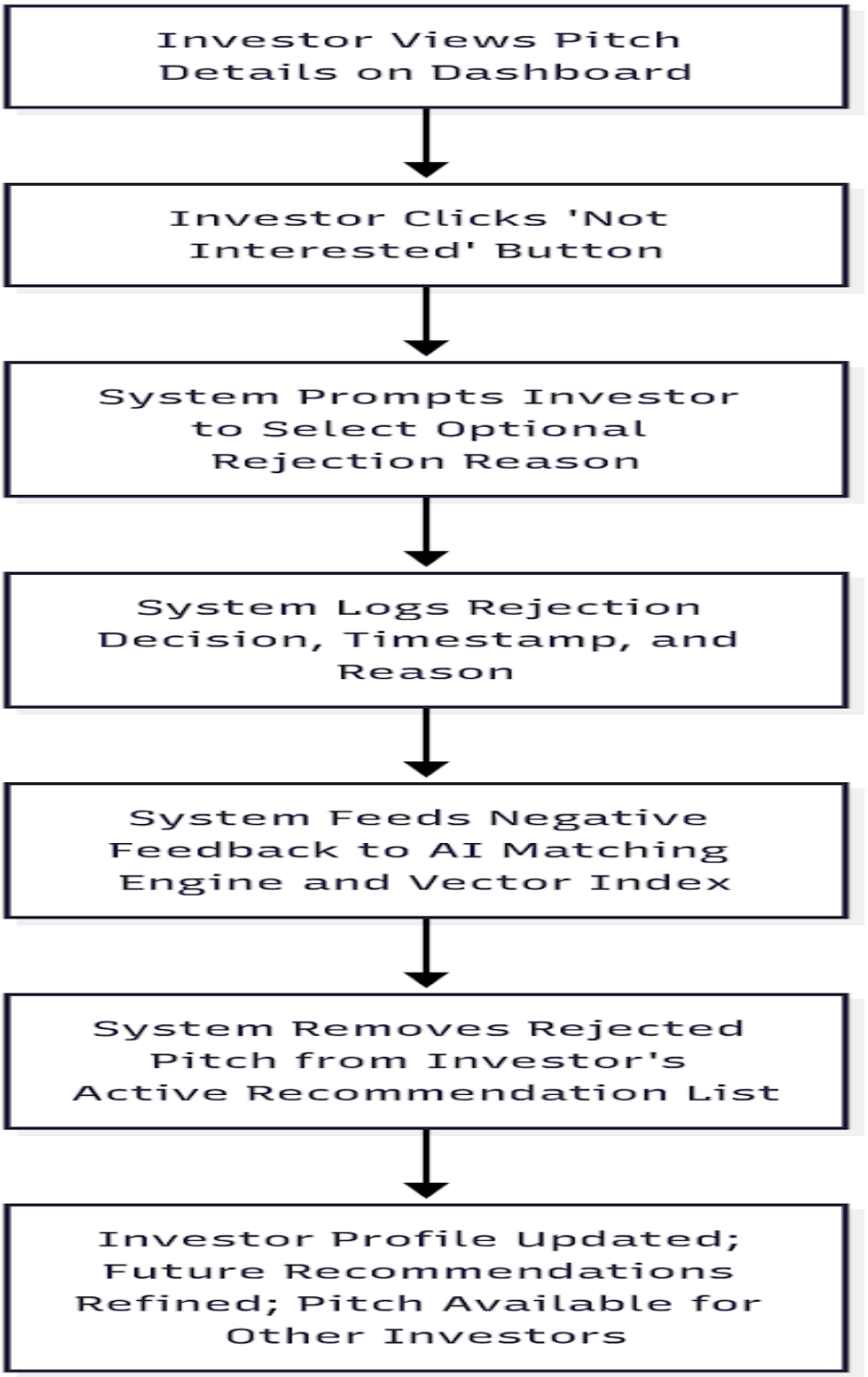


Fig: Activity diagram:- Scenario 23: Admin Removal or Suspension of Pitch

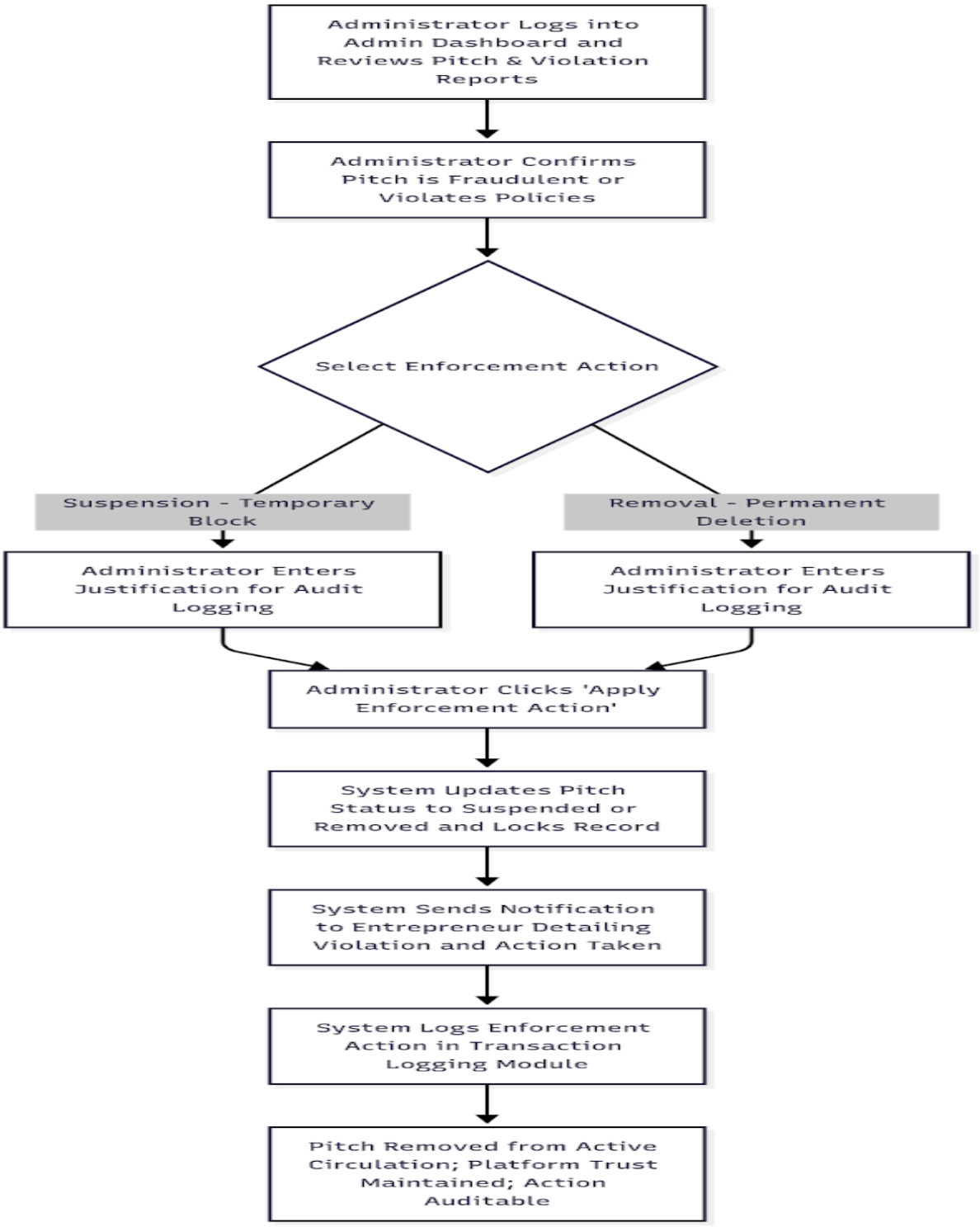


Fig: Activity diagram:- Scenario 24: Entrepreneur Account Suspension

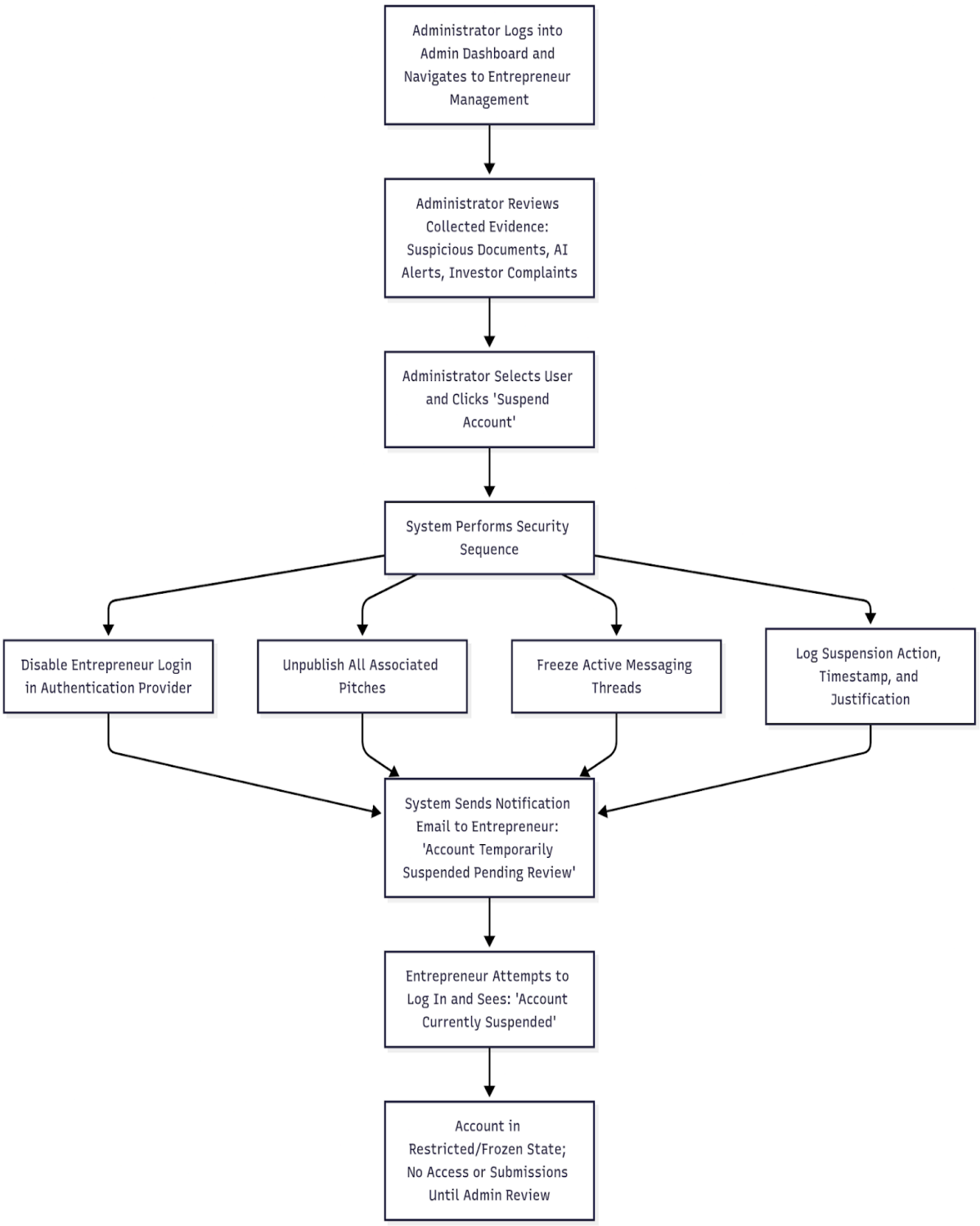


Fig: Activity diagram:- Scenario 25: Investor Account Suspension

Fig: Activity diagram:- Scenario 26: Investor Reports Suspicious Entrepreneur

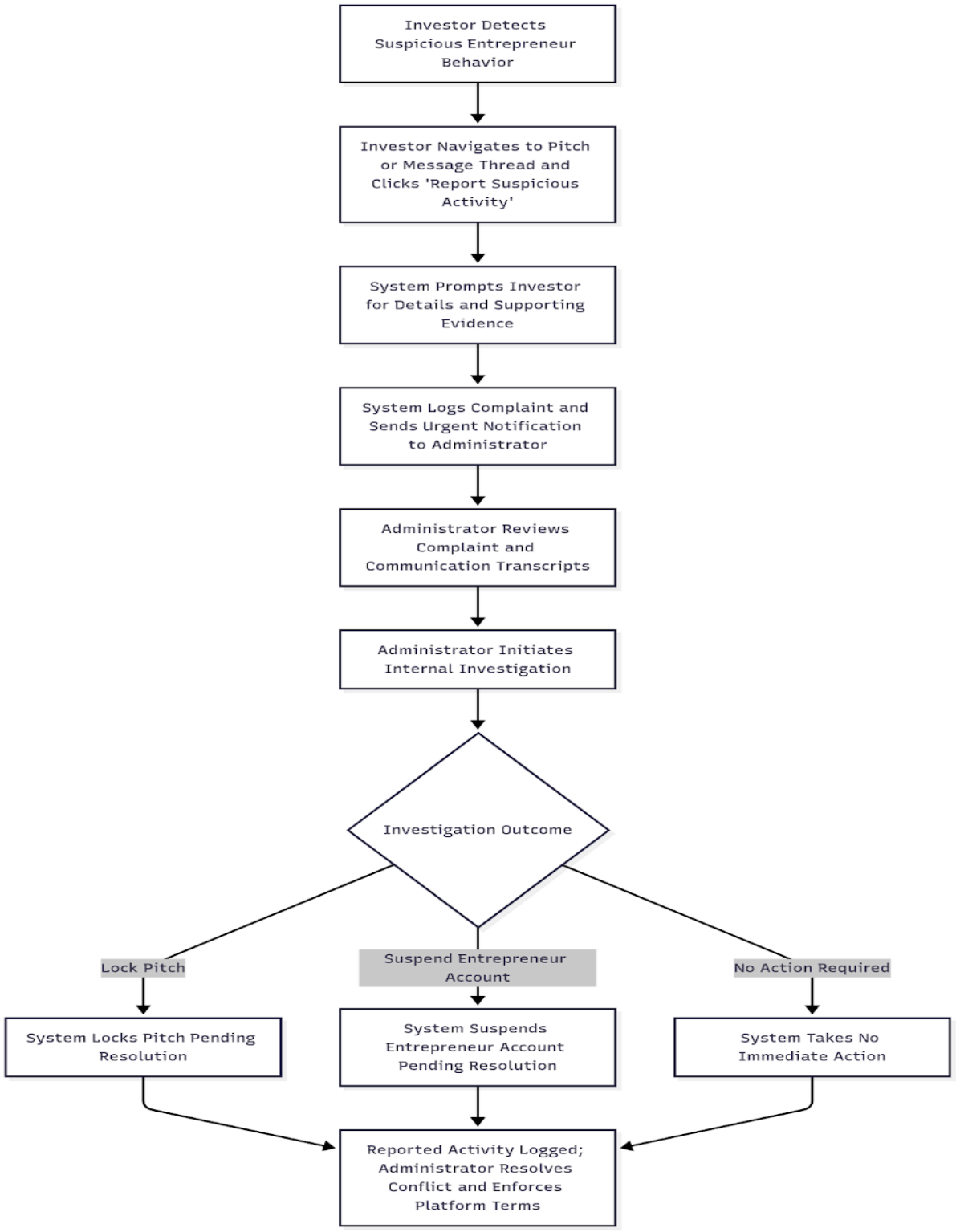
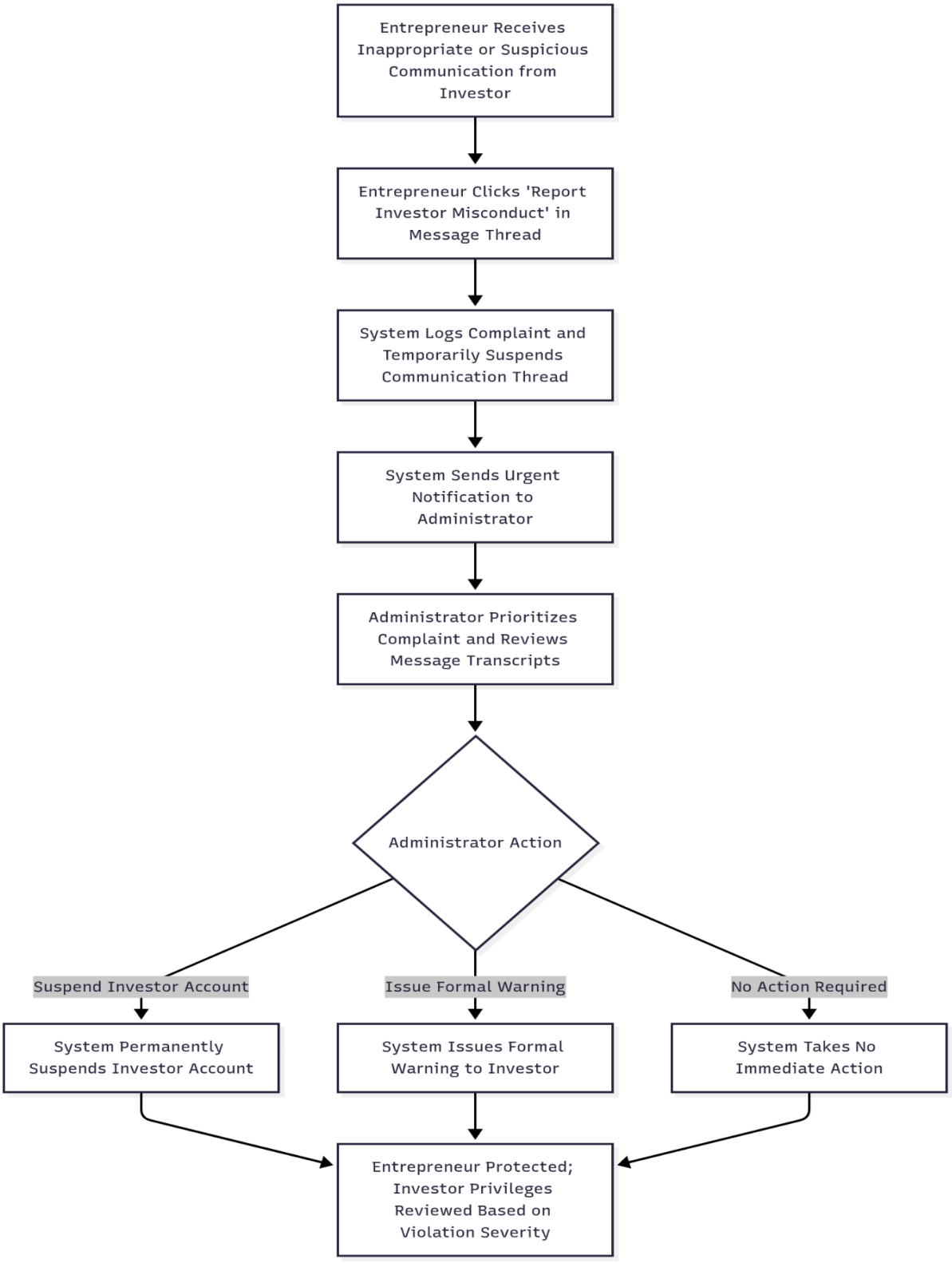


Fig: Activity diagram:- Scenario 27: Entrepreneur Reports Suspicious Investor


Entrepreneur Receives Inappropriate or Suspicious Communication from Investor
Entrepreneur Clicks 'Report Investor Misconduct' in Message Thread
System Logs Complaint and Temporarily Suspends Communication Thread
System Sends Urgent Notification to Administrator
Administrator Prioritizes Complaint and Reviews Message Transcripts
Administrator Action
Suspend Investor Account
Issue Formal Warning
No Action Required
System Permanently Suspends Investor Account
System Issues Formal Warning to Investor
System Takes No Immediate Action
Entrepreneur Protected; Investor Privileges Reviewed Based on Violation Severity

3.5.6. State chart diagram

Fig: State Diagram: Entrepreneur Sign Up & Initial Verification (UC01)

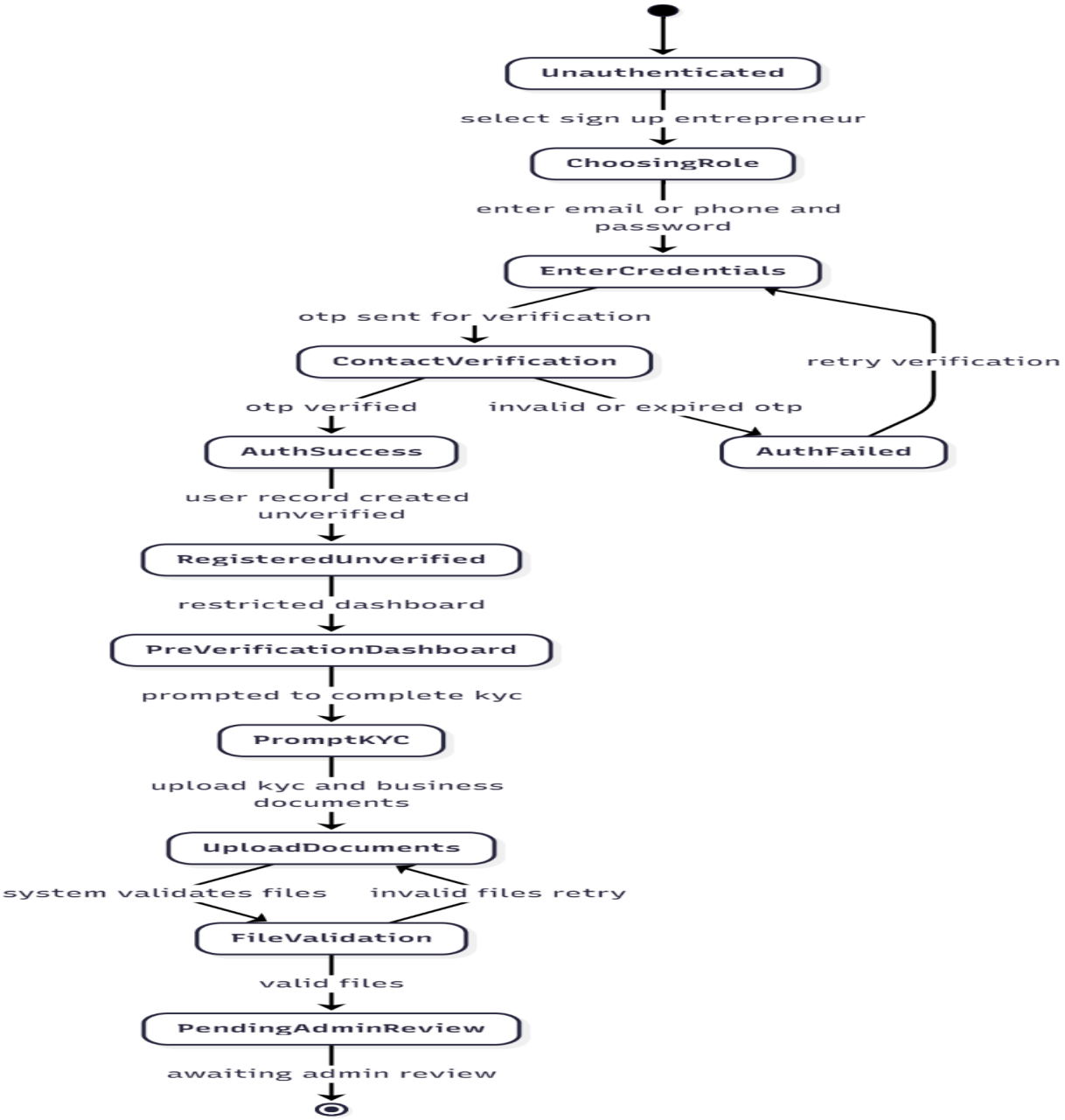


Fig: State Diagram: Investor Sign Up & Financial Verification (UC02)

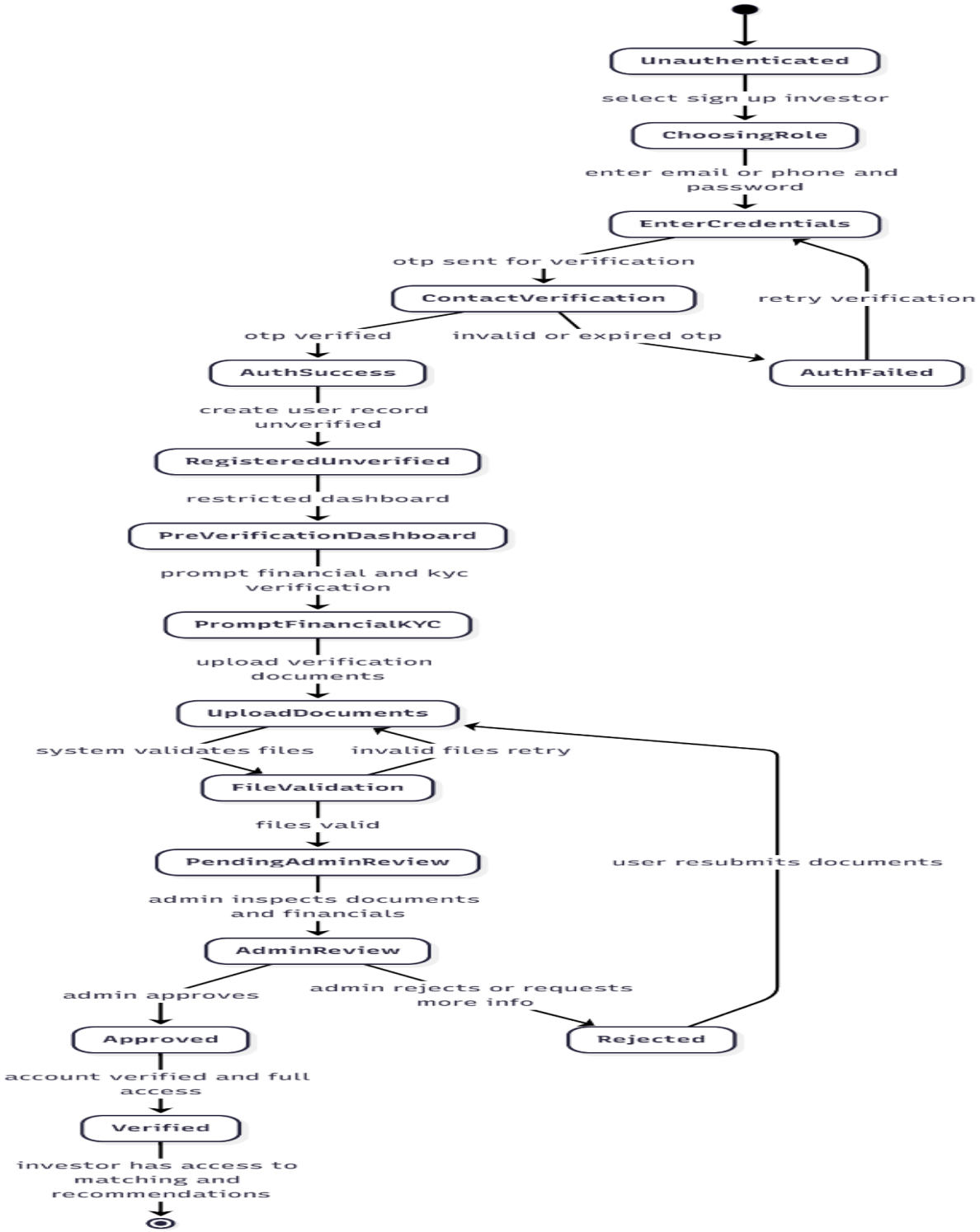


Fig: State Diagram: Full Pitch Creation and AI Submission (UC02)

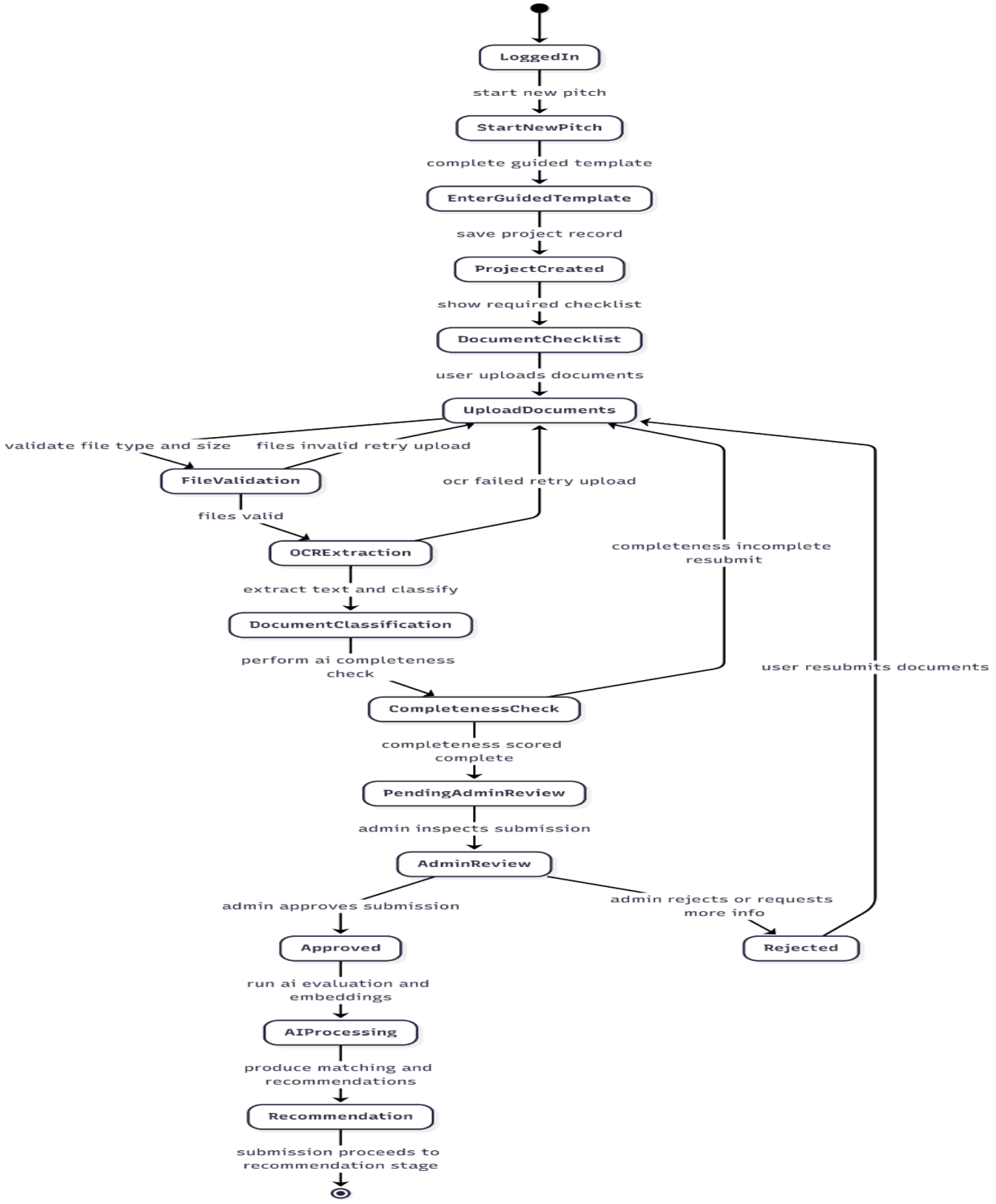


Fig: State Diagram: Administrator Correction and AI Override

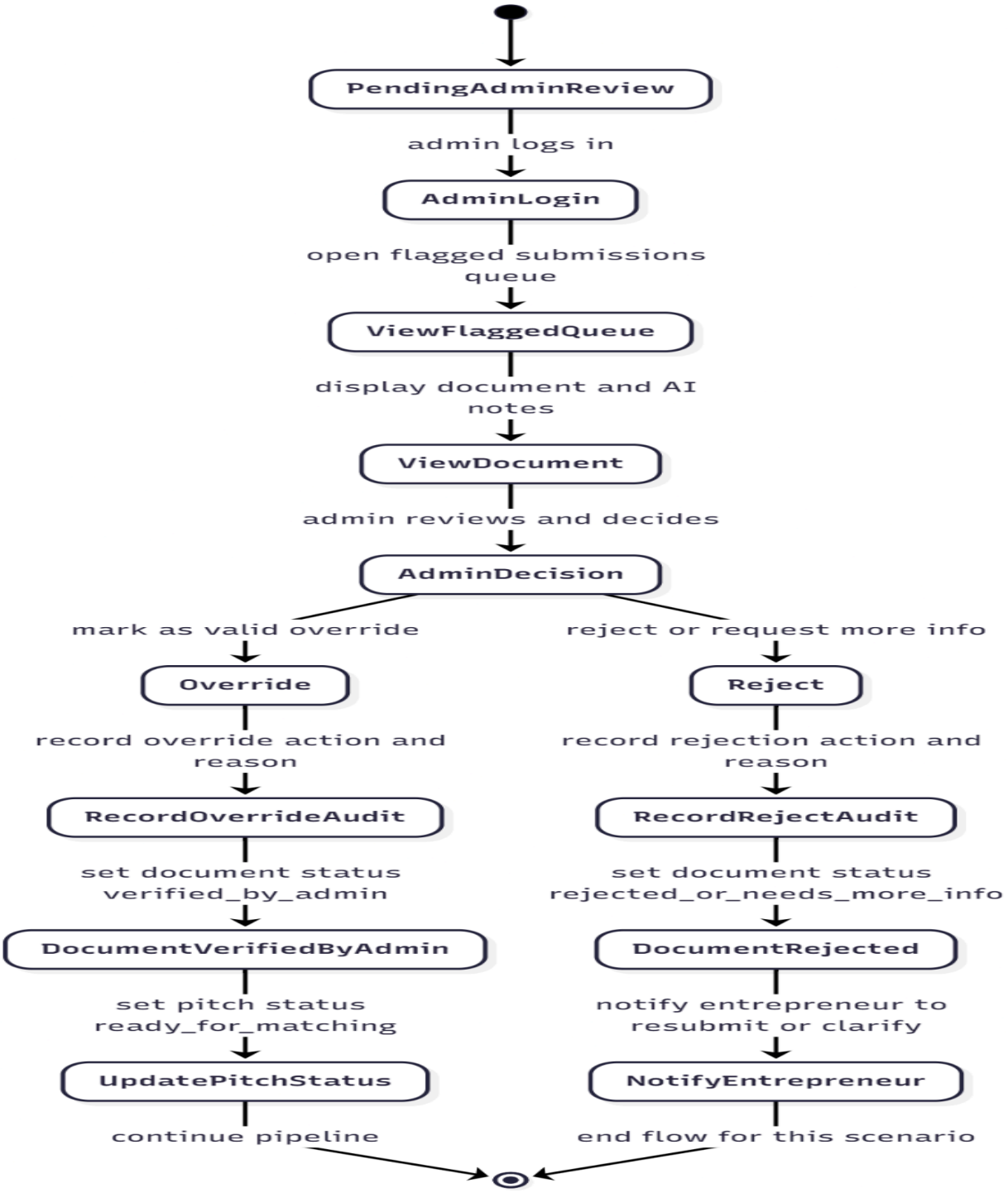


Fig: State Diagram: AI Fallback Scenario (Hybrid Checker)

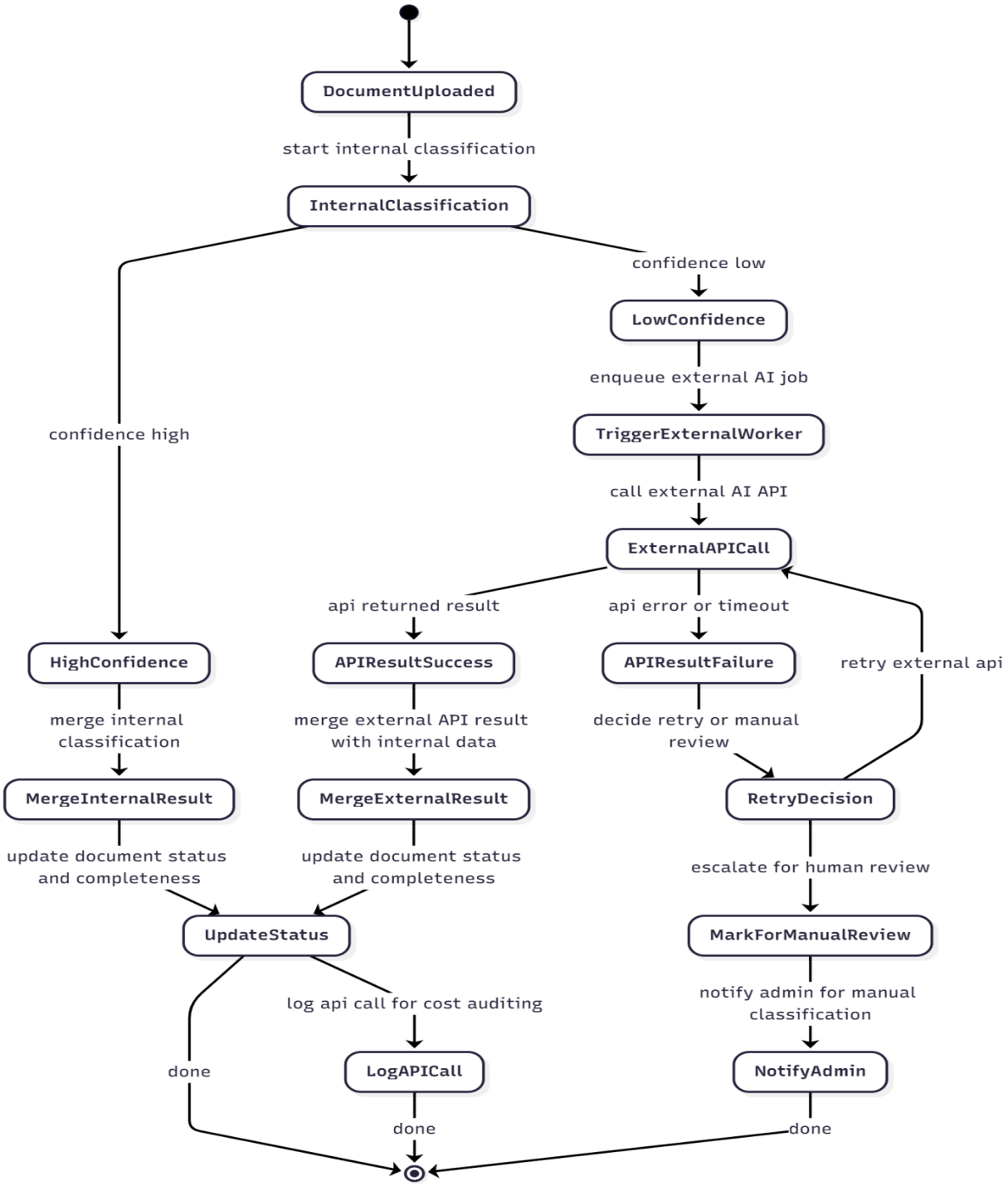


Fig: State Diagram: Admin Approves Entrepreneur Submission (Final Gate)

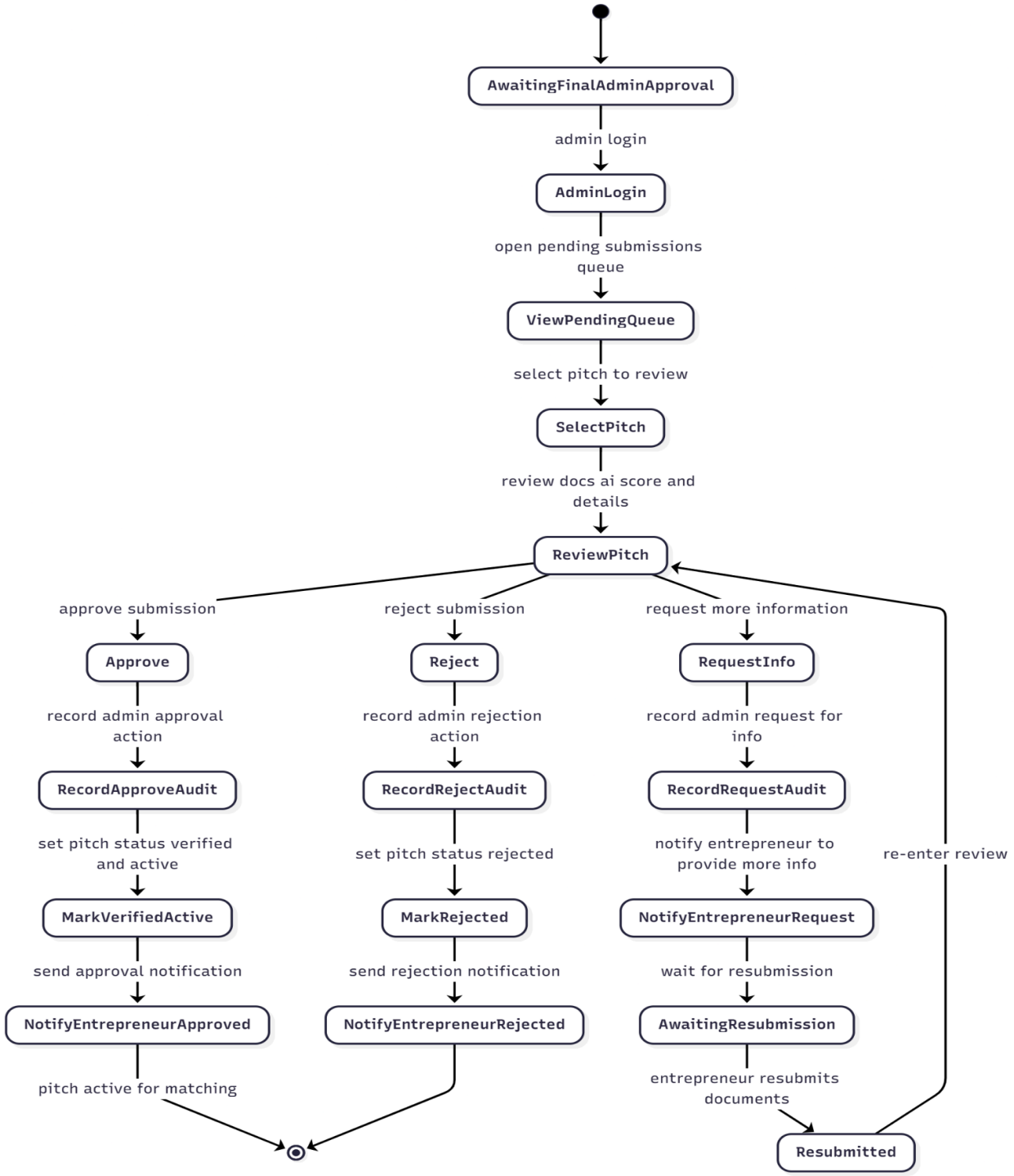


Fig: State Diagram: AI Evaluation and Scoring Pipeline

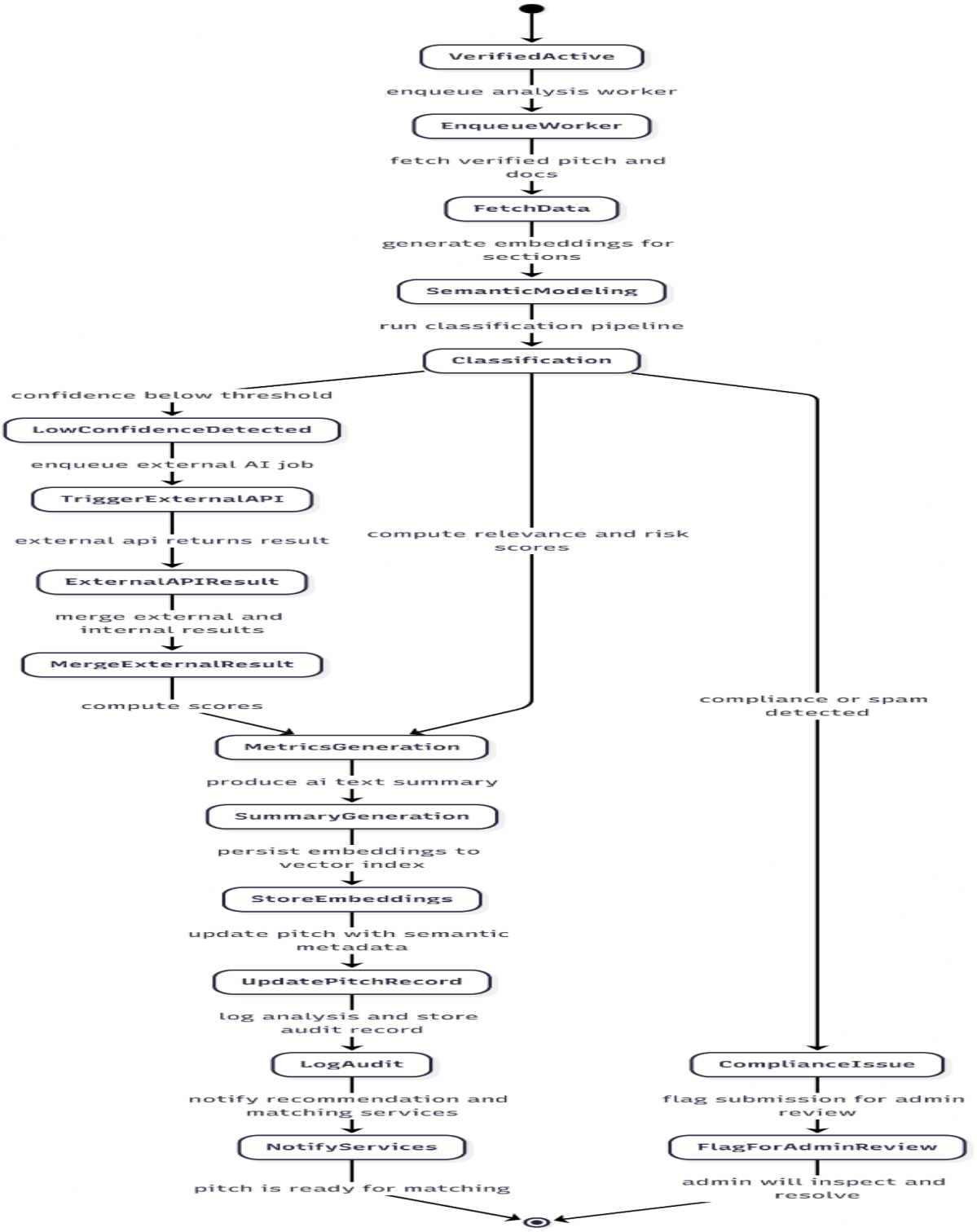


Fig: State Diagram: Investor Recommendation, Review, and Voice Output

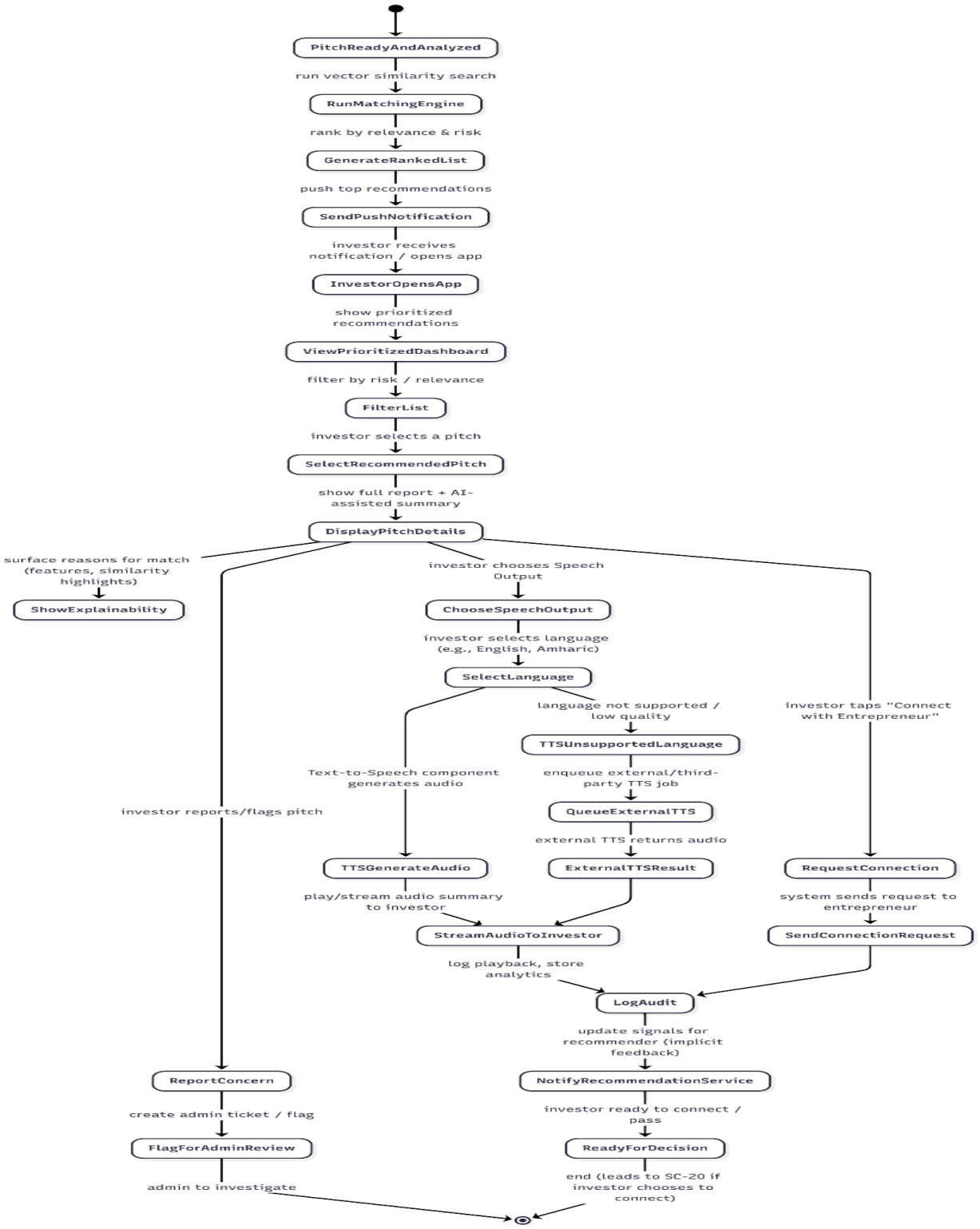
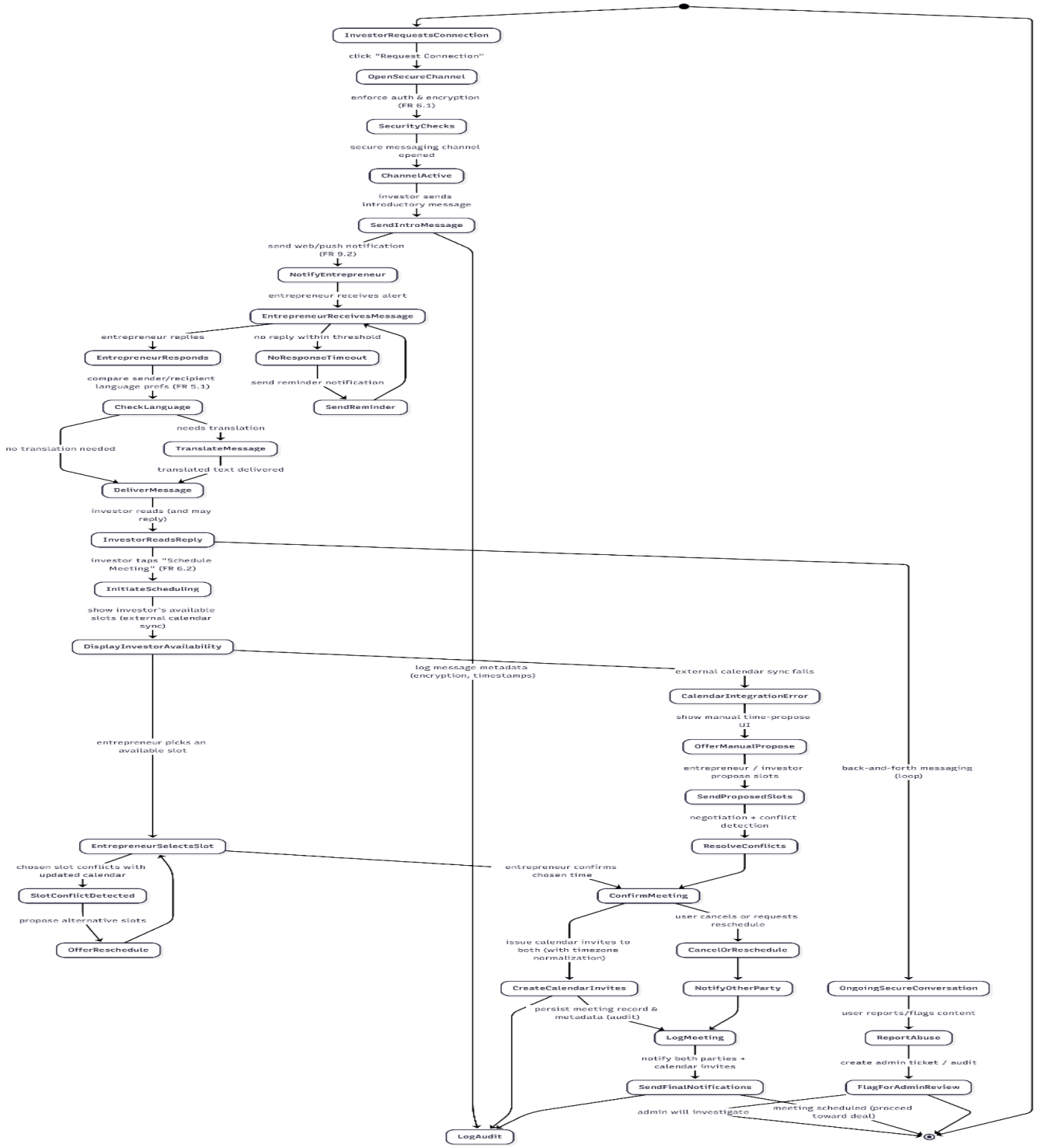


Fig: State Diagram: Communication and Meeting Scheduling



Chapter 4: System design

4.1. Overview

The Smart Entrepreneurial Pitching and Matching System (SEPMS) is designed as a modular and layered system that connects entrepreneurs, investors and administrators through a unified digital platform. The main goal of the system design is to clearly separate user interaction, business logic, intelligent processing and data management so that each part of the system can work efficiently and independently.

At a high level, the system is organized into four main layers. The first layer is the presentation layer, which consists of a web application developed using Next.js and a mobile application using Flutter. This layer is responsible for handling user interaction and displaying role based dashboards for entrepreneurs, investors and administrators. It does not contain business logic and only communicates with the backend through secure APIs.

The second layer is the backend or application layer, which is implemented using Node.js and Express. This layer acts as the core of the system. It handles authentication verification, role based authorization, pitch submission workflows, messaging, administrative actions and coordination with the AI services. All requests from the client applications pass through this layer.

The third layer is the AI processing layer, which handles intelligent operations such as document quality checking, semantic analysis, text embedding generation, investor to pitch matching and accessibility related features. These processes are executed asynchronously whenever possible so that the system remains responsive and scalable.

The final layer is the data and external services layer. This layer manages persistent data storage and third party integrations. MongoDB Atlas is used as the primary database for structured and semi structured data, including users, submissions and logs. A vector index is used to store and search semantic embeddings, while cloud storage is used to manage large files such as documents and videos. External services such as Firebase Authentication and Cloudinary are integrated through well defined interfaces.

Through this layered and modular design, SEPMS ensures that system components are loosely coupled, easier to maintain and capable of evolving as new features are added.

4.2. Purpose of the System Design

The purpose of the SEPMS system design is to transform the system requirements into a clear and practical structure that supports reliable implementation, testing and future expansion. The design decisions were made to ensure that both functional and non functional requirements are addressed in a balanced and realistic manner.

From a functional perspective, the system design supports the core operations of the platform, including user registration, role based access, guided pitch submission, document validation, AI assisted analysis recommendation generation and secure communication between entrepreneurs and investors. By separating the frontend from the backend and AI services, each feature can be implemented without interfering with other parts of the system.

From a non functional perspective, the design focuses on performance, scalability, security and maintainability. Heavy AI tasks such as semantic analysis and embedding generation are handled in background processes so that users do not experience delays. The use of stateless APIs and modular services allows the system to scale horizontally when user activity increases.

Security and privacy are also central to the system design. Authentication is handled by a trusted external provider, and authorization is enforced at the backend using role based access control. Sensitive data is protected through encrypted storage and controlled access rules, ensuring that users can only access resources they are permitted to view or modify.

The system design also directly supports the system models and object models defined in earlier chapters. The database collections, API endpoints and service modules correspond to the use cases, class diagrams and sequence flows already identified. This alignment makes the system easier to implement, test and debug, since each requirement can be traced to a specific design component.

Finally the design facilitates future development and experimentation. The modular structure allows new AI models, recommendation strategies or storage solutions to be introduced with minimal changes to the rest of the system. Administrative override mechanisms and audit logs also support controlled system operation and continuous improvement.

In summary the purpose of the SEPMS system design is to provide a clean, understandable and extensible foundation that meets current project requirements while remaining flexible for future growth.

4.3. Design Goals

The system is designed with simple and practical goals in mind. Since the platform includes many features and different user types, the design must be easy to understand and not too complex.

One major goal is modularity. The system is divided into parts such as user management, pitch submission, AI checking, recommendation, messaging, and admin control. Each part works on its own task. Because of this, if one module needs to be updated, it does not affect the whole system.

Another goal is keeping high cohesion and low coupling. Related functions are placed together, and modules do not depend too much on each other. For example, the AI part only sends results to the backend and does not directly control the user interface. This helps reduce errors and makes debugging easier.

The system also uses abstraction and encapsulation. Users do not see how the AI checks documents or how scores are calculated. They only see the final decision or feedback. This hides complexity and also improves system security.

Reusability is also considered during development. Common features like login, file upload, and notifications are reused in different parts of the system. Object-oriented ideas such as inheritance and polymorphism are used to manage user roles. All users share basic features, but entrepreneurs, investors, and admins have different permissions.

4.4. Proposed System Architecture

The system follows a layered architecture, which means the system is divided into different levels, and each level has a clear responsibility. This approach makes the system easier to manage and expand later.

4.4.1. Architecture Style

The system uses a multi-tier architecture with modular components. Each layer communicates with the others through APIs, and no layer directly controls the internal logic of another layer.

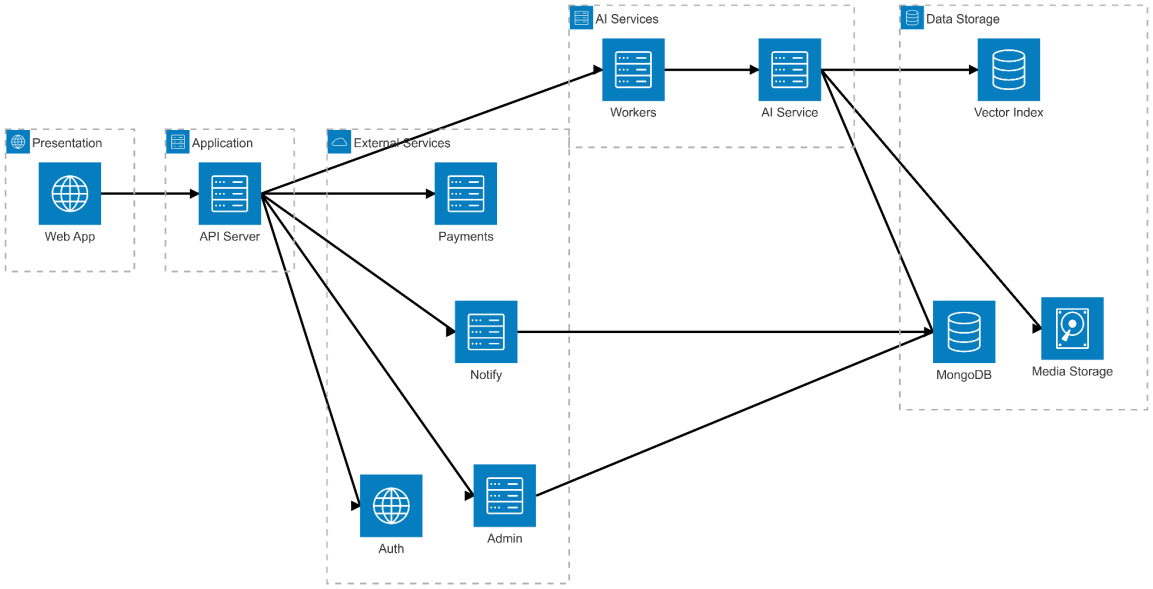


Fig: Architecture Diagram

4.4.2. System Architecture Description

Presentation Layer

This layer includes the web application built using Next.js and the mobile application developed with Flutter. It handles user interaction and shows different dashboards based on the user role. This layer does not handle business logic.

Backend Layer

The core application backend is built using Node.js and Express. It manages authentication, pitch submissions, database operations, messaging, and admin activities. The frontend communicates with the backend using REST APIs. It routes specific intelligent requests to the AI service.

AI Processing Layer

This layer is implemented as a separate service using python(FastAPI). It hosts the Scikit-learn classifiers, T5 summarization models, and OCR pipelines, generating texts and matching investor interests with projects. It exposes endpoints that the Node.js backend consumes to perform intelligent analysis.

Data Layer

MongoDB is used to store system data such as users, pitches, and logs. A vector database is used for similarity matching, and cloud storage is used for videos and documents. Separating data from logic improves performance and scalability.

4.5. Subsystem Decomposition

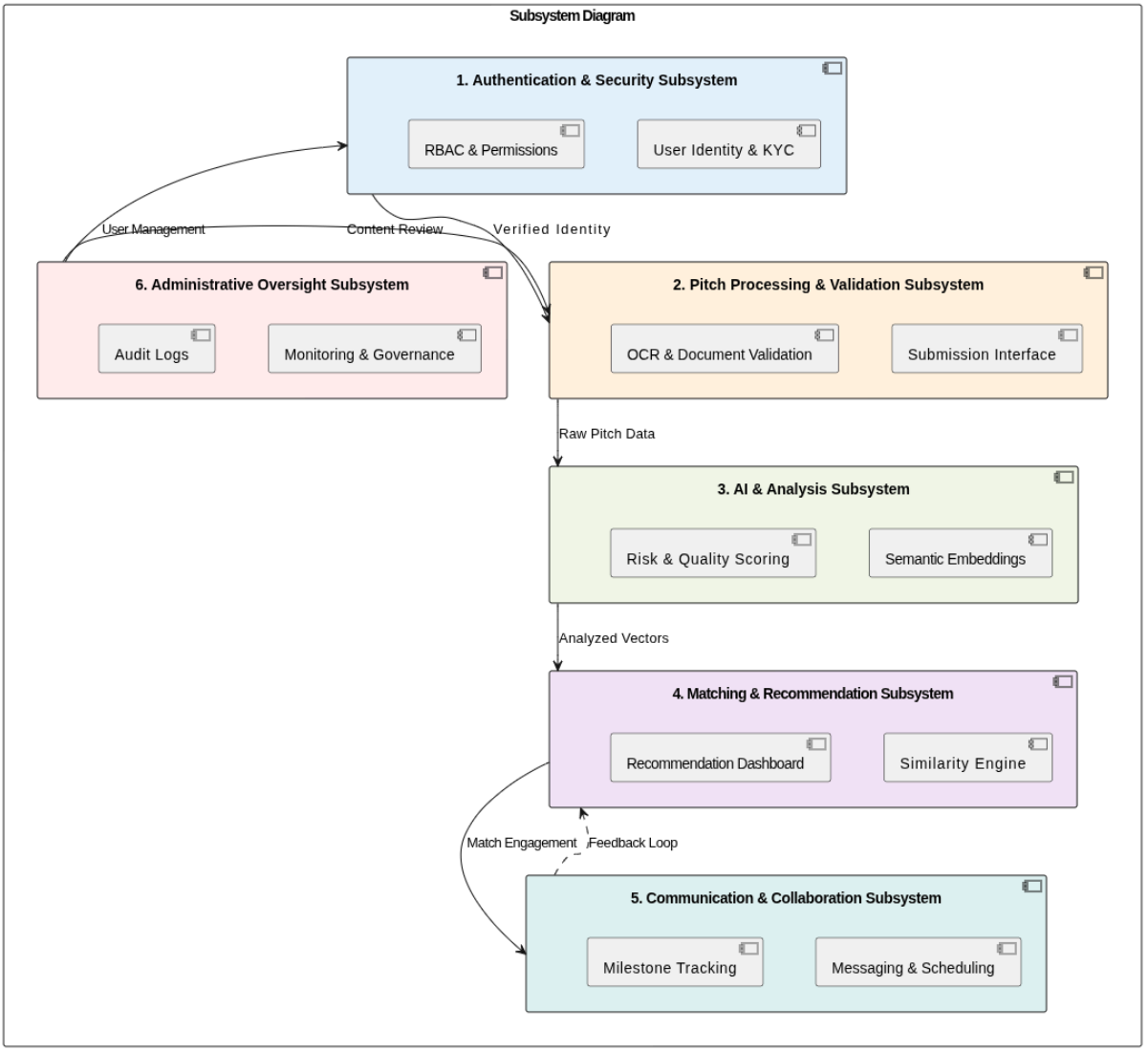
To ensure high modularity, scalability, and maintainability in the Smart Entrepreneurial Pitching & Matching System (SEPMS), the proposed architecture is decomposed into six core functional subsystems. Each subsystem encapsulates a cohesive set of closely related features derived from the system's functional requirements and use cases, promoting clear separation of concerns, loose coupling through well-defined interfaces (such as RESTful APIs and asynchronous event queues), and ease of independent development, testing, and deployment.

This domain-driven decomposition treats cross-cutting aspects like user interface elements (e.g., forms, dashboards, voice interactions) and data persistence (e.g., storage of profiles, pitches, documents, and logs) as integrated components within the relevant subsystems rather than standalone layers. The subsystems interact seamlessly for instance, the Pitch Processing and Validation Subsystem feeds validated pitch data to the AI and Analysis Subsystem for enrichment, which in turn supplies embeddings and scores to the Matching and Recommendation Subsystem, while notifications from any subsystem are routed through the Communication and Collaboration Subsystem. Administrative actions can override or monitor processes across subsystems via secure, audited interfaces.

The detailed subsystem decomposition is presented below, highlighting the rationale for grouping features and illustrating their contributions to the overall system goals of transparency, accessibility, and efficient entrepreneur-investor matching.

Subsystem Decomposition Diagram:

Fig: illustration of the subsystems and their interactions:



4.6. Subsystem Description

1. Authentication and Security Subsystem

This subsystem is responsible for managing user identity, access control, verification processes, and overall system security. It serves as the foundation for trusted interactions by ensuring only verified users can participate and that sensitive actions are properly gated and

audited. It encompasses all features related to onboarding, authentication, role management, and protective measures against misuse.

Features:

- User registration and authentication (sign-up with email/password , OTP/email verification, login, password recovery via time-limited OTP).
- Role-specific initial verifications (KYC document checks for entrepreneurs, financial standing verification for investors).
- Role-based access control and profile management (assignment of roles upon registration, customized profile views and restrictions based on role).
- Administrative approvals and manual overrides (final approval of user accounts after document review, manual corrections for AI-flagged issues).
- Account and pitch suspension/removal (suspension of entrepreneur or investor accounts for violations, removal of pitches).
- Mutual reporting mechanisms (entrepreneurs reporting suspicious investors, investors reporting suspicious entrepreneurs, with immediate actions like thread suspension).

2. Pitch Processing and Validation Subsystem

This subsystem handles the complete lifecycle of pitch creation and submission, focusing on guiding entrepreneurs through structured input and rigorously validating submissions for completeness, accuracy, and legitimacy before they become visible to investors. It integrates user-facing input forms with backend validation logic to prevent low-quality or fraudulent entries.

Features:

- Guided pitch submission interfaces (structured multistep forms for pitch details such as title, summary, problem, solution, team, financials, and funding goals; support for draft saving and resumption).
- Document and media upload handling (multistep/chunked uploads for large files, secure storage of supporting documents like business licenses, TIN, MoA/AoA, financial statements, and pitch videos).
- Automated pre-submission validations (detection of missing required documents with immediate feedback, wrong document type via OCR/content mismatch,

expired/outdated documents by date extraction, low-quality/unreadable documents based on confidence scores).

- Fraud and consistency checks (identification of suspicious or fraudulent documents through anomaly detection, cross-document conflict resolution such as name mismatches, handling of multi-business entrepreneurs by enforcing single-entity consistency).

3. AI and Analysis Subsystem

This subsystem provides all intelligent processing capabilities that enhance pitch quality, extract meaningful insights, and support accessibility features. It leverages machine learning, natural language processing, and voice technologies to automate analysis, generate enrichments, and handle cases where primary checks are inconclusive.

Features:

- Semantic content analysis and embedding generation (vector embeddings for pitch text, generation of risk indicators, structured summaries, and completeness metrics).
- Automated evaluation and scoring pipelines (quality flagging, completeness scoring using classifiers).
- AI fallback and hybrid checking (escalation to advanced external models like Gemini/GPT for low-confidence classifications while controlling costs).
- Voice and multilingual enhancements (text-to-speech output for pitch summaries in supported languages such as English and Amharic, support for voice-based interactions).

4. Matching and Recommendation Subsystem

This subsystem is dedicated to connecting entrepreneurs with suitable investors through intelligent, personalized matching. It processes analyzed pitch data to generate recommendations, presents them effectively to investors, and incorporates feedback to continuously improve matching accuracy.

Features:

- Semantic similarity-based recommendation generation (ranked lists of pitches for investors based on profile preferences, embedding similarity, filters for risk/relevance, and explainable scoring).
- Investor-facing recommendation review interfaces (personalized dashboards displaying matched pitches with AI-generated summaries and voice playback).
- Proposal handling and feedback loop (investor review of pitches, acceptance or rejection with optional negative feedback to refine future recommendations).

5. Communication and Collaboration Subsystem

This subsystem facilitates direct interaction between matched users, progress tracking for potential investments, and timely notifications to keep all parties informed. It bridges the gap from matching to actual collaboration while maintaining transparency and engagement.

Features:

- Secure in-app communication (real-time messaging with multilingual translation support).
- Meeting and coordination tools (calendar-based scheduling with available slot display and invite sending).
- Notification and alert delivery (real-time updates for events such as submission status, approvals, rejections, suspensions, meeting invites, and milestone progress).
- Simulated investment tracking (creation and management of project milestones, simulated deposit and release of funds based on verified completion, progress logging visible to both parties).

6. Administrative Oversight Subsystem

This subsystem equips administrators with tools to monitor system health, intervene in exceptional cases, and maintain platform integrity. It provides centralized control over content and user behavior while supporting analytics for ongoing improvement.

Features:

- Admin dashboards and monitoring (overview of flagged submissions, user activities, and system analytics/reports).

- Intervention and governance actions (review and resolution of reported issues, manual pitch removal, account suspensions, handling of mutual suspicion reports).
- Oversight of critical workflows (monitoring of AI performance, manual approval gates, audit logging of administrative actions).

4.7. Persistent Data Management

The persistent data management strategy in the Smart Entrepreneurial Pitching and matching System(SEPMS) is developed to support a hybrid system which can benefit from a combination of transactional integrity and fast semantic search , a specialized vector index and cloud storage for dealing with all kinds of data.

The purpose of this section is to detail the data models and schemas derived from the analysis stage and to show the storage component mapping.

4.7.1 Data Storage componentes

The system relies on three primary components , as dictated by the technical requirements

Component	Technology Used	Purpose and Rationale	Functional Requirement
Document Database	MongoDB Atlas	Stores user profiles, structured pitch content, transactional data (messages, meetings), and dynamic, non-uniform data like investor preferences. It supports fast development and flexible schemas.	API Backend , Data Management
Vector Database/Index	MongoDB Atlas Vector Index	Stores high-dimensional numerical embeddings of pitch and investor profile text. Essential for enabling semantic similarity search for the recommendation engine.	Semantic Matching
Cloud Storage	Cloudinary	Stores large, unstructured binary files, including business registration documents, videos, and pitch decks.	Document Upload and Storage

4.8. Component Diagram

The Component Diagram for the Smart Entrepreneurial Pitching and Matching System (SEPMS) illustrates the physical architecture of the system, showing how the major software modules and external services are organized , interface and communicate over the network.

The design adheres to a service -oriented approach separating the presentation layer, the application logic and specialized data/Ai services to enhance scalability and maintainability

4.8.1. Component Structure

The SEPMS consists of four main tires

1. Client Tier (Presentation)
2. Application (Backend logic)
3. Ai processing Layer
4. Data & External Service Tier

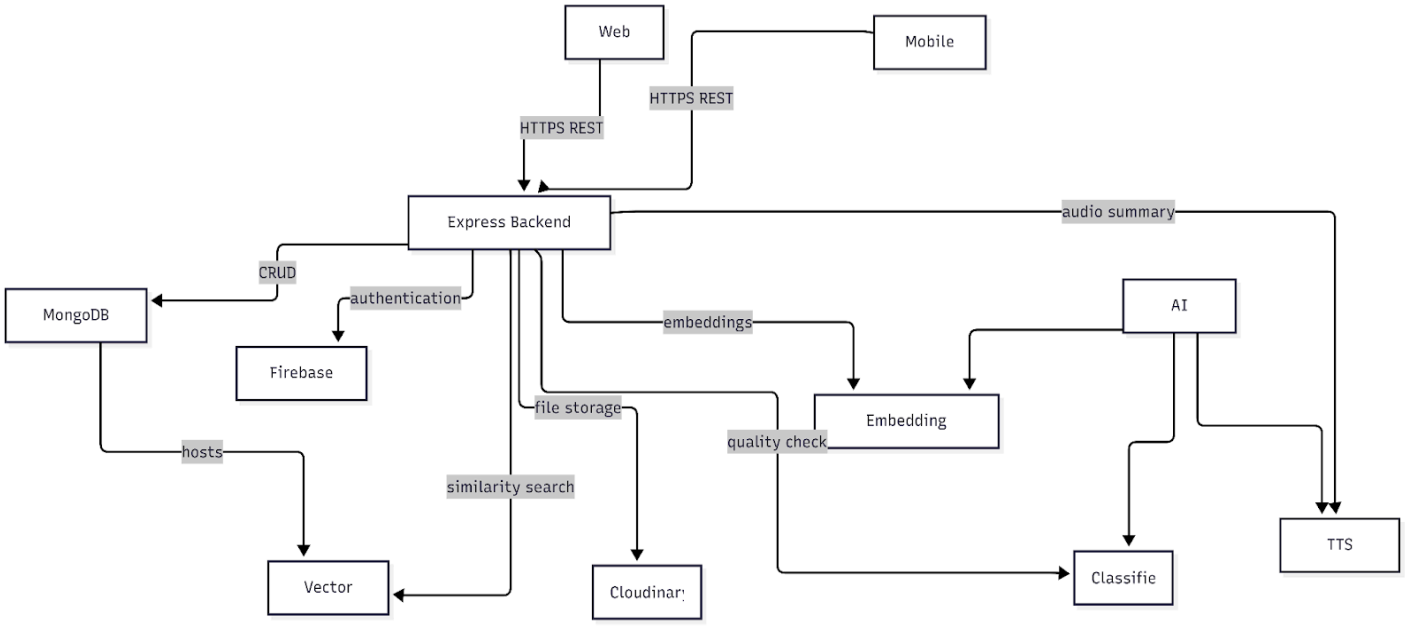


Fig: system Component Diagram

4.8.2 Component Descriptions

Component	Description	Technologies

Web App	Primary interface for investors and administrators. Designed for desktop use with full responsiveness.	Next.js, React, Tailwind CSS
Mobile App	Mobile interface for entrepreneurs to submit proposals and track progress on the go.	Flutter
Express Backend Server	Central API layer that handles business logic, input validation, and coordination with AI and data services.	Node.js, Express, Axios
MongoDB Atlas	Used as both the transactional document store and the vector index for semantic similarity matching.	MongoDB Atlas
AI / NLP Services	Components responsible for semantic analysis, quality checks, and accessibility features.	Python, Fast API, MiniLM, Scikit-learn, MongoDB Atlas Vector Search, Coqui (TTS)
Firebase Auth	Provides secure user authentication and account management without manual password handling.	Firebase Authentication
Cloudinary	Manages storage, optimization, and delivery of large media files such as documents, images, and videos.	Cloudinary

4.9. Database Diagram

This section presents the database diagram and data architecture. Given the systems requirements for handling variable structure pitch decks, high dimensional vector embeddings, and rapid prototyping, MongoDB(NoSQL DB) was selected as the persistence layer. To ensure data integrity and reliable application logic, a strict schema is enforced at the application layer using Mongoose. The database design decouples user authentication from profile data, separates unstructured media metadata from core business logic.

4.9.1 Entity Relationship Diagram

The following diagram illustrates the logical relationships between the systems collections. It highlights the central role of the Submission entity and its connections to Ai results, funding milestones and investor interactions.

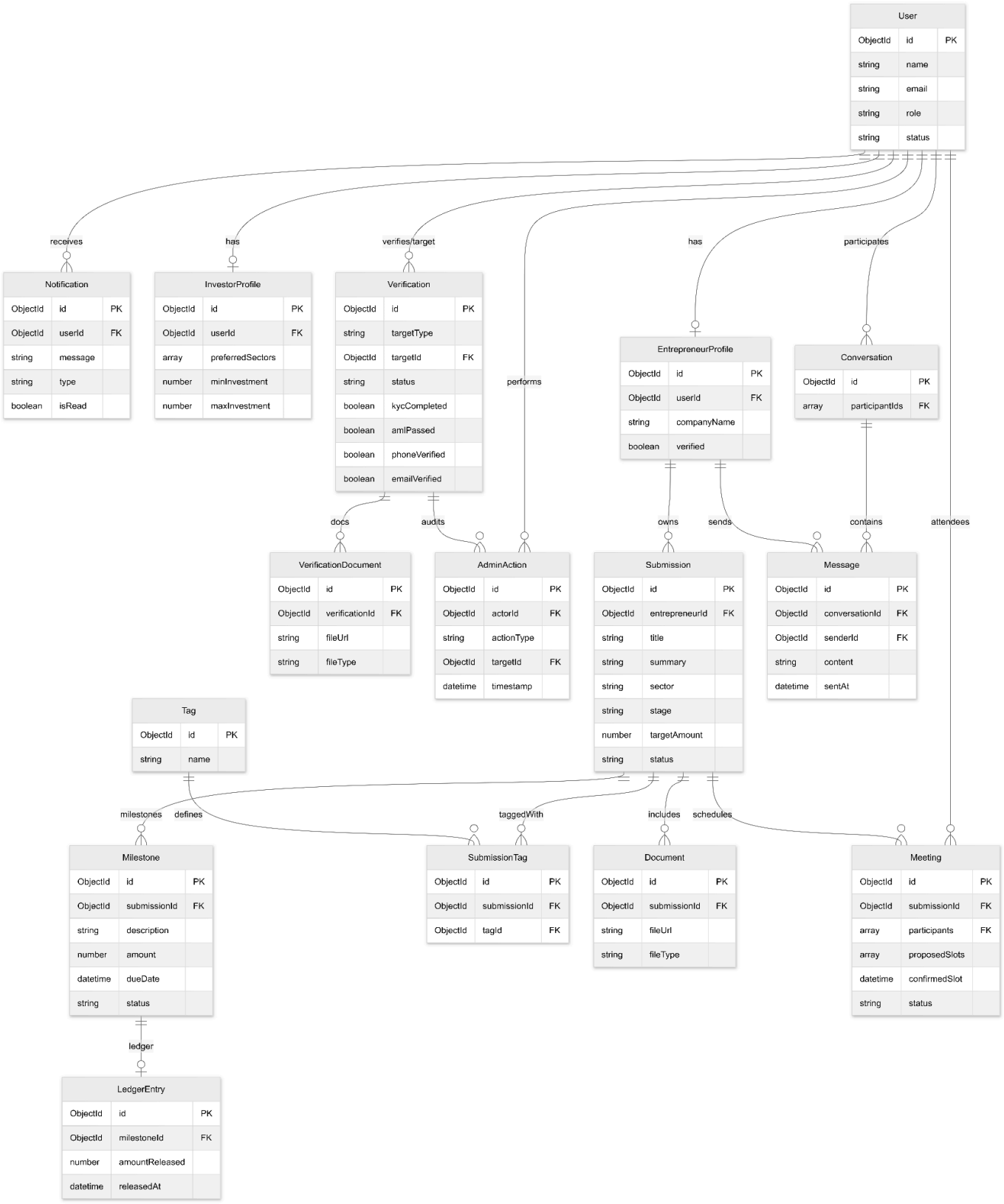

User
ObjectId id PK
string name
string email
string role
string status
receives
has
verifies/target
has
participates
Notification
ObjectId id PK
ObjectId userId FK
string message
string type
boolean isRead
InvestorProfile
ObjectId id PK
ObjectId userId FK
array preferredSectors
number minInvestment
number maxInvestment
Verification
ObjectId id PK
string targetType
ObjectId targetId FK
string status
boolean kycCompleted
boolean amlPassed
boolean phoneVerified
boolean emailVerified
performs
EntrepreneurProfile
ObjectId id PK
ObjectId userId FK
string companyName
boolean verified
Conversation
ObjectId id PK
array participantIds FK
docs
audits
owns
sends
contains
attendees
VerificationDocument
ObjectId id PK
ObjectId verificationId FK
string fileUrl
string fileType
AdminAction
ObjectId id PK
ObjectId actorId FK
string actionType
ObjectId targetId FK
datetime timestamp
Submission
ObjectId id PK
ObjectId entrepreneurId FK
string title
string summary
string sector
string stage
number targetAmount
string status
Message
ObjectId id PK
ObjectId conversationId FK
ObjectId senderId FK
string content
datetime sentAt
Tag
ObjectId id PK
string name
milestones
defines
taggedWith
includes
schedules
Milestone
ObjectId id PK
ObjectId submissionId FK
string description
number amount
datetime dueDate
string status
SubmissionTag
ObjectId id PK
ObjectId submissionId FK
ObjectId tagId FK
Document
ObjectId id PK
ObjectId submissionId FK
string fileUrl
string fileType
Meeting
ObjectId id PK
ObjectId submissionId FK
array participants FK
array proposedSlots
datetime confirmedSlot
string status
ledger
LedgerEntry
ObjectId id PK
ObjectId milestoneId FK
number amountReleased
datetime releasedAt

Fig: Entity Relationship Diagram

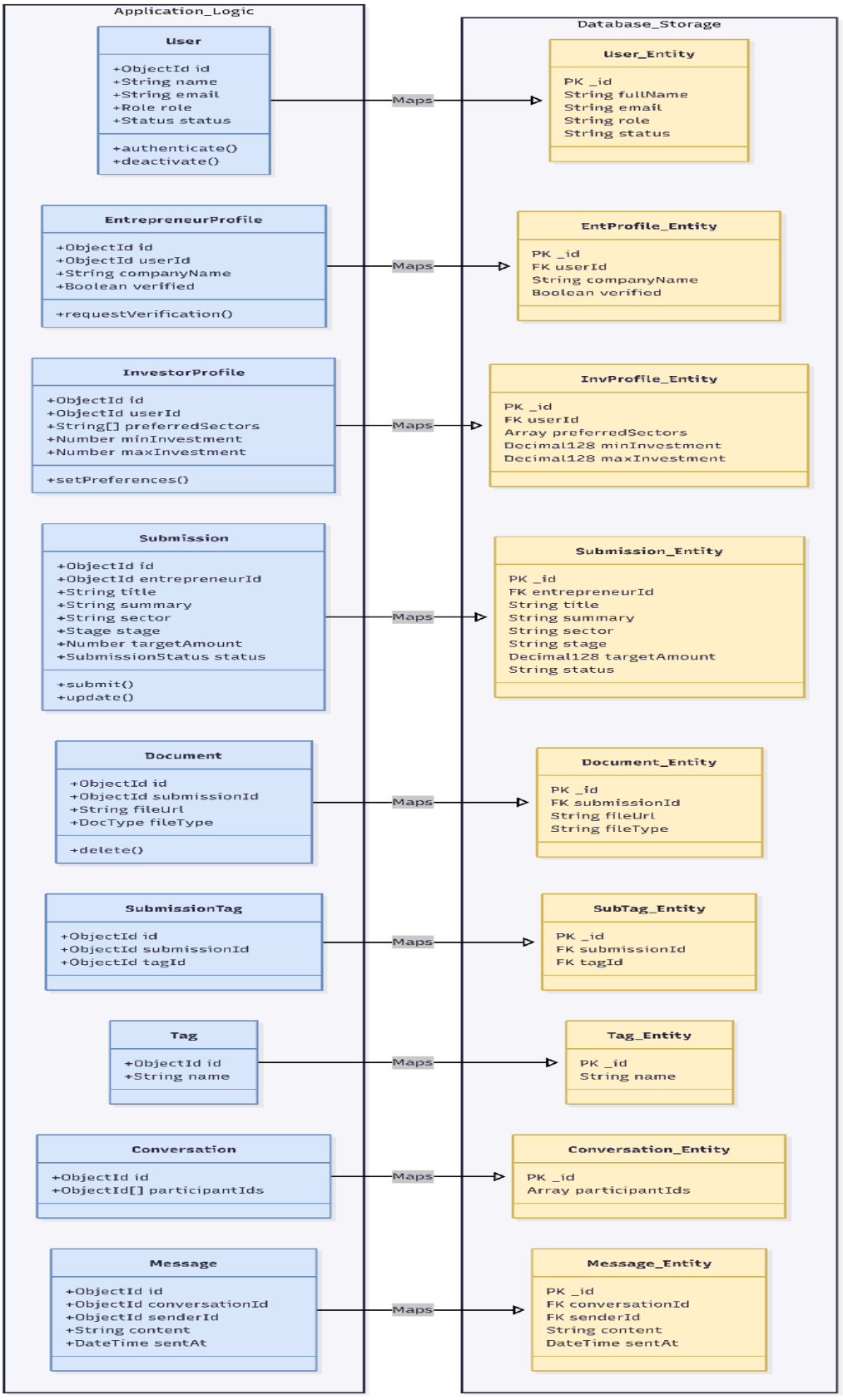


Fig. Mapping of implementation layer and database storage layer

4.9.2 Data collection Descriptions

The following subsection details the specific responsibilities and field definitions for the core collections within the database subsystem.

1) **Identity and profile management**

   This cluster manages user authentication and role specific attributes.

   User: foundational entity linked to the authentication provider. It store the critical role and status, email and firebaseuid

   Entrepreneur Profile: an extension of User entity, it tracks the verification journey specifically storing the companyName and other boolean flags to determine if a user can access pitch creation tools.

   Investor Profile: stores investment preferences such as preferredSectors and maxInvestment. These are critical for recommendation algorithms to filter matches before vector storing.

   Admin Profile: manages system administrators. It includes department, permissionLevel fields to enforce internal security.

2) **Submission and content management**

   Submission: represents the core business pitch. It includes fields like title, summary, description, sector fields. And a logic status field to determine if a pitch is visible to investors.

   Document: stores metadata for uploaded documents. This collection only stores the fileurl and filetype to maintain database performance.

3) **AI and vector Integration**

   These collections bridge the gap between the operational database and AI services subsystem.

   Embedding Entry: stores the reference to the semantic data. It contains the vectorKey and model version ensuring the system.

4) **Simulated funding**

Milestone defines the specific deliverables linked to amount. And ledgerEntry is an immutable log of transactions.

4.10 Access Control and Security Design

This section describes the access control and security architecture of the Smart Entrepreneurial Pitching and Matching System (SEPMS). The system is designed to protect sensitive personal, business, and intellectual property data while ensuring secure, role-based interactions between entrepreneurs, investors, and administrators.

4.10.1 Authentication Mechanism

The system uses **Firebase Authentication** as a centralized identity provider. Users authenticate directly with Firebase using supported methods such as email/password and federated identity providers.

Stateless Token Exchange:

Upon successful authentication, Firebase issues a **JSON Web Token (JWT)** to the client. This token represents the authenticated session and is used for all subsequent API requests.

API Verification:

Every request to the Express.js backend API must include the JWT in the `Authorization` header. The backend validates the token by:

- Verifying the JWT signature using Firebase public keys
- Confirming the token issuer (`iss`) matches the Firebase project
- Confirming the audience (`aud`) matches the backend API identifier
- Validating token expiration (`exp`) with an allowable clock skew of up to 5 minutes
- Extracting the Firebase user identifier (`sub`, Firebase UID) and attaching it to the request context

Only requests with valid and non-expired tokens are processed further by the API.

4.10.2 Token Lifecycle and Session Management

Firebase ID tokens are **short-lived** (approximately 1 hour), reducing the impact of token leakage.

Refresh Token Handling:
Token refresh is handled entirely by the Firebase client SDK using long-lived refresh tokens stored securely on the client side. Refresh tokens are never transmitted to or stored by the backend API.

Token Revocation:
In cases of account suspension, suspected compromise, or administrative action, Firebase token revocation is triggered. The backend enforces revocation by checking the token issue time against the user's `tokensValidAfterTime` value, ensuring that previously issued tokens cannot be reused.

This approach ensures controlled session duration, rapid invalidation of compromised credentials, and minimal backend session state.

4.10.3 Authorization Policies

Authorization determines which resources an authenticated user can access and what actions they may perform. The system enforces **role-based access control (RBAC)** at the API layer using middleware.

User Roles and Permissions:

- **Guest (Public Access):**
    - Access to landing pages and registration endpoints only
- **Entrepreneur:**
    - Create, read, update, and manage their own submissions and associated documents
    - View AI-generated analysis and feedback related to their submissions
    - Participate in messaging only after investor initiation (anti-spam measure)

- **Investor:**
    - View approved submissions and query the recommendation vector index
    - Initiate conversations and meeting requests
    - Trigger milestone-based simulated payments

- **Administrator:**
    - Full read access to all collections for oversight and compliance
    - Approve or reject accounts and submissions
    - Override AI decisions when necessary
    - Suspend users and view administrative audit logs

Authorization decisions are based on the intersection of user role, account verification status, and resource ownership.

4.10.4 User Identity Mapping and Data Scoping

Each authenticated user is uniquely identified by their **Firebase UID**. This UID serves as the primary external identity and is mapped one-to-one with internal database records.

Identity Mapping Strategy:

- The `users` collection stores a unique indexed Firebase UID for each user
- All domain entities (submissions, documents, messages, meetings) reference ownership via this UID
- Backend queries automatically scope data access using the UID extracted from the validated JWT

Row-Level Security:
All database queries are constrained to the requesting user's UID unless elevated privileges (administrator role) are present. This prevents unauthorized access to other users' data at the query level.

4.10.5 Secure Media Access Control

Uploaded documents and media files are stored in **Cloudinary** with private access settings.

Access Control Strategy:

- Media files are not publicly accessible by default
- Access is granted through **signed, short-lived URLs** generated server-side
- Signed URLs are issued only after verifying user authentication and authorization

Access Workflow:

1. The client requests access to a document through an authenticated API endpoint
2. The backend verifies resource ownership or role-based permission
3. The backend generates a time-limited signed Cloudinary URL
4. The signed URL is returned to the client for temporary access

This design ensures that sensitive business documents cannot be accessed or shared without authorization.

4.10.6 Data Privacy and Encryption

To protect sensitive personal, financial, and intellectual property data, the system applies multiple layers of encryption.

Data at Rest:

- MongoDB Atlas encryption at rest is enabled for all stored data
- Sensitive fields are additionally protected using **MongoDB Field-Level Encryption**

Key Management:

- Encryption keys are managed using MongoDB's Key Management Service (KMS) integration
- Keys are not embedded in source code or stored in repositories
- Access to encryption keys is restricted to database-level services

Data in Transit:

All communication between clients, backend services, and third-party providers is protected using TLS encryption.

4.10.7 Security Threat Mitigation

The system addresses common security threats through layered defenses:

- **Unauthorized API Access:**
  Mitigated through JWT validation, token expiry enforcement, and RBAC middleware
- **Compromised Accounts:**
  Mitigated through short-lived tokens, token revocation, and administrative suspension workflows
- **Unauthorized Media Access:**
  Mitigated through private storage and signed, time-limited URLs
- **Malicious File Uploads:**
  Mitigated through file type and size validation, asynchronous virus scanning, and OCR confidence checks
- **API Abuse and Spam:**
  Mitigated through rate limiting, request throttling, and role-based interaction constraints

4.10.8 Audit and Compliance Support

Security-relevant actions such as account approvals, AI overrides, suspensions, and administrative interventions are recorded in an **audit log**. This supports accountability, compliance review, and post-incident analysis.

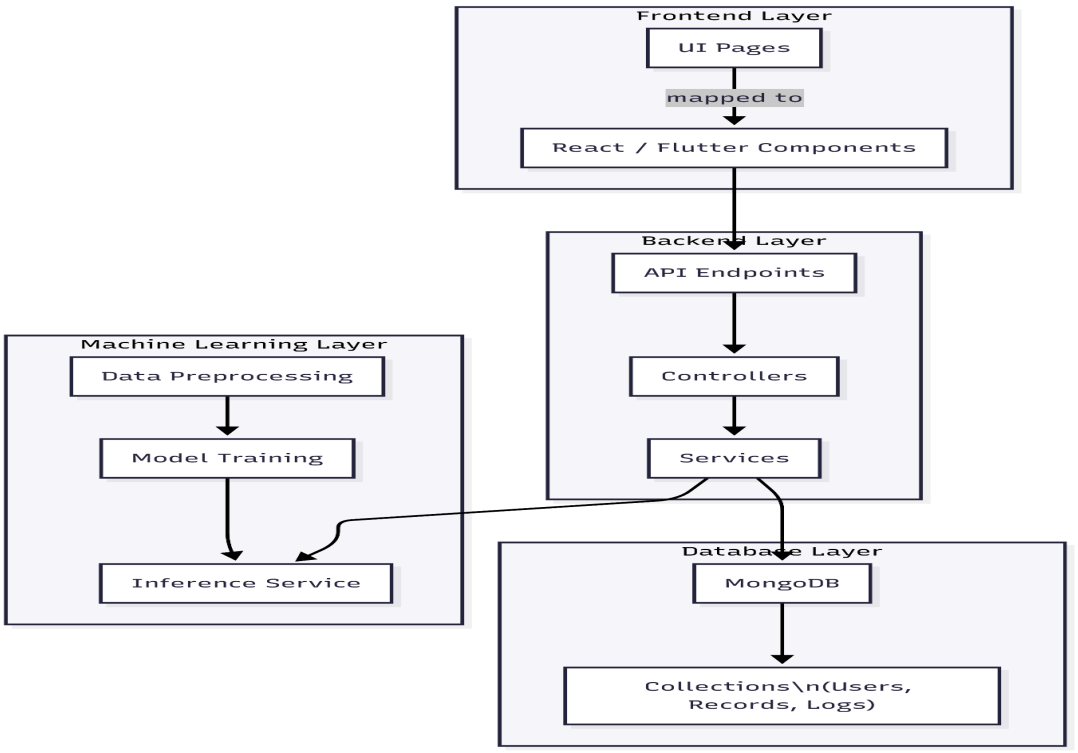
Chapter 5: Implementation

5.1. Overview

This chapter describes the planned implementation of the Smart Entrepreneurial Pitching & Matching System (SEPMS). At this stage, the focus is on outlining how the proposed system design will be translated into a working prototype during the next semester. The implementation phase will follow the system requirements and architectural decisions defined in earlier chapters.

The system implementation is structured around three main layers: the client-side applications, the server-side services, and the machine learning components. Each layer is designed to work independently while communicating through well-defined interfaces. The implementation will adopt consistent coding standards, modern development tools, and modular practices to ensure maintainability and scalability.

Although full implementation is scheduled for the next academic term, this chapter documents the intended development approach, selected technologies, and prototype plan. This ensures that the system can be developed systematically and evaluated against the defined objectives once coding begins.



5.2 Coding Standard

To maintain code quality and consistency across the SEPMS platform, standard coding conventions and best practices will be followed throughout development. These standards aim to improve readability, reduce errors, and support collaborative development among team members.

For frontend development, component-based architecture will be used, with meaningful naming conventions for components, variables, and functions. Files will be organized by feature rather than by type to improve clarity. Reusable UI components will be created to minimize duplication and maintain consistent user experience.

On the backend, RESTful API design principles will be followed. Routes, controllers, and services will be separated clearly, and descriptive naming will be used for endpoints and data models. Input validation and error handling will be applied consistently to prevent invalid data from entering the system.

Documentation comments will be added for critical functions and APIs. Version control practices such as frequent commits, descriptive commit messages, and branch-based development will be used to maintain a clean development history.

5.3 Development Tools

The implementation of SEPMS will rely on a set of modern and widely adopted development tools selected based on ease of use, scalability, and suitability for student projects.

For frontend development, Next.js and React will be used to build responsive web interfaces for entrepreneurs, investors, and administrators. Tailwind CSS and shadcn/ui will be used to design clean and consistent user interfaces without extensive custom styling.

The backend will be implemented using Node.js and Express, which provide flexibility for building RESTful APIs and integrating external AI services. Firebase Authentication will handle secure user authentication and role-based access control.

For data storage, MongoDB Atlas will be used due to its flexibility in handling structured and semi-structured data such as submissions, documents, and user profiles. Media files such as pitch videos and documents will be stored using Cloudinary.

For the Machine learning and AI-related components, Python was selected as the standard industry language for AI. We utilize FastAPI to create a high-performance service that exposes the AI models (like Scikit-learn, T5) to our main application. This ensures we can leverage powerful python-native libraries without compromising the speed of the Node.js backend. GitHub will be used for version control and collaboration, while VS Code will serve as the primary development environment.

5.4 Prototype

The prototype of SEPMS is intended to demonstrate the core system workflows rather than deliver a full production-ready platform. The prototype will evolve incrementally, starting with basic user authentication (login) and progressing toward AI-assisted submission and matching features.

Initial prototype versions will focus on validating design assumptions such as guided submission templates, role-based dashboards, secure login access, and document upload workflows. Feedback from advisors and sample users will be incorporated to refine navigation flow, form structure, and usability.

The prototype will also simulate key processes such as milestone payments and investor recommendations without integrating real financial systems. This approach allows the system logic and user interactions to be tested safely within an academic environment.

Visual artifacts such as interface mockups, UML diagrams, and later screenshots will be used to demonstrate how the system evolves from conceptual design to functional prototype.

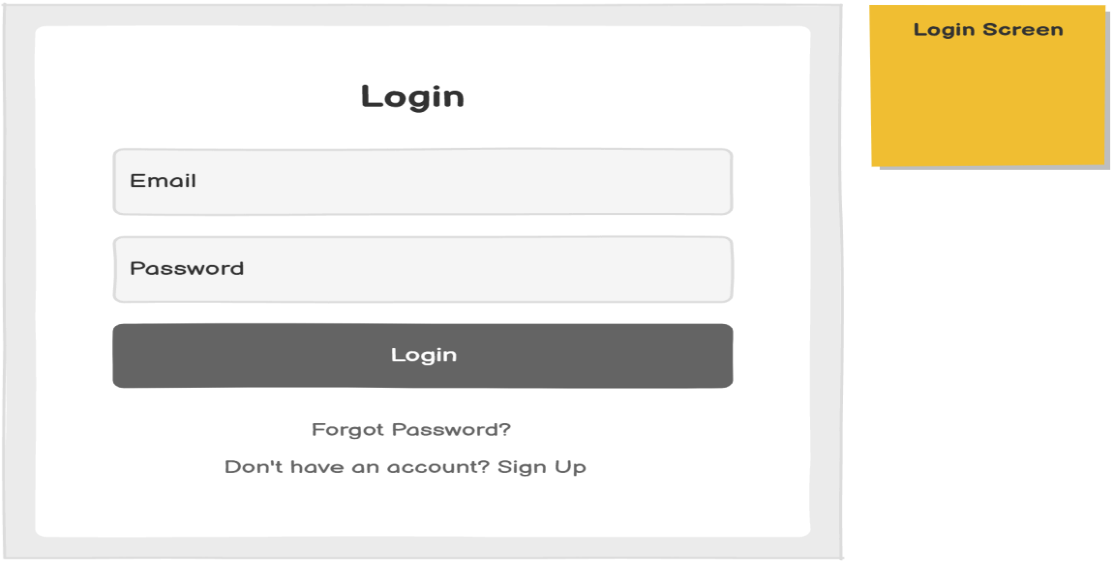


Fig: login **Prototype**

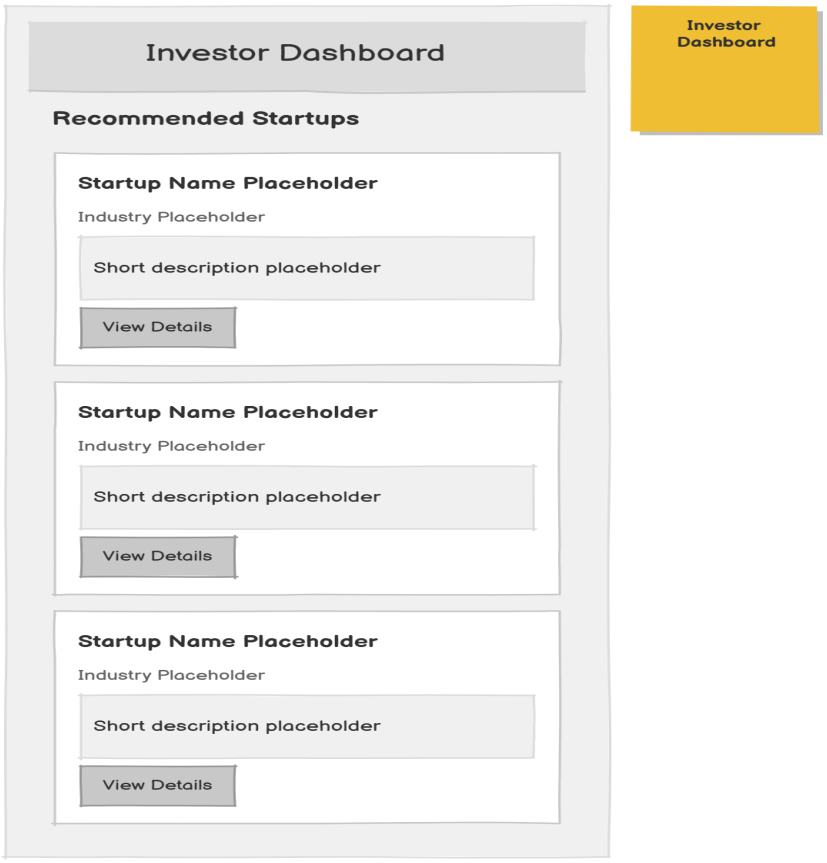


Fig: Investor Dashboard **Prototype**

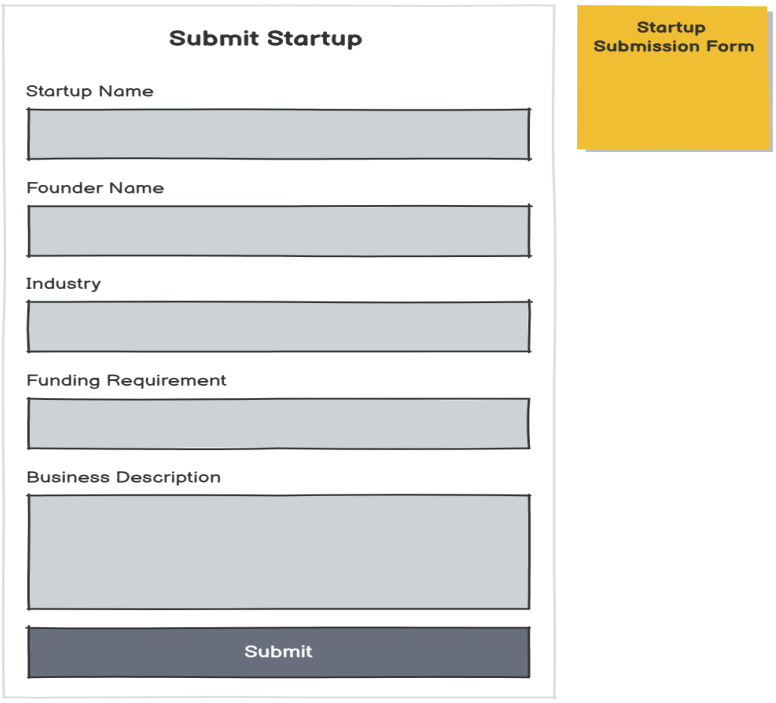


Fig: Startup Dashboard **Prototype**

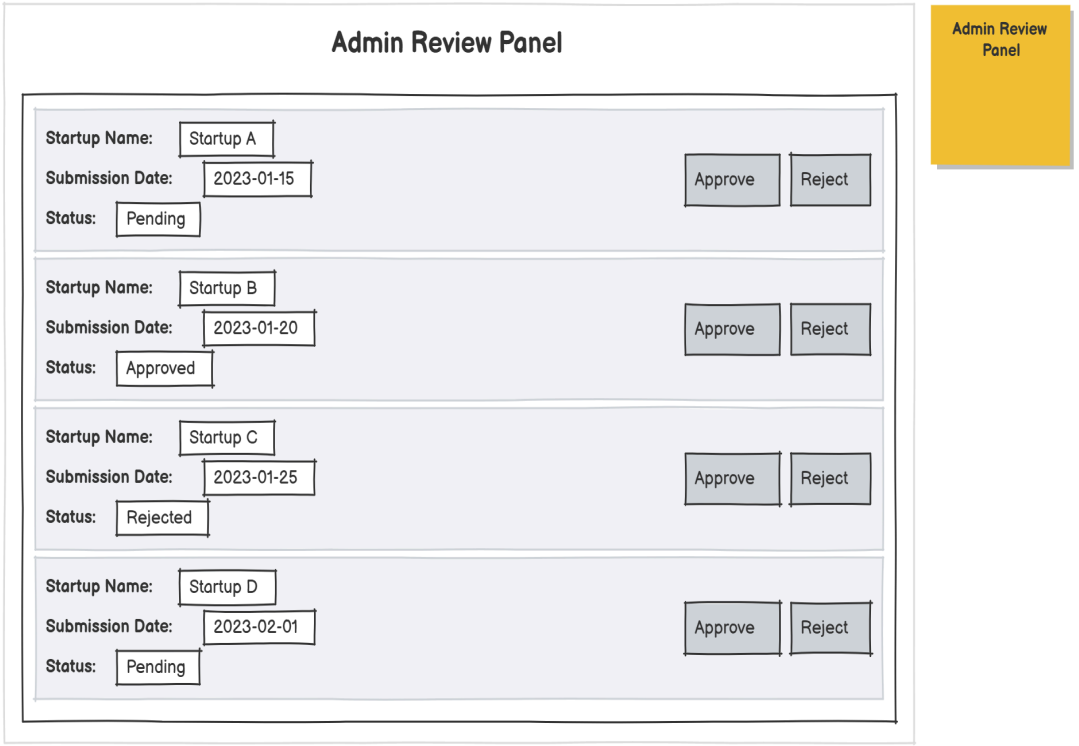


Fig: Admin Review panel **Prototype**

5.5 Implementation Detail

This section outlines the planned technical implementation of the SEPMS platform, divided into client-side, server-side, and machine learning components.

5.5.1 Client-Side

The client-side implementation will consist of a web application and a mobile application. The web interface will be built using Next.js and React, providing role-specific dashboards for entrepreneurs, investors, and administrators.

Entrepreneurs will interact with guided pitch submission forms, document upload interfaces, and submission status tracking pages. Investors will access recommendation lists, project summaries, and communication tools. Administrators will manage verification workflows, flagged submissions, and system oversight.

The mobile application, developed using Flutter, will provide similar functionality with an emphasis on accessibility and on-the-go usage. Client-side validation will be applied to forms to reduce submission errors before data is sent to the server.

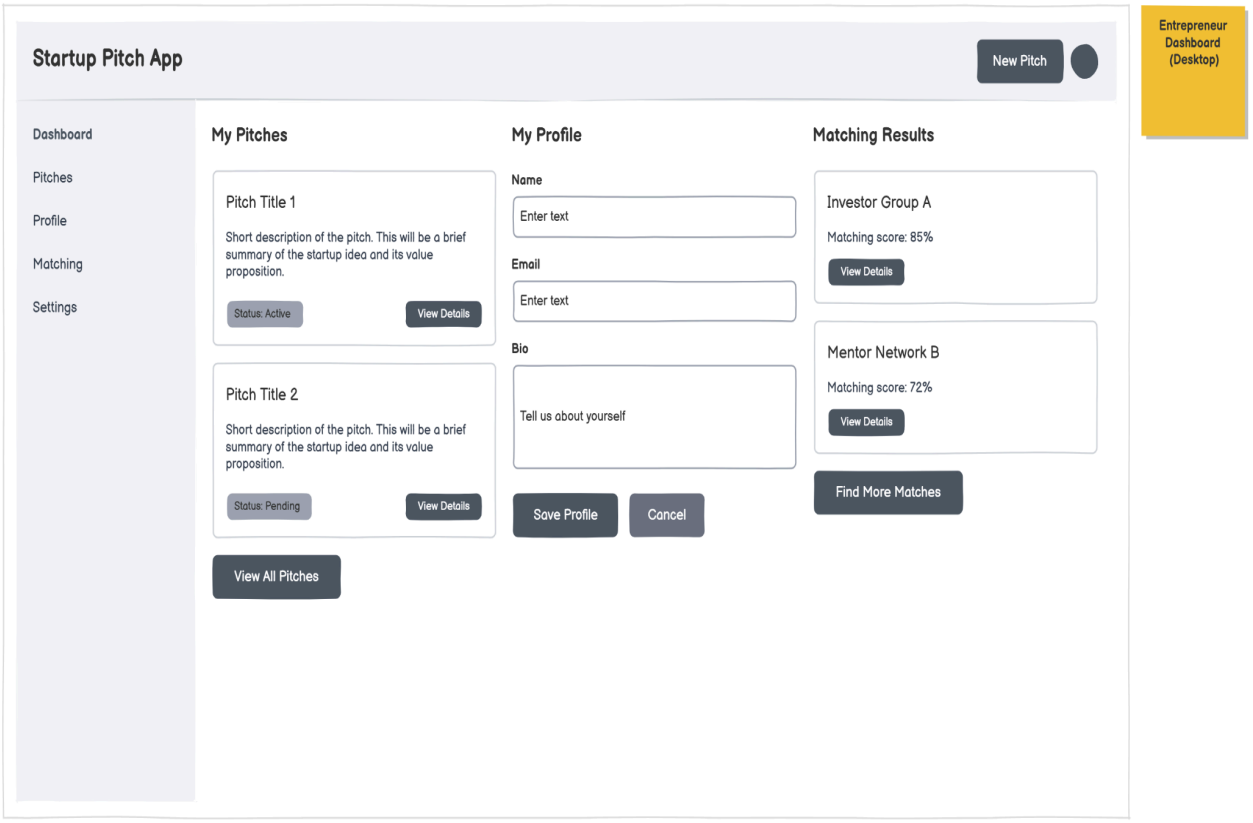


Fig: Entrepreneur Web Dashboard **Prototype**

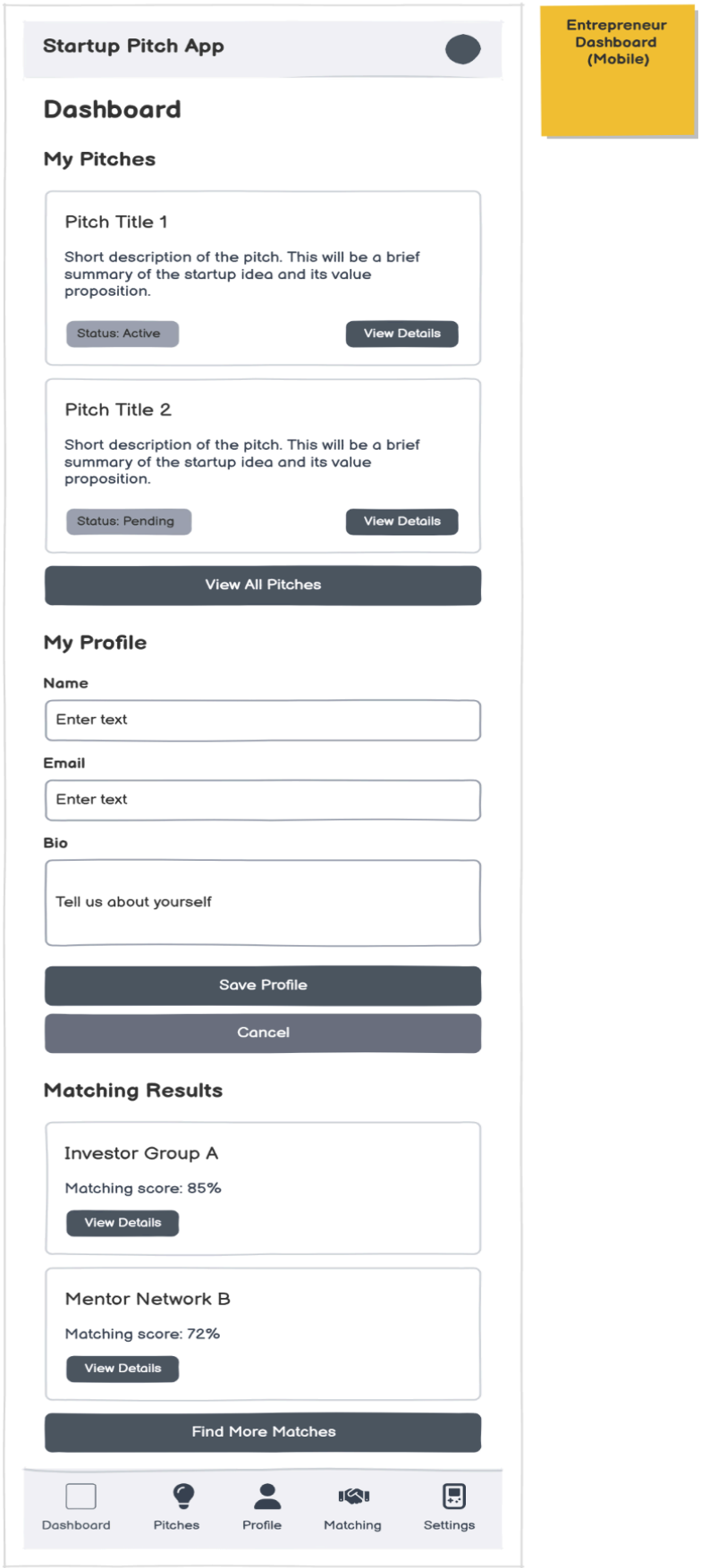


Fig: Entrepreneur Mobile Dashboard **Prototype**

5.5.2 Server-Side

The main server-side implementation will be developed using Node.js and Express which handles client requests, authentication handling, submission management, messaging, and administrative actions and database I/O while it communicates via HTTP requests with the local Python AI service for all machine learning inference tasks.

The server will manage interactions with the MongoDB database, ensuring secure storage and retrieval of user data, submissions, and verification documents. Middleware will be used to enforce authentication, authorization, and request validation.

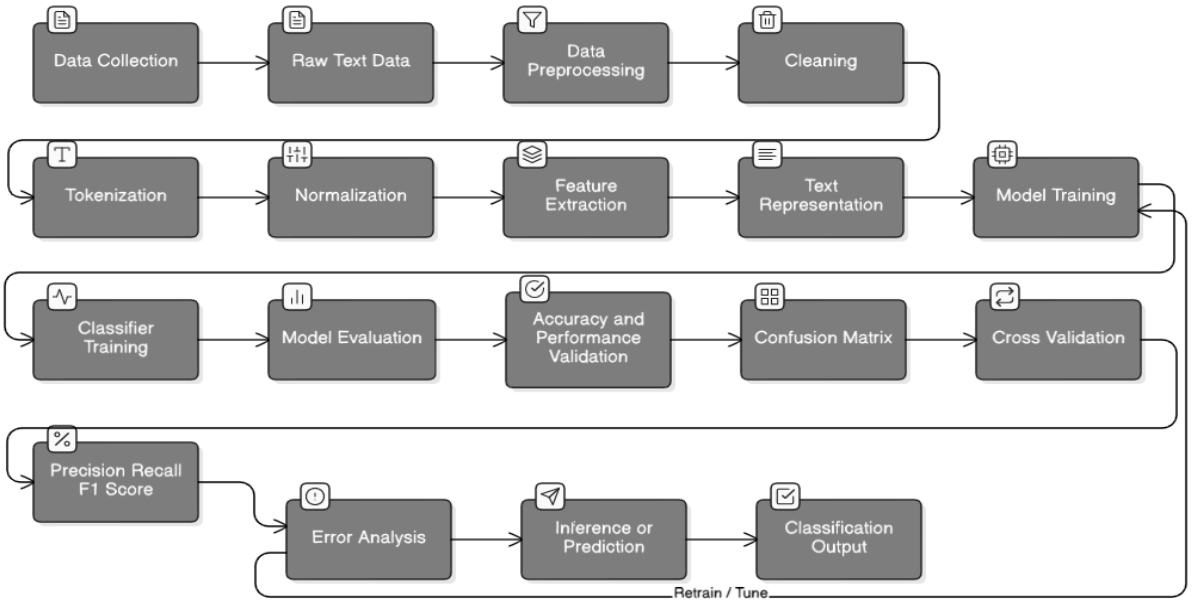
Security measures such as role-based access control, input sanitization, and secure file handling will be applied. The server architecture is designed to be modular so that additional services or features can be integrated in the future.

5.5.3 Machine Learning Model Training

Machine learning components in SEPMS are designed to assist rather than replace human decision-making. The system will use sentence embedding models to represent pitch descriptions and investor preferences in vector form, enabling semantic similarity matching.

A lightweight classifier implemented using scikit-learn will be used to detect low-quality or incomplete submissions. These models will be trained on curated sample data and evaluated using standard metrics such as accuracy and precision.

AI components will be integrated into the backend as services that can be updated or replaced independently. Human administrators will retain final control over approvals, ensuring transparency and accountability in system decisions.


Data Collection
Raw Text Data
Data Preprocessing
Cleaning
Tokenization
Normalization
Feature Extraction
Text Representation
Model Training
Classifier Training
Model Evaluation
Accuracy and Performance Validation
Confusion Matrix
Cross Validation
Precision Recall F1 Score
Error Analysis
Inference or Prediction
Classification Output
Retrain / Tune

Chapter 6: Conclusion and Recommendation

6.1 Conclusion

This Smart Entrepreneurial Pitching and Matching System (SEPMS), an innovative AI-enhanced platform aimed at transforming the entrepreneurial pitching and investment matching process, particularly in emerging markets such as Ethiopia. The proposal successfully addressed the initial objectives by identifying key problems in traditional and existing systems such as inconsistent pitch quality, mismatched investor-entrepreneur interests, fraud risks, and accessibility barriers and proposing a robust solution that incorporates structured submission forms, AI-assisted document verification , intelligent recommendation engines, multilingual support, voice features, and strong administrative oversight.

The project established a solid foundation with clear functional and non-functional requirements, extensive system modeling (including 27 scenarios, use cases, data dictionary for MongoDB collections, class diagrams, sequence diagrams, Fig: Activity diagrams, and state charts), feasibility assessments, and a planned methodology using modern tools like React, Node.js, and machine learning frameworks. The design emphasizes a hybrid AI-human approach to ensure trust, transparency, and inclusivity while mitigating risks.

This proposal demonstrates significant potential to streamline the investment process, reduce information asymmetry, and promote economic growth by empowering entrepreneurs and investors alike. It contributes to the field of AI applications in fintech and entrepreneurship platforms, offering a thoughtful, context-aware blueprint that prioritizes user needs in resource-constrained environments.

6.2 Recommendation

To further develop and enhance the proposed SEPMS, several recommendations are suggested for future work. First, advance the AI components by integrating more sophisticated models, such as computer vision for enhanced document authenticity checks and advanced natural language processing for automated pitch summarization and feedback generation. Second, extend multilingual capabilities to additional local languages

(e.g.,Amharic) and incorporate real-time translation services to improve accessibility across diverse user groups.

Future efforts should focus on implementing the full system, including the mobile application, secure communication channels, and integration with actual payment gateways for milestone tracking (while adhering to regulatory standards). Conducting real-world pilot testing with Ethiopian startups, incubators, and investors would provide valuable data to refine matching algorithms and validate system effectiveness. Additional features, such as blockchain-based intellectual property protection for pitches and analytics dashboards for trend insights, could be explored to increase security and value.

Collaboration with industry stakeholders and further research into ethical AI practices in fraud detection would strengthen the system. These improvements would transform the current proposal into a scalable, impactful platform that significantly advances entrepreneurial ecosystems in developing regions.

References

[1] E. Brynjolfsson and A. McAfee, The Second Machine Age: Work, Progress, and Prosperity in a Time of Brilliant Technologies. New York, NY, USA: W. W. Norton & Company, 2014.

[2] P. Gompers and J. Lerner, "The venture capital revolution," J. Econ. Perspect., vol. 15, no. 2, pp. 145–168, Spring 2001.

[3] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives," IEEE Trans. Pattern Anal. Mach. Intell., vol. 35, no. 8, pp. 1798–1828, Aug. 2013.

[4] S. Rasul, "Web based platform for startups and investors," 2023. [Online]. Available: https://scholar.google.com.

[5] Y.-F. W. Te, "Mitigating discriminatory biases in success prediction models for venture capitals," 2023. [Online]. Available: https://papers.academic-conferences.org. .